

pAML reproducibility analysis

Qun Li

22 Aug 2025 at 10:12:37

Table of Contents

Env	2
Load libraries	2
define functions	9
load data	11
extract different stage patient mutations	14
networkD3 Example	18
sampleCellsNumber	20
mutation features similarity	22
mutation features contribution	25
census genes	27
calMutFraction.....	33
celltype annotation similarity.....	48
sample stage mutation similarity	51
mutation Celltype similarity (scRNA vs scATAC)	55
mutation among celltype similarity merge all stages (scRNA)	56
mutation among celltype similarity merge all stages (scATA)	57
lm between scRNA and scATA mutation celltype similarity.....	59
mutation among celltype similarity by stages (scRNA)	61
stage diff mutations	66
GO for stage diff mutations.....	72
percentage celltype per sample across stages	76
transcription factor lineage analysis	88
transcription factor lineage plots	136
mutation Similarity DXREL for lineage switching	140
sessionInfo	159

Env

```
rm(list = ls())
setwd("/Users/liqun/Desktop/scMutraceReproducibility/Figure5/")
```

Load libraries

```
library(FSA)
## ## FSA v0.10.0. See citation('FSA') if used in publication.
## ## Run fishR() for related website and fishR('IFAR') for related book.
library(ineq)
library(grid)
library(vegan)
## Loading required package: permute
library(ArchR)

## 
##                               / |
##                               /   \
##                               /     \
##                               /       \
##                               /         \
##                               /           \
##                               /             \
##                               /               \
##                               /                 \
##                               /                   \
##                               /                     \
##                               /                       \
##                               /                         \
##                               /                           \
##                               /                             \
##                               /                               \
##                               /                                 \
##                               /                                   \
##                               /                                     \
##                               /                                       \
##                               /                                         \
##                               /                                           \
##                               /                                             \
##                               /                                               \
##                               /                                                 \
##                               /                                                   \
##                               /                                                     \
##                               /                                                       \
##                               /                                                         \
##                               /                                                           \
##                               /                                                             \
##                               /                                                               \
##                               /                                                                 \
##                               /                                                                   \
##                               /                                                                     \
##                               /                                                                       \
##                               /                                                                         \
##                               /                                                                           \
##                               /                                                                             \
##                               /                                                                               \
##                               /                                                                                 \
##                               /                                                                                   \
##                               /                                                                                       \
##                               /                                                                                        \
##                               /                                                                                            \
##                               /                                                                                                \
##                               /                                                                                                    \
##                               /                                                                                                        \
##                               /                                                                                                            \
##                               /                                                                                                                \
##                               /                                                                                                                    \
##                               /                                                                                                                        \
##                               /                                                                                                                            \
##                               /                                                                                                                                              \
##                               /                                                                                                      .
##                               /                                                                                                  .
##                               /                                                                                              .
##                               /                                                                                          .
##                               /                                                                                    .
##                               /                                                                              .
##                               /                                                                        .
##                               /                                                                    .
##                               /                                                              .
##                               /                                                          .
##                               /                                                      .
##                               /                                                  .
##                               /                                              .
##                               /                                          .
##                               /                                      .
##                               /                                .
##                               /                            .
##                               /                        .
##                               /                    .
##                               /                .
##                               /            .
##                               /        .
##                               /    .
##                               /
##                               .

## ArchR : Version 1.0.3
## For more information see our website : www.ArchRProject.com
## If you encounter a bug please report :
https://github.com/GreenleafLab/ArchR/issues
## Loading Required Packages...
## Loading Package : devtools v2.4.5
## Loading Package : grid v4.5.1
## Loading Package : gridExtra v2.3
## Loading Package : gtools v3.9.5
## Loading Package : gtable v0.3.6
## Loading Package : qqplot2 v4.0.0
```

```
## Loading Package : magrittr v2.0.4
## Loading Package : plyr v1.8.9
## Loading Package : stringr v1.5.2
## Loading Package : data.table v1.17.8
## Loading Package : matrixStats v1.5.0
## Loading Package : sparseMatrixStats v1.20.0
## Loading Package : S4Vectors v0.46.0
## Loading Package : GenomicRanges v1.60.0
## Loading Package : BiocGenerics v0.54.0
## Loading Package : Matrix v1.7.4
## Loading Package : Rcpp v1.1.0
## Loading Package : RcppArmadillo v15.0.2.2
## Loading Package : SummarizedExperiment v1.38.1
## Loading Package : rhdf5 v2.52.1
library(dplyr)
##
## Attaching package: 'dplyr'
## The following object is masked from 'package:Biobase':
##
##      combine
## The following objects are masked from 'package:GenomicRanges':
##
##      intersect, setdiff, union
## The following object is masked from 'package:GenomeInfoDb':
##
##      intersect
## The following objects are masked from 'package:IRanges':
##
##      collapse, desc, intersect, setdiff, slice, union
## The following objects are masked from 'package:S4Vectors':
##
##      first, intersect, rename, setdiff, setequal, union
## The following objects are masked from 'package:BiocGenerics':
##
##      combine, intersect, setdiff, setequal, union
## The following object is masked from 'package:generics':
##
##      explain
## The following object is masked from 'package:matrixStats':
##
##      count
## The following objects are masked from 'package:data.table':
##
##      between, first, last
## The following objects are masked from 'package:plyr':
##
##      arrange, count, desc, failwith, id, mutate, rename, summarise,
##      summarize
## The following object is masked from 'package:gridExtra':
##
```

```

##      combine
## The following objects are masked from 'package:stats':
##
##      filter, lag
## The following objects are masked from 'package:base':
##
##      intersect, setdiff, setequal, union
library(GO.db)
## Loading required package: AnnotationDbi
##
## Attaching package: 'AnnotationDbi'
## The following object is masked from 'package:dplyr':
##
##      select
##
library(tidyr)
##
## Attaching package: 'tidyr'
## The following objects are masked from 'package:Matrix':
##
##      expand, pack, unpack
## The following object is masked from 'package:S4Vectors':
##
##      expand
## The following object is masked from 'package:magrittr':
##
##      extract
library(ggpubr)
## Registered S3 methods overwritten by 'car':
##      method      from
##      hist.boot    FSA
##      confint.boot FSA
##
## Attaching package: 'ggpubr'
## The following object is masked from 'package:plyr':
##
##      mutate
library(Seurat)
## Loading required package: SeuratObject
## Loading required package: sp
##
## Attaching package: 'sp'
## The following object is masked from 'package:IRanges':
##
##      %over%
## 'SeuratObject' was built under R 4.5.0 but the current version is
## 4.5.1; it is recommended that you reinstall 'SeuratObject' as the ABI
## for R may have changed
##
## Attaching package: 'SeuratObject'

```

```

## The following object is masked from 'package:SummarizedExperiment':
##
##     Assays
## The following object is masked from 'package:GenomicRanges':
##
##     intersect
## The following object is masked from 'package:GenomeInfoDb':
##
##     intersect
## The following object is masked from 'package:IRanges':
##
##     intersect
## The following object is masked from 'package:S4Vectors':
##
##     intersect
## The following object is masked from 'package:BiocGenerics':
##
##     intersect
## The following objects are masked from 'package:base':
##
##     intersect, t
##
## Attaching package: 'Seurat'
## The following object is masked from 'package:SummarizedExperiment':
##
##     Assays
library(ggpmisc)
## Loading required package: ggpp
## Registered S3 methods overwritten by 'ggpp':
##   method                from
##   heightDetails.titleGrob ggplot2
##   widthDetails.titleGrob  ggplot2
##
## Attaching package: 'ggpp'
## The following objects are masked from 'package:ggpubr':
##
##     as_npc, as_npcx, as_npcy
## The following object is masked from 'package:ggplot2':
##
##     annotate
library(stringr)
library(ggplot2)
library(circlize)
## =====
## circlize version 0.4.16
## CRAN page: https://cran.r-project.org/package=circlize
## Github page: https://github.com/jokergoo/circlize
## Documentation: https://jokergoo.github.io/circlize\_book/book/
##
## If you use it in published research, please cite:

```

```

## Gu, Z. circlize implements and enhances circular visualization
##   in R. Bioinformatics 2014.
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(circlize))
## =====
library(pheatmap)
library(corrplot)
## corrplot 0.95 Loaded
library(bedtoolsr)
## Warning in fun(libname, pkgname): bedtoolsr was built with bedtools
## version 2.30.0 but you have version 2.31.1 installed. Function syntax may
## have changed and wrapper will not function correctly. To fix this, please
## install bedtools version 2.30.0 and either add it to your PATH or run:
## options(bedtools.path = "[bedtools path]")
library(tidyverse)
## — Attaching core tidyverse packages ————— tidyverse
2.0.0 —
## ✓ forcats   1.0.0      ✓ readr       2.1.5
## ✓ lubridate 1.9.4      ✓ tibble      3.3.0
## ✓ purrr     1.1.0
## — Conflicts —————
tidyverse_conflicts() —
## ✗ lubridate::%within%()      masks IRanges::%within%()
## ✗ ggpp::annotate()          masks ggplot2::annotate()
## ✗ dplyr::arrange()           masks plyr::arrange()
## ✗ dplyr::between()           masks data.table::between()
## ✗ dplyr::collapse()          masks IRanges::collapse()
## ✗ dplyr::combine()           masks Biobase::combine(),
BiocGenerics::combine(), gridExtra::combine()
## ✗ purrr::compact()          masks plyr::compact()
## ✗ dplyr::count()             masks matrixStats::count(), plyr::count()
## ✗ dplyr::desc()              masks IRanges::desc(), plyr::desc()
## ✗ tidyr::expand()            masks Matrix::expand(), S4Vectors::expand()
## ✗ tidyr::extract()           masks magrittr::extract()
## ✗ dplyr::failwith()          masks plyr::failwith()
## ✗ dplyr::filter()            masks stats::filter()
## ✗ dplyr::first()             masks S4Vectors::first(), data.table::first()
## ✗ lubridate::hour()          masks data.table::hour()
## ✗ dplyr::id()                masks plyr::id()
## ✗ lubridate::isoweek()       masks data.table::isoweek()
## ✗ dplyr::lag()               masks stats::lag()
## ✗ dplyr::last()              masks data.table::last()
## ✗ lubridate::mday()          masks data.table::mday()
## ✗ lubridate::minute()        masks data.table::minute()
## ✗ lubridate::month()         masks data.table::month()
## ✗ ggpubr::mutate()           masks dplyr::mutate(), plyr::mutate()

```

```

## x tidyr::pack()           masks Matrix::pack()
## x BiocGenerics::Position() masks ggplot2::Position(), base::Position()
## x lubridate::quarter()    masks data.table::quarter()
## x purrr::reduce()         masks GenomicRanges::reduce(),
IRanges::reduce()
## x dplyr::rename()         masks S4Vectors::rename(), plyr::rename()
## x lubridate::second()     masks S4Vectors::second(),
data.table::second()
## x lubridate::second<-()   masks S4Vectors::second<-()
## x AnnotationDbi::select() masks dplyr::select()
## x purrr::set_names()      masks magrittr::set_names()
## x dplyr::slice()          masks IRanges::slice()
## x GenomicRanges::subtract() masks magrittr::subtract()
## x dplyr::summarise()      masks plyr::summarise()
## x dplyr::summarize()      masks plyr::summarize()
## x purrr::transpose()     masks data.table::transpose()
## x tidyr::unpack()         masks Matrix::unpack()
## x lubridate::wday()       masks data.table::wday()
## x lubridate::week()       masks data.table::week()
## x lubridate::yday()       masks data.table::yday()
## x lubridate::year()       masks data.table::year()
## ⓘ Use the conflicted package (<http://conflicted.r-lib.org/>) to force
all conflicts to become errors
library(patchwork)
library(gridExtra)
library(data.table)
library(UpSetR)
library(ggsankey)
library(RColorBrewer)
library(org.Hs.eg.db)
##
library(GenomicRanges)
library(ComplexHeatmap)
## =====
## ComplexHeatmap version 2.24.1
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in
multidimensional
## genomic data. Bioinformatics 2016.
##
##
## The new InteractiveComplexHeatmap package can directly export static

```

```
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(ComplexHeatmap))
## =====
## ! pheatmap() has been masked by ComplexHeatmap::pheatmap(). Most of the arguments
##   in the original pheatmap() are identically supported in the new function. You
##   can still use the original function by explicitly calling pheatmap::pheatmap().
##
##
## Attaching package: 'ComplexHeatmap'
##
## The following object is masked from 'package:pheatmap':
##
##   pheatmap
library(clusterProfiler)
## clusterProfiler v4.16.0 Learn more at https://yulab-smu.top/contribution-knowledge-mining/
##
## Please cite:
##
## G Yu. Thirteen years of clusterProfiler. The Innovation. 2024,
## 5(6):100722
##
## Attaching package: 'clusterProfiler'
##
## The following object is masked from 'package:purrr':
##
##   simplify
##
## The following object is masked from 'package:AnnotationDbi':
##
##   select
##
## The following object is masked from 'package:IRanges':
##
##   slice
##
## The following object is masked from 'package:S4Vectors':
##
##   rename
##
## The following objects are masked from 'package:plyr':
##
##   arrange, mutate, rename, summarise
##
```



```

## The following object is masked from 'package:stats':
##
## filter
library(networkD3)
##
## Attaching package: 'networkD3'
##
## The following object is masked from 'package:Seurat':
##
## JS
##
## The following object is masked from 'package:SeuratObject':
##
## JS
library(htmlwidgets)
##
## Attaching package: 'htmlwidgets'
##
## The following object is masked from 'package:networkD3':
##
## JS
##
## The following object is masked from 'package:Seurat':
##
## JS
##
## The following object is masked from 'package:SeuratObject':
##
## JS

```

define functions

```

cosine_similarity <- function(a, b, na.rm = TRUE) {
  a <- as.numeric(a)
  b <- as.numeric(b)

  if (na.rm) {
    keep <- !(is.na(a) | is.na(b))
    a <- a[keep]
    b <- b[keep]
  }

  denom <- sqrt(sum(a^2)) * sqrt(sum(b^2))
  if (denom == 0) return(as.numeric(NA))

  sim <- sum(a * b) / denom

  sim <- max(min(sim, 1), -1)
}

```

```

    return(as.numeric(sim))
}

retriveCellsByLocation <- function(CSM, locList){
  CSM$CellTag <- rownames(CSM)
  CSM_tmp <- CSM[, c("CellTag",locList)]
  data_MutPerCell_CSM_tmp <- data.frame(
    CellTag = rownames(CSM_tmp),
    MutNum = rowSums(CSM_tmp == "Mut"),
    WTNum = rowSums(CSM_tmp == "WT")
  )
  cells_Total <-
data_MutPerCell_CSM_tmp[rowSums(data_MutPerCell_CSM_tmp[,c("MutNum",
"WTNum")]) > 0, ]$CellTag
  cells_WT <- data_MutPerCell_CSM_tmp[data_MutPerCell_CSM_tmp[, "WTNum"] >
0,]$CellTag
  cells_Mut <- data_MutPerCell_CSM_tmp[data_MutPerCell_CSM_tmp[, "MutNum"] >
0,]$CellTag
  cells <- list(Total = cells_Total, Mut = cells_Mut, WT = cells_WT)
  return(cells)
}

retriveLocsByCells <- function(CSM, cellList){
  CSM$CellTag <- rownames(CSM)
  CSM_tmp <- CSM[CSM$CellTag %in% cellList,]
  data_CellPerMut_CSM_tmp <- data.frame(
    Location = colnames(CSM_tmp),
    CellMutNum = colSums(CSM_tmp == "Mut"),
    CellWTNum = colSums(CSM_tmp == "WT")
  )
  pos_Total <- data_CellPerMut_CSM_tmp[rowSums(data_CellPerMut_CSM_tmp[,
c("CellMutNum", "CellWTNum")]) > 0,]$Location
  pos_Mut <- data_CellPerMut_CSM_tmp[data_CellPerMut_CSM_tmp[, "CellMutNum"]
> 0,]$Location
  pos_WT <- data_CellPerMut_CSM_tmp[data_CellPerMut_CSM_tmp[, "CellWTNum"] >
0,]$Location
  pos <- list(Total = pos_Total, Mut = pos_Mut, WT = pos_WT)
  return(pos)
}

get_mut_cols <- function(df, loc) {
  row <- df[df$Location == loc, ]
  mut_cols <- names(row)[row == "Mut"]
  mut_cols[mut_cols != "Location"]
}

```

load data

```
# Load AML census data
AML_census <- readRDS("../Data/AML/AML_census.rds")
AML_genes_tiers <- read.table("../Data/AML/AML_tiergenes.txt", header = T,
sep = "\t")

# Load SSM data
AML_SSM <- readRDS("../Data/AML/AML_SSM.rds")
AML_SSM_scRNA <- AML_SSM$scRNA
AML_SSM_scATA <- AML_SSM$scATA

# CSM
AML_ATA_CSM_list <- readRDS("../Data/AML/AML_ATA_CSM_list.rds")
AML_RNA_CSM_list <- readRDS("../Data/AML/AML_RNA_CSM_list.rds")

# meta data for celltype annotation similarity
AML_scRNA_meta_list <- readRDS("../Data/AML/AML_scRNA_meta.rds")
AML_scATA_meta_list <- readRDS("../Data/AML/AML_scATA_metaQun.rds")

AML_scRNA_meta <- do.call(rbind, AML_scRNA_meta_list)
rownames(AML_scRNA_meta) <- sub("\\.", "#", rownames(AML_scRNA_meta))

AML_scATA_meta <- do.call(rbind, AML_scATA_meta_list)
AML_scATA_meta$SampleID <- sub("_.*", "", AML_scATA_meta$Sample)

rownames(AML_scATA_meta) <- sub("^AML[0-9]+\\.", "",
rownames(AML_scATA_meta))

cellTypes <- unique(union(AML_scRNA_meta$Classified_Celltype,
AML_scATA_meta$predictedGroup_Un))

# extract mutations and get bed format with census tag
AML_SSM_scRNA_bed <- separate(AML_SSM_scRNA, Location, into = c("chr", "pos",
"ref", "alt"), sep = "_", remove = FALSE)[,1:5]
AML_SSM_scRNA_bed$end <- AML_SSM_scRNA_bed$pos
AML_SSM_scRNA_bed <- AML_SSM_scRNA_bed[,c("chr", "pos", "end", "ref", "alt",
"Location")]
AML_SSM_scRNA_bed$Tag <- ifelse(AML_SSM_scRNA_bed$Location %in%
AML_census$mutation$Location, "Census", "NotCensus")
AML_scRNA_NotCensus_locs <- AML_SSM_scRNA_bed %>% dplyr::filter(Tag ==
"NotCensus") %>% pull(Location)

AML_SSM_scATA_bed <- separate(AML_SSM_scATA, Location, into = c("chr", "pos",
"ref", "alt"), sep = "_", remove = FALSE)[,1:5]
AML_SSM_scATA_bed$end <- AML_SSM_scATA_bed$pos
AML_SSM_scATA_bed <- AML_SSM_scATA_bed[,c("chr", "pos", "end", "ref", "alt",
"Location")]
```



```

## [31] TRUE TRUE TRUE TRUE TRUE TRUE TRUE
col_freq_databases %in% colnames(AML_scRNA_ann)
## [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
TRUE
## [16] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
TRUE
## [31] TRUE TRUE TRUE TRUE TRUE TRUE TRUE

AML_scATA_ann <- AML_scATA_ann %>%
  mutate(across(all_of(col_freq_databases),
    ~ as.numeric(replace(., . == ".", 0))))

AML_scRNA_ann <- AML_scRNA_ann %>%
  mutate(across(all_of(col_freq_databases),
    ~ as.numeric(replace(., . == ".", 0))))

# Annotate bed census and nonCensus
# scATA
AML_SSM_scATA_bed <- AML_SSM_scATA_bed %>%
  left_join(
    AML_scATA_ann %>%
      dplyr::select(Location, all_of(col_freq_databases)),
    by = "Location"
  ) %>%
  mutate(
    freq = if_else(
      if_any(all_of(col_freq_databases), ~ . > 0.0001),
      "high", "rare"
    )
  ) %>%
  dplyr::select(names(AML_SSM_scATA_bed), freq)

# scRNA
AML_SSM_scRNA_bed <- AML_SSM_scRNA_bed %>%
  left_join(
    AML_scRNA_ann %>%
      dplyr::select(Location, all_of(col_freq_databases)),
    by = "Location"
  ) %>%
  mutate(
    freq = if_else(
      if_any(all_of(col_freq_databases), ~ . > 0.0001),
      "high", "rare"
    )
  ) %>%
  dplyr::select(names(AML_SSM_scRNA_bed), freq)

# Load mutation features data
# generate using sigProfiler suite

```

```
AML_mutation_features <- read.table("../Data/AML/Mutational_Matrix_SBS.txt",
header = T, sep = "\t", row.names = 1)
AML_mutation_features_contribution <-
read.table("../Data/AML/COSMIC_SBS96_Activities.txt", header = T, sep = "\t")
```

extract different stage patient mutations

```
AML_patient_ATA_mutations <- list()
AML_patient_RNA_mutations <- list()
Stages <- c("DX", "REM", "REL")
for (stage in Stages){
  colnames_stage <- grep(paste0("_", stage, "$"), colnames(AML_SSM_scATA),
value = TRUE)
  samples_stage_loc <- AML_SSM_scATA %>%
    dplyr::select(Location, colnames_stage) %>%
    dplyr::filter(if_any(all_of(colnames_stage), ~ . == "Mut")) %>%
    dplyr::pull(Location)
  AML_patient_ATA_mutations[[stage]] <- samples_stage_loc
}
## Warning: Using an external vector in selections was deprecated in
tidyselect 1.1.0.
##  Please use `all_of()` or `any_of()` instead.
## # Was:
## data %>% select(colnames_stage)
##
## # Now:
## data %>% select(all_of(colnames_stage))
##
## See <https://tidyselect.r-lib.org/reference/faq-external-vector.html>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

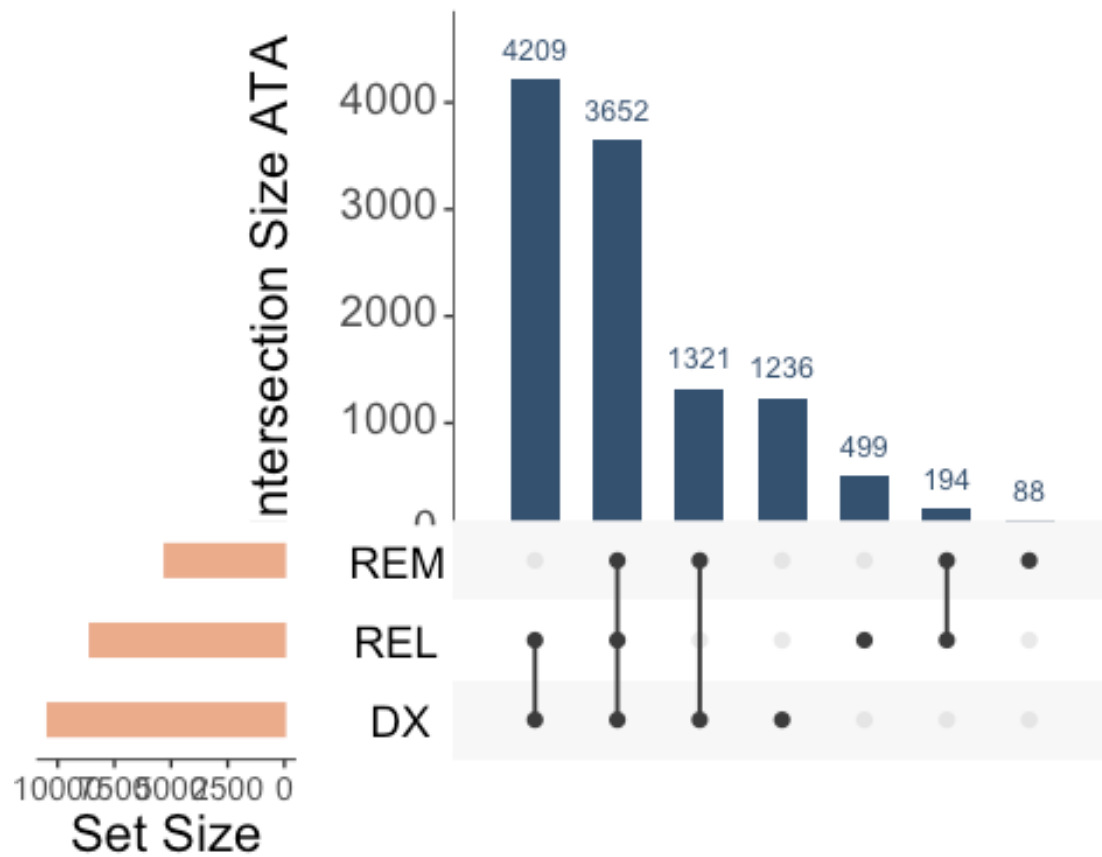
for (stage in Stages){
  colnames_stage <- grep(paste0("_", stage, "$"), colnames(AML_SSM_scRNA),
value = TRUE)
  samples_stage_loc <- AML_SSM_scRNA %>%
    dplyr::select(Location, colnames_stage) %>%
    dplyr::filter(if_any(all_of(colnames_stage), ~ . == "Mut")) %>%
    dplyr::pull(Location)
  AML_patient_RNA_mutations[[stage]] <- samples_stage_loc
}

upset(fromList(AML_patient_ATA_mutations),
      nsets = length(AML_patient_ATA_mutations),
      order.by = "freq",
      mainbar.y.label = "Intersection Size ATA",
      mb.ratio = c(0.6, 0.4),
      main.bar.color = "#345170",
```

```

    sets.bar.color = "#EBAC8C",
    text.scale = c(2, 2, 2, 1.5, 2, 1.5)
)
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## ⓘ Please use tidy evaluation idioms with `aes()`.
## ⓘ See also `vignette("ggplot2-in-packages")` for more information.
## ⓘ The deprecated feature was likely used in the UpSetR package.
## Please report the issue to the authors.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## ⓘ Please use `linewidth` instead.
## ⓘ The deprecated feature was likely used in the UpSetR package.
## Please report the issue to the authors.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
## Warning: The `size` argument of `element_line()` is deprecated as of
ggplot2 3.4.0.
## ⓘ Please use the `linewidth` argument instead.
## ⓘ The deprecated feature was likely used in the UpSetR package.
## Please report the issue to the authors.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

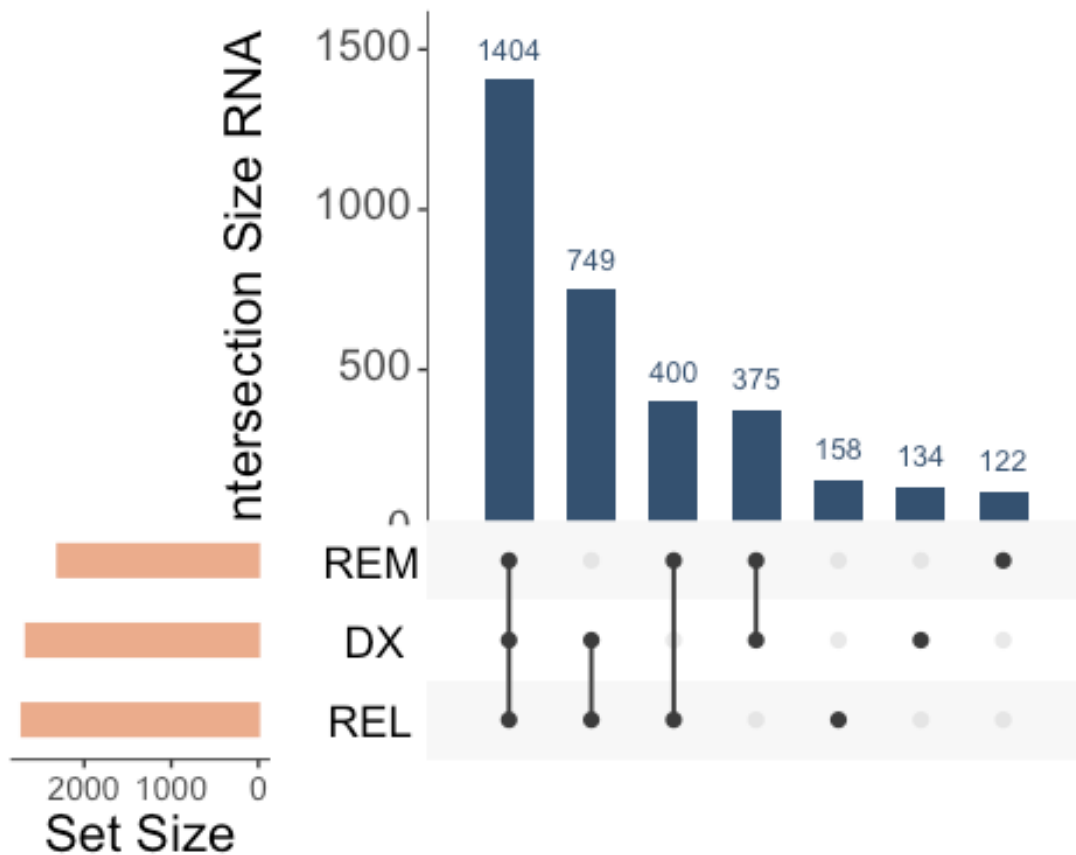
```



```

upset(fromList(AML_patient_RNA_mutations),
      nsets = length(AML_patient_RNA_mutations),
      order.by = "freq",
      mainbar.y.label = "Intersection Size RNA",
      mb.ratio = c(0.6, 0.4),
      main.bar.color = "#345170",
      sets.bar.color = "#EBAC8C",
      text.scale = c(2, 2, 2, 1.5, 2, 1.5)
)

```

```
# export csv
AML_SSM_scATA_DX <- data.frame(Location = AML_patient_ATA_mutations$DX) %>%
  separate(Location, into = c("chr", "pos", "ref", "alt"), sep = "_", remove
= TRUE) %>%
  mutate(end = pos) %>% select(chr, pos, end, ref, alt, everything())

AML_SSM_scATA_REM <- data.frame(Location = AML_patient_ATA_mutations$REM) %>%
  separate(Location, into = c("chr", "pos", "ref", "alt"), sep = "_", remove
= TRUE) %>%
  mutate(end = pos) %>% select(chr, pos, end, ref, alt, everything())

AML_SSM_scATA_REL <- data.frame(Location = AML_patient_ATA_mutations$REL) %>%
  separate(Location, into = c("chr", "pos", "ref", "alt"), sep = "_", remove
= TRUE) %>%
  mutate(end = pos) %>% select(chr, pos, end, ref, alt, everything())

AML_SSM_scATA_all <- data.frame(Location =
unique(unlist(AML_patient_ATA_mutations))) %>%
  separate(Location, into = c("chr", "pos", "ref", "alt"), sep = "_", remove
= TRUE) %>%
  mutate(end = pos) %>% select(chr, pos, end, ref, alt, everything())
```

```

AML_SSM_scrNA_DX <- data.frame(Location = AML_patient_RNA_mutations$DX) %>%
  separate(Location, into = c("chr", "pos", "ref", "alt"), sep = "_", remove
= TRUE) %>%
  mutate(end = pos) %>% select(chr, pos, end, ref, alt, everything())

AML_SSM_scrNA_REM <- data.frame(Location = AML_patient_RNA_mutations$REM) %>%
  separate(Location, into = c("chr", "pos", "ref", "alt"), sep = "_", remove
= TRUE) %>%
  mutate(end = pos) %>% select(chr, pos, end, ref, alt, everything())

AML_SSM_scrNA_REL <- data.frame(Location = AML_patient_RNA_mutations$REL) %>%
  separate(Location, into = c("chr", "pos", "ref", "alt"), sep = "_", remove
= TRUE) %>%
  mutate(end = pos) %>% select(chr, pos, end, ref, alt, everything())

AML_SSM_scrNA_all <- data.frame(Location =
unique(unlist(AML_patient_RNA_mutations))) %>%
  separate(Location, into = c("chr", "pos", "ref", "alt"), sep = "_", remove
= TRUE) %>%
  mutate(end = pos) %>% select(chr, pos, end, ref, alt, everything())

#write.csv(AML_SSM_scATA_DX, "../Results/AML/AML_SSM_scATA_DX.csv", row.names
= FALSE)
#write.csv(AML_SSM_scATA_REM, "../Results/AML/AML_SSM_scATA_REM.csv",
row.names = FALSE)
#write.csv(AML_SSM_scATA_REL, "../Results/AML/AML_SSM_scATA_REL.csv",
row.names = FALSE)
#write.csv(AML_SSM_scrNA_DX, "../Results/AML/AML_SSM_scrNA_DX.csv", row.names
= FALSE)
#write.csv(AML_SSM_scrNA_REM, "../Results/AML/AML_SSM_scrNA_REM.csv",
row.names = FALSE)
#write.csv(AML_SSM_scrNA_REL, "../Results/AML/AML_SSM_scrNA_REL.csv",
row.names = FALSE)
#write.csv(AML_SSM_scATA_all, "../Results/AML/AML_SSM_scATA_all.csv",
row.names = FALSE)
#write.csv(AML_SSM_scrNA_all, "../Results/AML/AML_SSM_scrNA_all.csv",
row.names = FALSE)

```

networkD3 Example

```

# A connection data frame is a list of flows with intensity for each flow
links <- data.frame(
  source=c("Dx", "Rem", "Dx", "Dx", "Rem", "Rel"),
  target=c("Rem", "Rel", "Rel", "Dx", "Rem", "Rel"),
  value=c(1321 + 3652, 194 + 3652, 4209 + 3652, 1236, 88, 499)
)

```

```

links <- data.frame(
  source=c("Dx","Rem", "Dx", "Dx", "Rem", "Rel"),
  target=c("Rem","Rel", "Rel", "Dx", "Rem", "Rel"),
  value=c(375 + 1404, 400 + 1404, 749 + 1404, 134, 122, 158)
)

links <- data.frame(
  source=c("Dx_ATA","Rem_ATA", "Dx_ATA", "Dx_ATA", "Rem_ATA", "Rel_ATA",
"Dx_RNA","Rem_RNA", "Dx_RNA", "Dx_RNA", "Rem_RNA", "Rel_RNA"),
  target=c("Rem_ATA","Rel_ATA", "Rel_ATA", "Dx_ATA", "Rem_ATA", "Rel_ATA",
"Rem_RNA","Rel_RNA", "Rel_RNA", "Dx_RNA", "Rem_RNA", "Rel_RNA"),
  value=c(1321 + 3652, 194 + 3652, 4209 + 3652, 1236, 88, 499, 375 + 1404,
400 + 1404, 749 + 1404, 134, 122, 158)
)

# merge duplicate links
links <- links %>%
  group_by(source, target) %>%
  summarise(value = sum(value), .groups = "drop")

nodes <- data.frame(
  name = unique(c(links$source, links$target))
)

links$IDsource <- match(links$source, nodes$name)-1
links$IDtarget <- match(links$target, nodes$name)-1

# plot
p <- sankeyNetwork(
  Links = links,
  Nodes = nodes,
  Source = "IDsource",
  Target = "IDtarget",
  Value = "value",
  NodeID = "name",
  sinksRight = FALSE
)

p

```



```

      "AML17_DX", "AML17_REL",
      "AML18_DX", "AML18_REL",
      "AML19_DX", "AML19_REL",
      "AML20_DX", "AML20_REM",
      "AML21_DX", "AML21_REM", "AML21_REL",
      "AML22_DX", "AML22_REM", "AML22_REL",
      "AML23_DX", "AML23_REM", "AML23_REL",
      "AML24_DX", "AML24_REM", "AML24_REL",
      "AML25_DX", "AML25_REM", "AML25_REL",
      "AML26_DX", "AML26_REM", "AML26_REL",
      "AML27_DX", "AML27_REM", "AML27_REL",
      "AML28_REM", "AML28_REL")

```

```

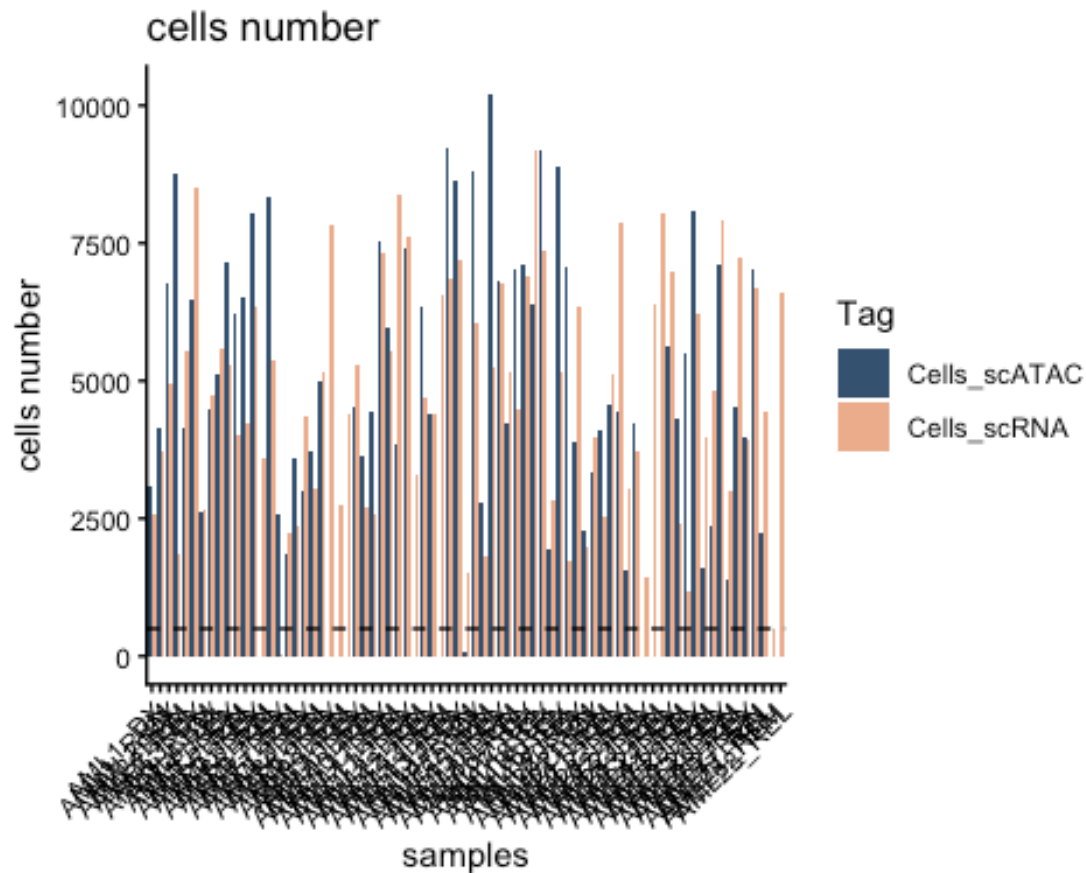
AML_sampleInfo_plot <- AML_sampleInfo[, c("sampleAlias", "Cells_scRNA",
"Cells_scATAC")] %>%
  pivot_longer(
    cols = starts_with("Cells"),
    names_to = "Tag",
    values_to = "CellsNum"
  )

AML_sampleInfo_plot$sampleAlias <- factor(AML_sampleInfo_plot$sampleAlias,
levels = sample_order)

p_sampleCellsNumber <- ggplot(AML_sampleInfo_plot, aes(x = sampleAlias, y =
CellsNum, fill = Tag)) +
  geom_bar(stat = "identity", position = "dodge") +
  geom_hline(yintercept = 500, linetype = "dashed", color = "black") +
  scale_fill_manual(values = c("Cells_scRNA" = "#EBAC8C", "Cells_scATAC" =
"#345170")) +
  labs(title = "cells number", x = "samples", y = "cells number") +
  theme_classic() + theme(axis.text.x = element_text(angle = 45, hjust = 1))

p_sampleCellsNumber

```



```
#ggsave("../Figures/AML/p_sampleCellsNumber.pdf", p_sampleCellsNumber, width
= 14, height = 4)
```

mutation features similarity

```
comparisionTypes <- colnames(AML_mutation_features)

data_cosine_similarity <- as.data.frame(matrix(data = 0, nrow =
length(comparisionTypes), ncol = length(comparisionTypes)))

rownames(data_cosine_similarity) <- comparisionTypes
colnames(data_cosine_similarity) <- comparisionTypes

for (i in 1:nrow(data_cosine_similarity)){
  type_i <- rownames(data_cosine_similarity)[i]
  for (j in 1:ncol(data_cosine_similarity)){
    type_j <- colnames(data_cosine_similarity)[j]
    cs <- cosine_similarity(AML_mutation_features[,type_i],
AML_mutation_features[,type_j])
    data_cosine_similarity[i, j] <- cs
  }
}
```

```

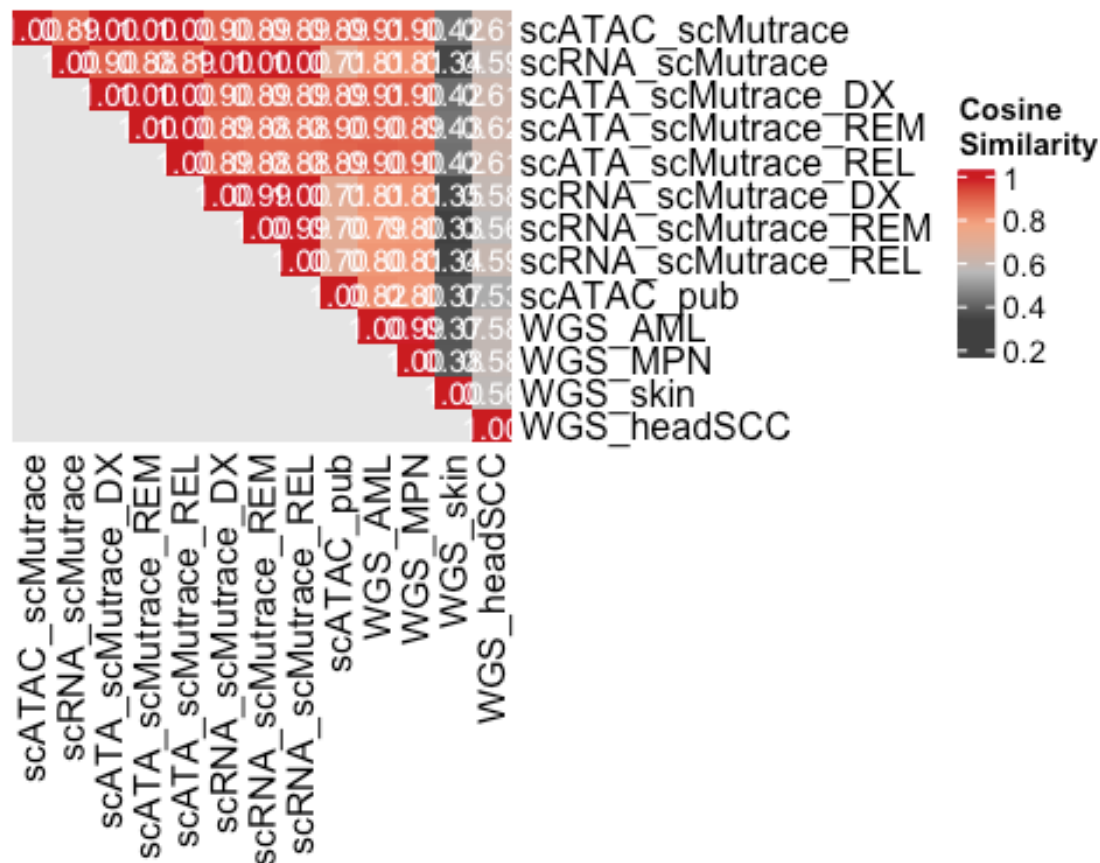
mat <- as.matrix(data_cosine_similarity)
rownames(mat) <- rownames(data_cosine_similarity)
colnames(mat) <- colnames(data_cosine_similarity)
names_order <- c("scATAC_scMutrace", "scRNA_scMutrace", "scATA_scMutrace_DX",
"scATA_scMutrace_REM",
"scATA_scMutrace_REL", "scRNA_scMutrace_DX",
"scRNA_scMutrace_REM", "scRNA_scMutrace_REL",
"scATAC_pub", "WGS_AML", "WGS_MPN", "WGS_skin",
"WGS_headSCC")
mat <- mat[names_order, names_order]
mat_upper <- mat
mat_upper[lower.tri(mat_upper)] <- NA

mat_min <- min(mat, na.rm = TRUE)
mat_max <- max(mat, na.rm = TRUE)

col_fun_2 <- colorRamp2(
  breaks = seq(mat_min, mat_max, length.out = 4),
  colors = c("#404040", "#bababa", "#f4a582", "#ca0020")
)

Heatmap(mat_upper,
  name = "Cosine\nSimilarity",
  na_col = "grey90",
  col = col_fun_2,
  cluster_rows = FALSE,
  cluster_columns = FALSE,
  cell_fun = function(j, i, x, y, width, height, fill) {
    if (!is.na(mat_upper[i, j])) {
      grid.text(sprintf("%.2f", mat_upper[i, j]), x, y,
        gp = gpar(fontsize = 10, col = "white"))
    }
  })

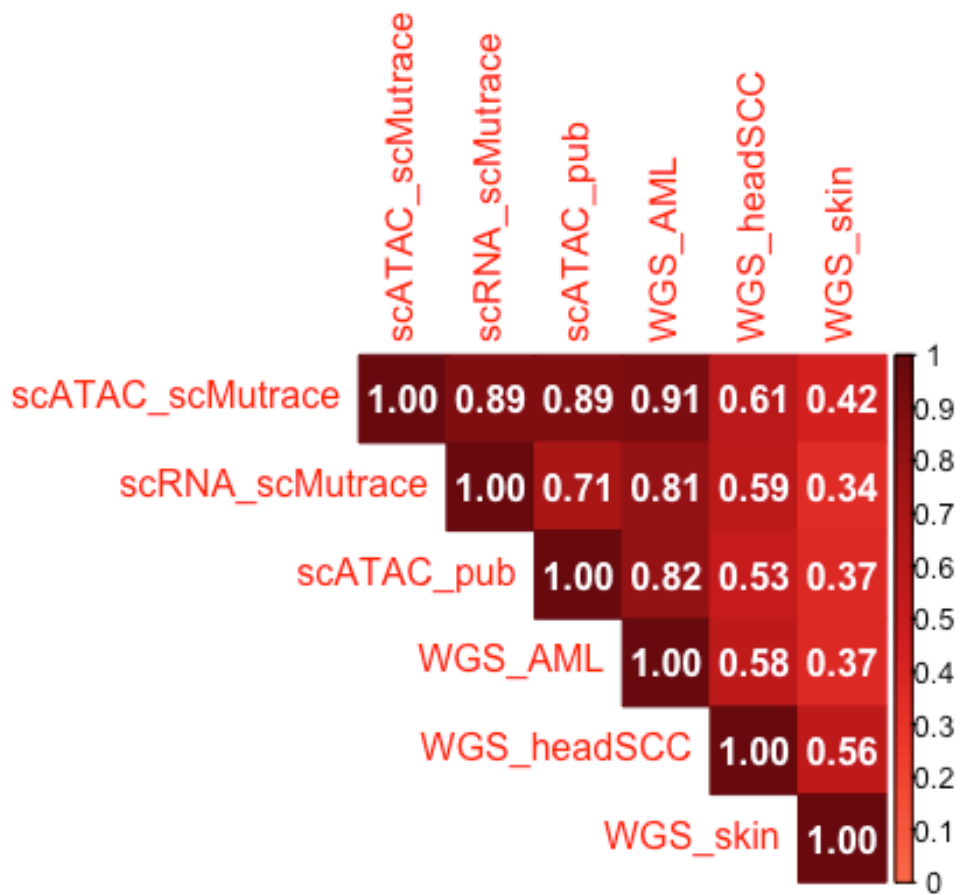
```



```
names_order_show <- c("scATAC_scMutrace", "scRNA_scMutrace",
                      "scATAC_pub", "WGS_AML", "WGS_headSCC", "WGS_skin")

mat_show <- mat[names_order_show, names_order_show]

corrplot(
  mat_show,
  order = "original",
  method = 'color',
  type = 'upper',
  col = colorRampPalette(brewer.pal(11, "Reds"))(100),
  col.lim = c(0, 1),
  addCoef.col = "white"
)
## Warning in brewer.pal(11, "Reds"): n too large, allowed maximum for
## palette Reds is 9
## Returning the palette you asked for with that many colors
```

mutation features contribution

```
sbs_color <- c(
  "SBS1" = "#a6cee3", "SBS2" = "#33a02c", "SBS5" = "#1f78b4",
  "SBS7a" = "#e31a1c", "SBS7b" = "#fc8d59", "SBS7c" = "#fdd49e",
  "SBS13" = "#252525", "SBS15" = "#bdbdbd", "SBS15" = "#d9d9d9",
  "SBS18" = "#f0f0f0", "SBS19" = "#969696", "SBS23" = "#969696",
  "SBS31" = "#737373",
  "SBS39" = "#525252", "SBS54" = "#252525", "SBS87" = "purple"
)

AML_mutation_features_contribution <-
read.table("../Data/AML/COSMIC_SBS96_Activities.txt", header = T, sep = "\t")
AML_mutation_features_contribution_long <- AML_mutation_features_contribution
%>%
  pivot_longer(
    cols = -Samples,
    names_to = "Signature",
    values_to = "Count"
  )

AML_mutation_features_contribution_long_pct <-
```

```

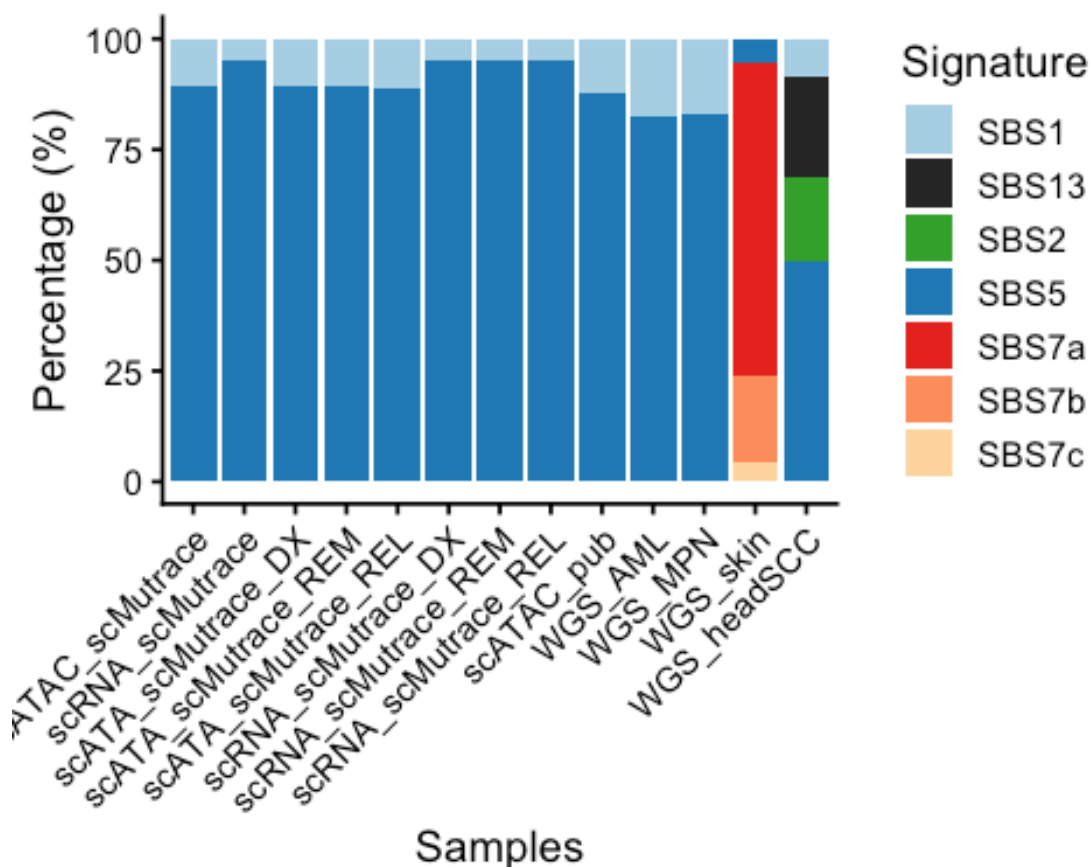
AML_mutation_features_contribution_long %>%
  group_by(Samples) %>%
  mutate(Percent = Count / sum(Count) * 100)

AML_mutation_features_contribution_long_pct$Samples <-
factor(AML_mutation_features_contribution_long_pct$Samples, levels =
c("scATAC_scMutrace", "scRNA_scMutrace", "scATA_scMutrace_DX",
"scATA_scMutrace_REM",
"scATA_scMutrace_REL", "scRNA_scMutrace_DX",
"scRNA_scMutrace_REM", "scRNA_scMutrace_REL",
"scATAC_pub", "WGS_AML", "WGS_MPN", "WGS_skin",
"WGS_headSCC"))

p_mutation_featureDistribution <-
ggplot(AML_mutation_features_contribution_long_pct, aes(x = Samples, y =
Percent, fill = Signature)) +
  geom_bar(stat = "identity") +
  scale_fill_manual(values = sbs_color) +
  theme_classic(base_size = 14) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  ylab("Percentage (%)")

p_mutation_featureDistribution

```



```
#ggsave("../Figures/AML/p_mutation_featureDistribution.pdf",  
p_mutation_featureDistribution, width = 6.98, height = 5.73)
```

census genes

```
scATA_rare_locs <- AML_SSM_scATA_bed %>% filter(freq == "rare") %>%  
pull(Location)  
scrNA_rare_locs <- AML_SSM_scrNA_bed %>% filter(freq == "rare") %>%  
pull(Location)  
  
AML_scATA_ann_tier <- AML_scATA_ann[AML_scATA_ann$Gene.refGene %in%  
AML_genes_tiers$Gene,] %>% filter(Location %in% scATA_rare_locs)  
AML_scrNA_ann_tier <- AML_scrNA_ann[AML_scrNA_ann$Gene.refGene %in%  
AML_genes_tiers$Gene,] %>% filter(Location %in% scrNA_rare_locs)  
  
AML_SSM_scATA_ann_tier <- merge(AML_SSM_scATA, AML_scATA_ann_tier, by =  
"Location")  
AML_SSM_scrNA_ann_tier <- merge(AML_SSM_scrNA, AML_scrNA_ann_tier, by =  
"Location")  
  
all_tier_genes <- unique(union(AML_scrNA_ann_tier$Gene.refGene,  
AML_scATA_ann_tier$Gene.refGene))  
  
all_tier_genes_loc_scrNA <- AML_scrNA_ann_tier %>%  
  dplyr::select(Location, Gene.refGene) %>%  
  dplyr::filter(Gene.refGene %in% all_tier_genes)  
  
all_tier_genes_loc_scATA <- AML_scATA_ann_tier %>%  
  dplyr::select(Location, Gene.refGene) %>%  
  dplyr::filter(Gene.refGene %in% all_tier_genes)  
  
# remove samples based on cells number  
sample_selected_scrNA <- c("AML1_DX", "AML1_REM",  
  "AML2_DX", "AML2_REM", "AML2_REL",  
  "AML3_DX", "AML3_REM",  
  "AML4_DX", "AML4_REL",  
  "AML5_DX", "AML5_REM", "AML5_REL",  
  "AML6_DX", "AML6_REM", "AML6_REL",  
  "AML7_REM", "AML7_REL",  
  "AML8_DX", "AML8_REM", "AML8_REL",  
  "AML9_DX", "AML9_REM", "AML9_REL",  
  "AML10_DX", "AML10_REM", "AML10_REL",  
  "AML11_DX", "AML11_REM", "AML11_REL",  
  "AML12_DX", "AML12_REM", "AML12_REL",  
  "AML13_DX", "AML13_REM", "AML13_REL",  
  "AML14_DX", "AML14_REM",  
  "AML15_DX", "AML15_REM", "AML15_REL",
```

```

"AML16_DX", "AML16_REM", "AML16_REL",
"AML17_DX", "AML17_REL",
"AML18_DX", "AML18_REL",
"AML19_DX", "AML19_REL",
"AML20_DX", "AML20_REM",
"AML21_DX", "AML21_REM", "AML21_REL",
"AML22_DX", "AML22_REM", "AML22_REL",
"AML23_DX", "AML23_REM", "AML23_REL",
"AML24_DX", "AML24_REM", "AML24_REL",
"AML25_DX", "AML25_REM", "AML25_REL",
"AML26_DX", "AML26_REM", "AML26_REL",
"AML27_DX", "AML27_REM", "AML27_REL",
"AML28_REM", "AML28_REL")

```

```
sample_selected_scRNA <- paste0(sample_selected_scRNA, "_scRNA")
```

```

sample_selected_scATA <- c("AML1_DX", "AML1_REM",
"AML2_DX", "AML2_REM", "AML2_REL",
"AML3_DX", "AML3_REM",
"AML4_DX", "AML4_REL",
"AML5_DX", "AML5_REM", "AML5_REL",
"AML6_DX", "AML6_REL",
"AML7_DX", "AML7_REM", "AML7_REL",
"AML8_DX", "AML8_REM", "AML8_REL",

"AML10_DX", "AML10_REM", "AML10_REL",
"AML11_DX", "AML11_REM", "AML11_REL",
"AML12_DX", "AML12_REL",
"AML13_DX", "AML13_REL",
"AML14_DX",
"AML15_DX", "AML15_REM", "AML15_REL",
"AML16_DX", "AML16_REM", "AML16_REL",
"AML17_DX", "AML17_REL",
"AML18_DX", "AML18_REL",
"AML19_DX", "AML19_REL",
"AML20_DX", "AML20_REM",
"AML21_DX", "AML21_REM", "AML21_REL",
"AML22_DX", "AML22_REM", "AML22_REL",

"AML24_DX", "AML24_REM", "AML24_REL",
"AML25_DX", "AML25_REM", "AML25_REL",
"AML26_DX", "AML26_REM", "AML26_REL",
"AML27_DX", "AML27_REM", "AML27_REL")

```

census genes group

```

oncogene_RAS <- c("AFF3", "CBFB", "CBL", "CSF3R", "EGFR", "ELF4", "ETS2",
"ETV6", "EZH2", "FLT3", "GATA1", "HRAS", "IDH1",
"IKZF1", "KIT", "KRAS", "MEIS1", "MET", "MLLT1", "MLLT3",
"MLLT4", "MYC", "NPM1", "NRAS", "NUP214",

```

```

        "PLXNB1", "PTPN11", "RUNX1", "RUNX1T1", "SPI1", "SYK",
        "TAF1", "VIM", "WT1")

suppressor <- c("APC", "ARID1A", "ASXL1", "AXIN1", "CBFA2T2", "CBL", "CEBPA",
"CREBBP", "DICER1", "ERF", "ETS2", "ETV6",
        "EZH2", "FAT1", "FLT3", "IDH1", "IKZF1", "KDM5A", "LLGL1",
"MAD1L1", "NF1", "NPM1", "PHF6", "PTEN",
        "PTPN11", "PTPRJ", "PTPRT", "RUNX1", "RYR1", "SETD2", "SPI1",
"SYK", "TET2", "TP53", "TRIM24", "VIM", "WT1")

transcription_factors <- c("AFF3", "ANKRD30A", "BCOR", "CBFB", "CEBPA",
"CREBBP", "ELF1", "ELF4", "EP300", "FOXP2", "GATA1",
        "GATA2", "HDAC9", "HOXA5", "HOXB3", "IKZF1",
"MED23", "MEIS1", "NCOR1", "NONO", "PHF2", "RUNX1",
        "RUNX1T1", "SIN3A", "SRCAP", "TAF1", "TAF1L",
"THRAP3", "TLE2", "TRIM24", "TRPS1", "TRRAP", "VEZF1", "WT1")

epigenetic_modifier <- c("ASXL1", "ASXL2", "ATRX", "CHD4", "CREBBP", "EP300",
"EZH2", "IDH1", "IDH2", "JMJD1C", "KDM5A", "MLLT1", "MLLT10",
        "MLLT3", "MLLT4", "MLLT6", "MYH11", "MYH2", "MYH4",
"SETD2", "SRCAP", "TET2", "TRRAP")

tyrosine_kinase <- c("CNKSR1", "EGFR", "EPHA6", "FLT3", "FLT4", "KIT", "MET",
"PTK6", "ROR1", "SYK", "TYK2")

nuclear_transport <- c("NPM1", "NUP205", "NUP214")

phosphatase <- c("ANKRD28", "APC", "AXIN1", "KIAA1244", "PPM1F", "PPP1R14B",
"PPP1R14C", "PPP1R15A", "PPP1R15B",
        "PPP2R2D", "PPP4R4", "PPP6R3", "PTEN", "PTPN11", "PTPRA",
"PTPRF", "PTPRH", "PTPRJ", "PTPRT", "TPTE")

structural_integrity <- c("BSN", "COL12A1", "CSPG4", "DNAH2", "DNAH9",
"FLNB", "LAMA1", "MACF1", "MAP7", "NCAM1", "PCLO", "RELN", "TRO")

ETS_Family <- c("ELF1", "ELF4", "ERF", "ETS2", "ETV6", "SPI1")

cohesin <- c("RAD21", "SMC3", "STAG2")

splicing <- c("DHX15", "NONO", "SUPT5H", "U2AF1")

growth_factor <- c("CSF3R", "VEGFC")

genes_groups <- c(oncogene_RAS, suppressor, transcription_factors,
epigenetic_modifier,
        tyrosine_kinase, nuclear_transport, phosphatase,
structural_integrity,
        ETS_Family, cohesin, splicing, growth_factor)

```

```

sample_selected_scATA <- paste0(sample_selected_scATA, "_scATA")

AML_sample_mutations <- as.data.frame(matrix(data = 0, nrow =
length(sample_selected_scRNA) + length(sample_selected_scATA), ncol =
length(all_tier_genes)))
colnames(AML_sample_mutations) <- all_tier_genes
rownames(AML_sample_mutations) <- c(sample_selected_scRNA,
sample_selected_scATA)
AML_sample_mutations$id <- rownames(AML_sample_mutations)

AML_sample_mutations <- separate(AML_sample_mutations, col = "id", into =
c("patient", "stage", "type"), sep = "_", remove = TRUE)
AML_sample_mutations$sampleID <- paste(AML_sample_mutations$patient,
AML_sample_mutations$stage, sep = "_")
AML_sample_mutations <- AML_sample_mutations[,c("patient", "stage", "type",
"sampleID", all_tier_genes)]

col_AML_scRNA_ann_tier <- setdiff(colnames(AML_SSM_scRNA_ann_tier),
colnames(AML_SSM_scRNA_ann_tier)[grep("_L",
colnames(AML_SSM_scRNA_ann_tier))])
AML_SSM_scRNA_ann_tier <- AML_SSM_scRNA_ann_tier[,col_AML_scRNA_ann_tier]

# scRNA
for (i in 1:nrow(AML_SSM_scRNA_ann_tier)){
  gene_tmp <- AML_SSM_scRNA_ann_tier[i,]$Gene.refGene
  mut_samples <- colnames(AML_SSM_scRNA_ann_tier)[AML_SSM_scRNA_ann_tier[i,]
== "Mut"]
  for (mut_sample in mut_samples){
    if (paste0(mut_sample, "_scRNA") %in% rownames(AML_sample_mutations)){
      AML_sample_mutations[paste0(mut_sample, "_scRNA"), gene_tmp] <- 1
    }
  }
}

# scATA
for (i in 1:nrow(AML_SSM_scATA_ann_tier)){
  gene_tmp <- AML_SSM_scATA_ann_tier[i,]$Gene.refGene
  mut_samples <- colnames(AML_SSM_scATA_ann_tier)[AML_SSM_scATA_ann_tier[i,]
== "Mut"]
  for (mut_sample in mut_samples){
    if (paste0(mut_sample, "_scATA") %in% rownames(AML_sample_mutations)){
      AML_sample_mutations[paste0(mut_sample, "_scATA"), gene_tmp] <- 1
    }
  }
}

col_fun <- c(

```

```

"0" = "#404040",
"1" = "#ca0020"
)

sample_order_plotMut <- c(
  "AML1_DX_scRNA", "AML1_DX_scATA", "AML1_REM_scRNA", "AML1_REM_scATA",
  "AML2_DX_scRNA", "AML2_DX_scATA", "AML2_REM_scRNA", "AML2_REM_scATA",
  "AML2_REL_scRNA", "AML2_REL_scATA",
  "AML3_DX_scRNA", "AML3_DX_scATA", "AML3_REM_scRNA", "AML3_REM_scATA",
  "AML4_DX_scRNA", "AML4_DX_scATA", "AML4_REL_scRNA", "AML4_REL_scATA",
  "AML5_DX_scRNA", "AML5_DX_scATA", "AML5_REM_scRNA", "AML5_REM_scATA",
  "AML5_REL_scRNA", "AML5_REL_scATA",
  "AML6_DX_scRNA", "AML6_DX_scATA", "AML6_REM_scRNA", "AML6_REL_scRNA",
  "AML6_REL_scATA",
  "AML7_DX_scATA", "AML7_REM_scRNA", "AML7_REM_scATA", "AML7_REL_scRNA",
  "AML7_REL_scATA",
  "AML8_DX_scRNA", "AML8_DX_scATA", "AML8_REM_scRNA", "AML8_REM_scATA",
  "AML8_REL_scRNA", "AML8_REL_scATA",
  "AML9_DX_scRNA", "AML9_REM_scRNA", "AML9_REL_scRNA",
  "AML10_DX_scRNA", "AML10_DX_scATA", "AML10_REM_scRNA", "AML10_REM_scATA",
  "AML10_REL_scRNA", "AML10_REL_scATA",
  "AML11_DX_scRNA", "AML11_DX_scATA", "AML11_REM_scRNA", "AML11_REM_scATA",
  "AML11_REL_scRNA", "AML11_REL_scATA",
  "AML12_DX_scRNA", "AML12_DX_scATA", "AML12_REM_scRNA", "AML12_REL_scRNA",
  "AML12_REL_scATA",
  "AML13_DX_scRNA", "AML13_DX_scATA", "AML13_REM_scRNA", "AML13_REL_scRNA",
  "AML13_REL_scATA",
  "AML14_DX_scRNA", "AML14_DX_scATA", "AML14_REM_scRNA",
  "AML15_DX_scRNA", "AML15_DX_scATA", "AML15_REM_scRNA", "AML15_REM_scATA",
  "AML15_REL_scRNA", "AML15_REL_scATA",
  "AML16_DX_scRNA", "AML16_DX_scATA", "AML16_REM_scRNA", "AML16_REM_scATA",
  "AML16_REL_scRNA", "AML16_REL_scATA",
  "AML17_DX_scRNA", "AML17_DX_scATA", "AML17_REL_scRNA", "AML17_REL_scATA",
  "AML18_DX_scRNA", "AML18_DX_scATA", "AML18_REL_scRNA", "AML18_REL_scATA",
  "AML19_DX_scRNA", "AML19_DX_scATA", "AML19_REL_scRNA", "AML19_REL_scATA",
  "AML20_DX_scRNA", "AML20_DX_scATA", "AML20_REM_scRNA", "AML20_REM_scATA",
  "AML21_DX_scRNA", "AML21_DX_scATA", "AML21_REM_scRNA", "AML21_REM_scATA",
  "AML21_REL_scRNA", "AML21_REL_scATA",
  "AML22_DX_scRNA", "AML22_DX_scATA", "AML22_REM_scRNA", "AML22_REM_scATA",
  "AML22_REL_scRNA", "AML22_REL_scATA",
  "AML23_DX_scRNA", "AML23_REM_scRNA", "AML23_REL_scRNA",
  "AML24_DX_scRNA", "AML24_DX_scATA", "AML24_REM_scRNA", "AML24_REM_scATA",
  "AML24_REL_scRNA", "AML24_REL_scATA",
  "AML25_DX_scRNA", "AML25_DX_scATA", "AML25_REM_scRNA", "AML25_REM_scATA",
  "AML25_REL_scRNA", "AML25_REL_scATA",
  "AML26_DX_scRNA", "AML26_DX_scATA", "AML26_REM_scRNA", "AML26_REM_scATA",
  "AML26_REL_scRNA", "AML26_REL_scATA",
  "AML27_DX_scRNA", "AML27_DX_scATA", "AML27_REM_scRNA", "AML27_REM_scATA",
  "AML27_REL_scRNA", "AML27_REL_scATA",
  "AML28_REM_scRNA", "AML28_REL_scRNA"
)

```

```

)

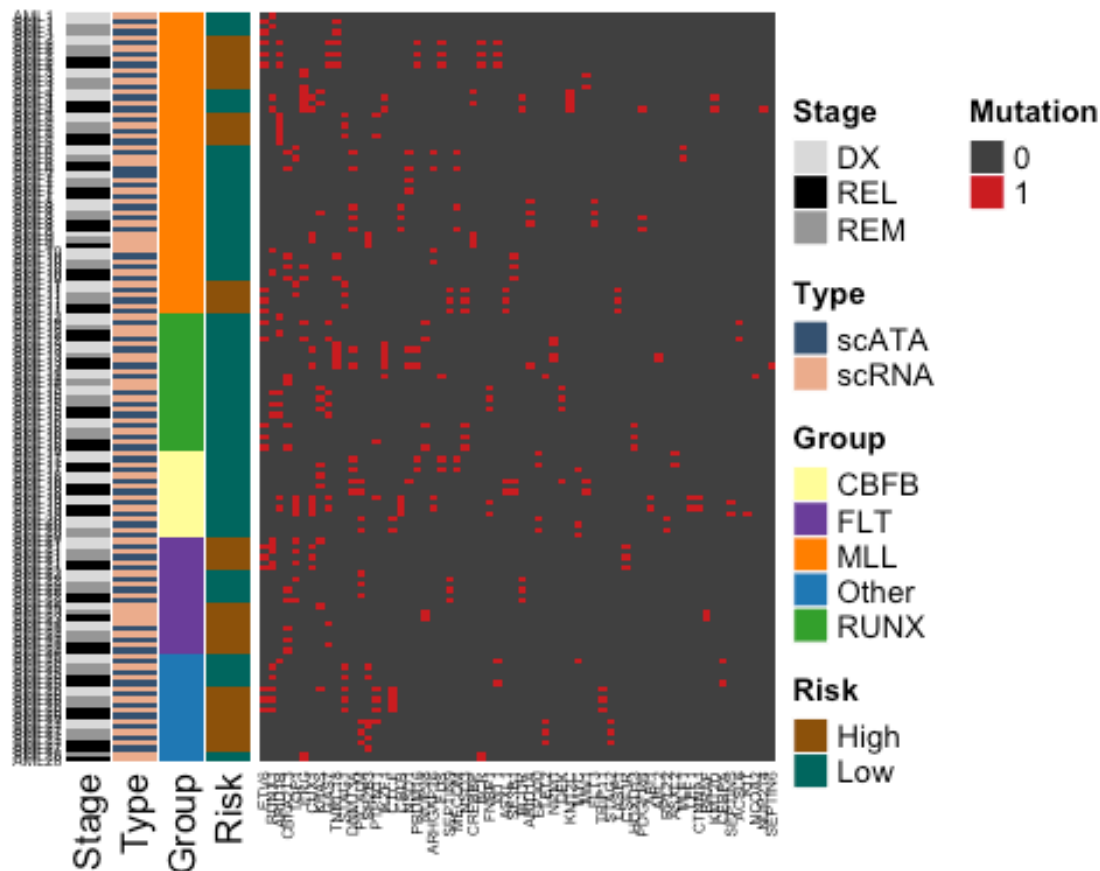
sampleID_p <- sapply(sample_order_plotMut, function(x) strsplit(x,
"_")[[1]][1])
Stages_p <- sapply(sample_order_plotMut, function(x) strsplit(x,
"_")[[1]][2])
Types_p <- sapply(sample_order_plotMut, function(x) strsplit(x, "_")[[1]][3])
group_p <- AML_sampleInfo$SubgroupAnnotation[match(paste(sampleID_p,
Stages_p, sep = "_"), AML_sampleInfo$sampleAlias)]
risk_p <- AML_sampleInfo$RiskGroup[match(paste(sampleID_p, Stages_p, sep =
"_"), AML_sampleInfo$sampleAlias)]

row_anno <- rowAnnotation(
  Sample = anno_text(sampleID_p, gp = gpar(fontsize = 5)),
  Stage = Stages_p,
  Type = Types_p,
  Group = group_p,
  Risk = risk_p,
  col = list(Stage = c("DX" = "#d9d9d9", "REM" = "#969696", "REL" =
"#000000"),
              Risk = c("Low" = "#01665e", "High" = "#8c510a"),
              Type = c("scRNA" = "#EBAC8C", "scATA" = "#345170"),
              Group = c("RUNX" = "#33a02c", "MLL" = "#ff7f00", "FLT" =
"#6a3d9a", "CBFB" = "#ffff99", "Other" = "#1f78b4"))
)

mat <- as.matrix(AML_sample_mutations[sample_order_plotMut,
5:ncol(AML_sample_mutations)])
zero_count <- colSums(mat == 0)
mat_sorted <- mat[, order(zero_count, decreasing = FALSE)]

Heatmap(mat_sorted,
  name = "Mutation",
  left_annotation = row_anno,
  col = col_fun,
  show_row_names = FALSE,
  cluster_rows = FALSE,
  cluster_columns = FALSE,
  column_names_gp = gpar(fontsize = 5))

```

calMutFraction

```
sample_selected_scRNA <- c("AML1_DX", "AML1_REL",
  "AML2_DX", "AML2_REL", "AML2_REL",
  "AML3_DX", "AML3_REL",
  "AML4_DX", "AML4_REL",
  "AML5_DX", "AML5_REL", "AML5_REL",
  "AML6_DX", "AML6_REL", "AML6_REL",
  "AML7_REL", "AML7_REL",
  "AML8_DX", "AML8_REL", "AML8_REL",
  "AML9_DX", "AML9_REL", "AML9_REL",
  "AML10_DX", "AML10_REL", "AML10_REL",
  "AML11_DX", "AML11_REL", "AML11_REL",
  "AML12_DX", "AML12_REL", "AML12_REL",
  "AML13_DX", "AML13_REL", "AML13_REL",
  "AML14_DX", "AML14_REL",
  "AML15_DX", "AML15_REL", "AML15_REL",
  "AML16_DX", "AML16_REL", "AML16_REL",
  "AML17_DX", "AML17_REL",
  "AML18_DX", "AML18_REL",
  "AML19_DX", "AML19_REL",
  "AML20_DX", "AML20_REL",
```

```

"AML21_DX", "AML21_REM", "AML21_REL",
"AML22_DX", "AML22_REM", "AML22_REL",
"AML23_DX", "AML23_REM", "AML23_REL",
"AML24_DX", "AML24_REM", "AML24_REL",
"AML25_DX", "AML25_REM", "AML25_REL",
"AML26_DX", "AML26_REM", "AML26_REL",
"AML27_DX", "AML27_REM", "AML27_REL",
"AML28_REM", "AML28_REL")

```

```

sample_selected_scATA <- c("AML1_DX", "AML1_REM",
"AML2_DX", "AML2_REM", "AML2_REL",
"AML3_DX", "AML3_REM",
"AML4_DX", "AML4_REL",
"AML5_DX", "AML5_REM", "AML5_REL",
"AML6_DX", "AML6_REL",
"AML7_DX", "AML7_REM", "AML7_REL",
"AML8_DX", "AML8_REM", "AML8_REL",

"AML10_DX", "AML10_REM", "AML10_REL",
"AML11_DX", "AML11_REM", "AML11_REL",
"AML12_DX", "AML12_REL",
"AML13_DX", "AML13_REL",
"AML15_DX", "AML15_REM", "AML15_REL",
"AML16_DX", "AML16_REM", "AML16_REL",
"AML17_DX", "AML17_REL",
"AML18_DX", "AML18_REL",
"AML19_DX", "AML19_REL",
"AML20_DX", "AML20_REM",
"AML21_DX", "AML21_REM", "AML21_REL",
"AML22_DX", "AML22_REM", "AML22_REL",
"AML24_DX", "AML24_REM", "AML24_REL",
"AML25_DX", "AML25_REM", "AML25_REL",
"AML26_DX", "AML26_REM", "AML26_REL",
"AML27_DX", "AML27_REM", "AML27_REL")

```

```

num_cutoff <- 0
AML_mutationFraction <- data.frame(
  Location = character(),
  Gene = character(),
  Sample = character(),
  Type = character(),
  DX_Mut_Num = numeric(),
  REM_Mut_Num = numeric(),
  REL_Mut_Num = numeric(),
  DX_Total_Num = numeric(),
  REM_Total_Num = numeric(),
  REL_Total_Num = numeric(),
  DX = numeric(),
  REM = numeric(),

```

```

REL = numeric(),
stringsAsFactors = FALSE
)

loc_total <- unique(c(AML_scATA_NotCensus_locs, AML_scRNA_NotCensus_locs))
line_i <- 0

for (loc in loc_total){
  message(paste0("Processing ", loc))
  # gene
  gene <- NA
  if (loc %in% AML_scATA_ann$Location){
    gene <- AML_scATA_ann %>%
      filter(Location == loc) %>%
      pull(Gene.refGene)
  } else if (loc %in% AML_scRNA_ann$Location){
    gene <- AML_scRNA_ann %>%
      filter(Location == loc) %>%
      pull(Gene.refGene)
  }

  # ----- ATA -----
  samples_ata_loc <- AML_SSM_scATA %>%
    filter(Location == loc) %>%
    select(any_of(sample_selected_scATA)) %>%
    unlist() %>%
    { names(.)[. == "Mut"] }

  if(length(samples_ata_loc) > 0){
    message(paste0(" Processing scATA..."))
    sample_ids <- unique(str_extract(samples_ata_loc, "AML[0-9]+"))
    for (sample_id in sample_ids){
      line_i <- line_i + 1
      cells_all <- retrieveCellsByLocation(AML_ATA_CSM_list[[sample_id]],
loc)
      cells_mut <- cells_all$Mut
      cells_total <- cells_all$Total

      num_mut_dx <- sum(grepl("DX", cells_mut))
      num_mut_rem <- sum(grepl("REM", cells_mut))
      num_mut_rel <- sum(grepl("REL", cells_mut))
      num_total_dx <- sum(grepl("DX", cells_total))
      num_total_rem <- sum(grepl("REM", cells_total))
      num_total_rel <- sum(grepl("REL", cells_total))

      # QC
      if (num_total_dx >= num_cutoff & num_total_rem >= num_cutoff &
num_total_rel >= num_cutoff){

```

```

frac_dx <- ifelse(num_total_dx > 0, num_mut_dx/num_total_dx, 0)
frac_rem <- ifelse(num_total_rem > 0, num_mut_rem/num_total_rem, 0)
frac_rel <- ifelse(num_total_rel > 0, num_mut_rel/num_total_rel, 0)

AML_mutationFraction[line_i, ] <- list(
  Location = loc,
  Gene = gene,
  Sample = sample_id,
  Type = "scATA",
  DX_Mut_Num = num_mut_dx,
  REM_Mut_Num = num_mut_rem,
  REL_Mut_Num = num_mut_rel,
  DX_Total_Num = num_total_dx,
  REM_Total_Num = num_total_rem,
  REL_Total_Num = num_total_rel,
  DX = frac_dx,
  REM = frac_rem,
  REL = frac_rel
)
}
}
}

# ----- RNA -----
samples_rna_loc <- AML_SSM_scRNA %>%
  filter(Location == loc) %>%
  select(any_of(sample_selected_scRNA)) %>%
  unlist() %>%
  { names(.)[. == "Mut"] }

if(length(samples_rna_loc) > 0){
  sample_ids <- unique(str_extract(samples_rna_loc, "AML[0-9]+"))
  message(paste0(" Processing scRNA..."))
  for (sample_id in sample_ids){
    line_i <- line_i + 1
    cells_all <- retrieveCellsByLocation(AML_RNA_CSM_list[[sample_id]],
loc)
    cells_mut <- cells_all$Mut
    cells_total <- cells_all$Total

    num_mut_dx <- sum(grepl("DX", cells_mut))
    num_mut_rem <- sum(grepl("REM", cells_mut))
    num_mut_rel <- sum(grepl("REL", cells_mut))
    num_total_dx <- sum(grepl("DX", cells_total))
    num_total_rem <- sum(grepl("REM", cells_total))
    num_total_rel <- sum(grepl("REL", cells_total))

    # QC
    if (num_total_dx >= num_cutoff & num_total_rem >= num_cutoff &

```

```

num_total_rel >= num_cutoff){

  frac_dx <- ifelse(num_total_dx > 0, num_mut_dx/num_total_dx, 0)
  frac_rem <- ifelse(num_total_rem > 0, num_mut_rem/num_total_rem, 0)
  frac_rel <- ifelse(num_total_rel > 0, num_mut_rel/num_total_rel, 0)

  AML_mutationFraction[line_i, ] <- list(
    Location = loc,
    Gene = gene,
    Sample = sample_id,
    Type = "scRNA",
    DX_Mut_Num = num_mut_dx,
    REM_Mut_Num = num_mut_rem,
    REL_Mut_Num = num_mut_rel,
    DX_Total_Num = num_total_dx,
    REM_Total_Num = num_total_rem,
    REL_Total_Num = num_total_rel,
    DX = frac_dx,
    REM = frac_rem,
    REL = frac_rel
  )
}
}
}
}

## Processing 10_100057605_G_A
## Processing scATA...
## Processing 10_100184601_G_A
## Processing scATA...
## Processing scRNA...
## Processing 10_100187183_T_C
## Processing scATA...
## Processing 10_100228684_A_G

AML_mutationFraction <- na.omit(AML_mutationFraction)

stage_color_setting <- c(
  "DX" = "#4DAF4A",
  "REM" = "#377EB8",
  "REL" = "#E41A1C"
)

common_loc_scATA_scRNA <- unique(intersect(AML_SSM_scATA$Location,
AML_SSM_scRNA$Location))
common_loc_scATA_scRNA_census <- intersect(AML_scATA_ann_tier$Location,
AML_scRNA_ann_tier$Location)

AML_mutationFraction_commonLocs <- AML_mutationFraction %>% filter(Location
%in% common_loc_scATA_scRNA)

```

```

AML_mutationFraction_commonLocs$Tag <-
paste(AML_mutationFraction_commonLocs$Location,
AML_mutationFraction_commonLocs$Sample, sep = "-")

AML_mutationFraction_commonLocs <- AML_mutationFraction_commonLocs %>%
  group_by(Tag) %>%
  filter(n() > 1) %>%
  ungroup()

df_fractionSimilarity_DX_plot <- AML_mutationFraction_commonLocs %>%
  #filter(DX_Num >= 5) %>%
  select(Location, Sample, Type, DX) %>%
  pivot_wider(names_from = Type, values_from = DX) %>%
  drop_na() %>%
  mutate(Stage = "DX")

df_fractionSimilarity_REM_plot <- AML_mutationFraction_commonLocs %>%
  #filter(REM_Num >= 5) %>%
  select(Location, Sample, Type, REM) %>%
  pivot_wider(names_from = Type, values_from = REM) %>%
  drop_na() %>%
  mutate(Stage = "REM")

df_fractionSimilarity_REL_plot <- AML_mutationFraction_commonLocs %>%
  #filter(REL_Num >= 5) %>%
  select(Location, Sample, Type, REL) %>%
  pivot_wider(names_from = Type, values_from = REL) %>%
  drop_na() %>%
  mutate(Stage = "REL")

df_fractionSimilarity_all <- bind_rows(
  df_fractionSimilarity_DX_plot,
  df_fractionSimilarity_REM_plot,
  df_fractionSimilarity_REL_plot
)

df_fractionSimilarity_long <- AML_mutationFraction_commonLocs %>%
  select(Location, Sample, Type, DX, REM, REL) %>%
  pivot_longer(cols = c(DX, REM, REL), names_to = "Stage", values_to =
"Fraction") %>%
  pivot_wider(names_from = Type, values_from = Fraction)

df_fractionSimilarity_long$Stage <- factor(df_fractionSimilarity_long$Stage,
levels = c("DX", "REM", "REL"))

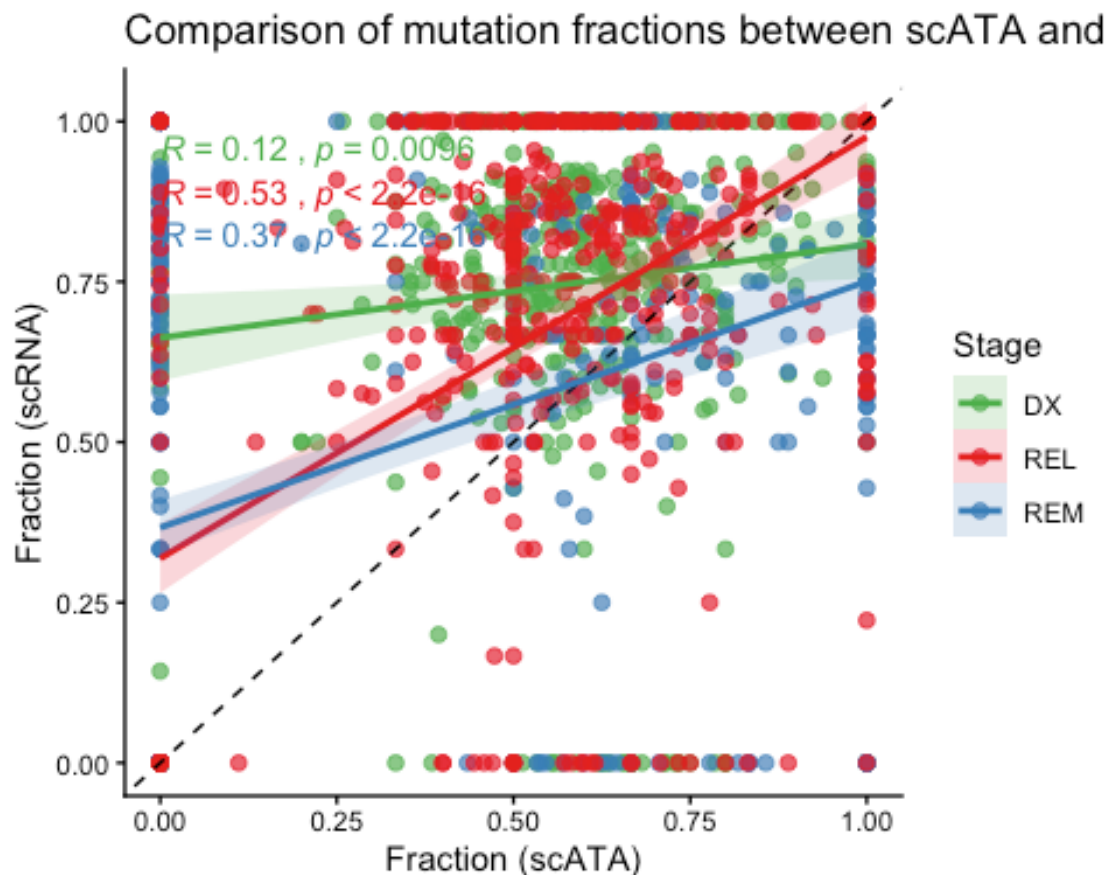
ggplot(df_fractionSimilarity_all, aes(x = scATA, y = scRNA, color = Stage)) +
  geom_point(size = 2, alpha = 0.7) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "black")
+

```

```

geom_smooth(method = "lm", se = TRUE, aes(fill = Stage), alpha = 0.2) +
stat_cor(aes(label = paste(..r.label.., ..p.label.., sep = "~`,`~"), group
= Stage),
  method = "pearson",
  show.legend = FALSE) +
scale_color_manual(values = stage_color_setting) +
scale_fill_manual(values = stage_color_setting) +
labs(
  x = "Fraction (scATA)",
  y = "Fraction (scRNA)",
  title = "Comparison of mutation fractions between scATA and scRNA"
) +
theme_classic()
## Warning: The dot-dot notation (`..r.label..`) was deprecated in ggplot2
3.4.0.
## i Please use `after_stat(r.label)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
## `geom_smooth()` using formula = 'y ~ x'

```



```
df_wide <- df_fractionSimilarity_long %>%
```

```

pivot_wider(
  names_from = Stage,
  values_from = c(scATA, scRNA)
)

```

```

p_scRNA_fractionMutation <- ggplot(df_fractionSimilarity_long, aes(x =
as.factor(Stage), y = scRNA, color=Stage)) +
  labs(x="", y="Fraction of mutant cells") +
  geom_boxplot(width=0.5, lwd = 1, outliers = FALSE) +
  geom_line(aes(group=Location), color="gray", position =
position_dodge(0.2), alpha = 0.1) +
  geom_point(aes(fill=Stage, group=Location), size = 0.1, position =
position_dodge(0.2)) +
  scale_color_manual(values = stage_color_setting) + theme_classic() +
  stat_compare_means(
    comparisons = list(
      c("DX", "REL"),
      c("DX", "REM"),
      c("REL", "REM")
    ),
    method = "wilcox.test",
    label = "p.signif"
  )

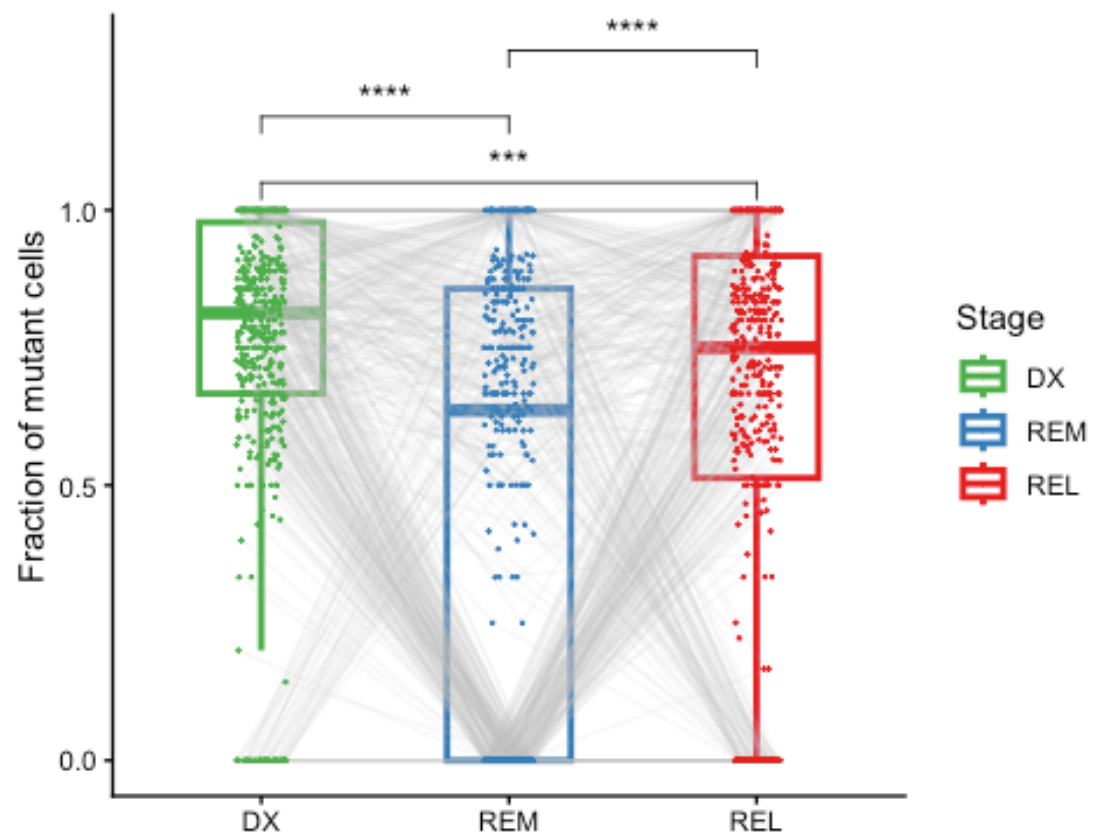
```

```

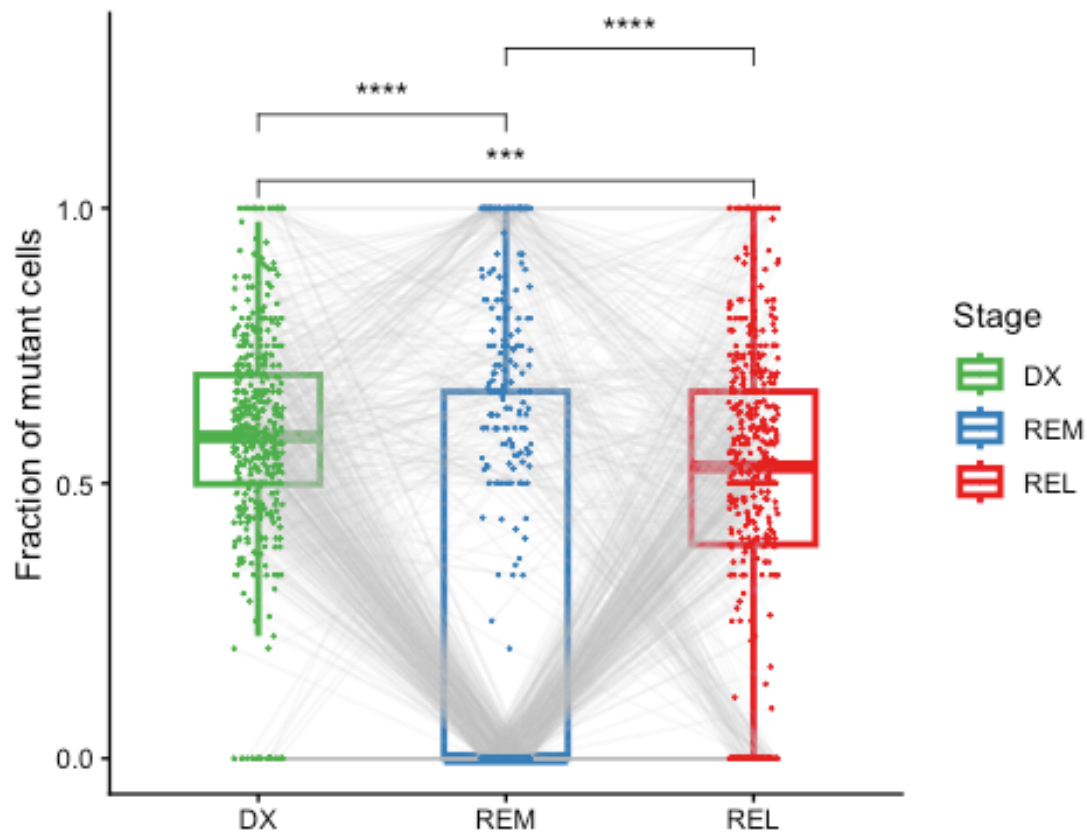
p_scATA_fractionMutation <- ggplot(df_fractionSimilarity_long, aes(x =
as.factor(Stage), y = scATA, color=Stage)) +
  labs(x="", y="Fraction of mutant cells") +
  geom_boxplot(width=0.5, lwd = 1, outliers = FALSE) +
  geom_line(aes(group=Location), color="gray", position =
position_dodge(0.2), alpha = 0.1) +
  geom_point(aes(fill=Stage, group=Location), size = 0.1, position =
position_dodge(0.2)) +
  scale_color_manual(values = stage_color_setting) + theme_classic() +
  stat_compare_means(
    comparisons = list(
      c("DX", "REL"),
      c("DX", "REM"),
      c("REL", "REM")
    ),
    method = "wilcox.test",
    label = "p.signif"
  )

```

p_scRNA_fractionMutation



p_scATA_fractionMutation



#

RNAIntegration Example

```
data_save_block <- readRDS("../Data/AML/JointCCA_AML8.rds")
scRNA_data <-
  readRDS("/Users/liqun/Desktop/scMutraceReproducibility/Figure5/Data/AML/singleCellMatrix/scRNA/AML8_scRNA.rds")
meta_scRNA <- scRNA_data@meta.data
df_AML08_CCA <- data.frame(
  UMAP1 = data_save_block$UMAP1,
  UMAP2 = data_save_block$UMAP2,
  Assay = data_save_block$Assay,
  Group = data_save_block$Group
)

getSampleStage <- function(x, meta_scRNA) {
  if (grepl("_reference$", x)) {
    cell_id <- sub("_reference$", "", x)
    if (cell_id %in% rownames(meta_scRNA)) {
      return(meta_scRNA[cell_id, "orig.ident"])
    } else {
      return(NA)
    }
  } else if (grepl("_query$", x)) {

```

```

    sample <- sub("#.*", "", x)
    return(sample)
  } else {
    return(NA)
  }
}

df_AML08_CCA$stage <- sapply(rownames(df_AML08_CCA), getSampleStage,
meta_scrNA = meta_scrNA)

assay_colors <- c("ATAC" = "#272E6A", "RNA" = "#D51F26")

stage_colors <- c(
  "AML8_DX" = "#d9d9d9",
  "AML8_REM" = "#969696",
  "AML8_REL" = "#000000")

group_colors <- c("B.Cell" = "#7DB18A", "CD16.Monocytes" = "#6F4A98",
"CD4.Memory" = "#4C502D",
  "CD4.Naive" = "#201C19", "CD8.Effector" = "#2A5B31",
"CD8.Memory" = "#69AD46",
  "CD8.Naive" = "#5B521A", "cDC" = "#8587B1", "CLP" =
"#B9D5AB", "Early.Basophil" = "#CD6D82",
  "Early.Erythrocyte" = "#B12579", "GMP" = "#25316E", "HSC" =
"#EAA63A", "Late.Erythrocyte" = "#EAAFC0",
  "Monocytes" = "#406DB1", "NK" = "#4C662E", "pDC" =
"#62B1E1", "Plasma" = "#D2E7EB",
  "Pre.B.Cell" = "#45948E", "Progenitor" = "#AA872C",
"Unknown" = "#EAEAEA")

p_UMAPbyAssay <- ggplot(df_AML08_CCA, aes(x = UMAP1, y = UMAP2, color =
Assay)) +
  geom_point(size = 0.5, alpha = 0.6) +
  scale_color_manual(values = assay_colors) +
  theme_classic() +
  labs(title = "UMAP by Assay")

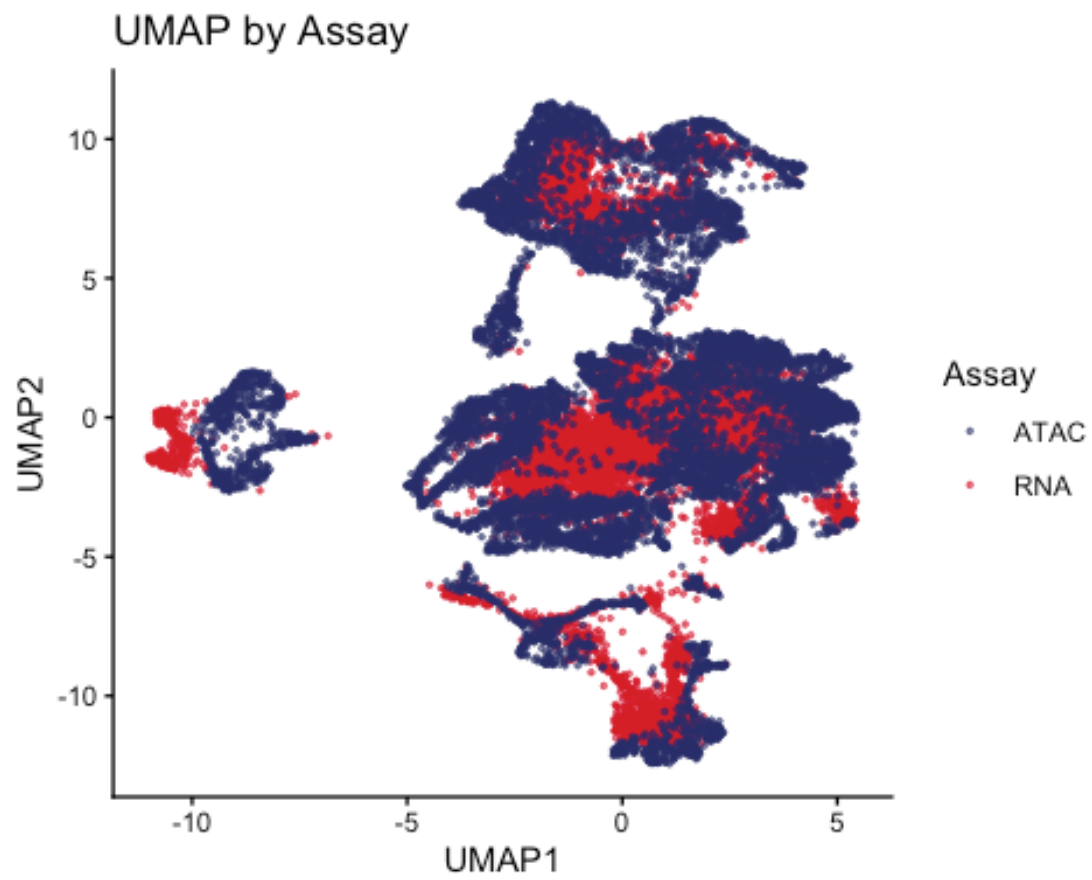
p_UMAPbyGroup <- ggplot(df_AML08_CCA, aes(x = UMAP1, y = UMAP2, color =
Group)) +
  geom_point(size = 0.5, alpha = 0.6) +
  scale_color_manual(values = group_colors) +
  theme_classic() +
  labs(title = "UMAP by Group")

p_UMAPbyStage <- ggplot(df_AML08_CCA, aes(x = UMAP1, y = UMAP2, color =
stage)) +
  geom_point(size = 0.5, alpha = 0.6) +
  scale_color_manual(values = stage_colors) +
  theme_classic() +

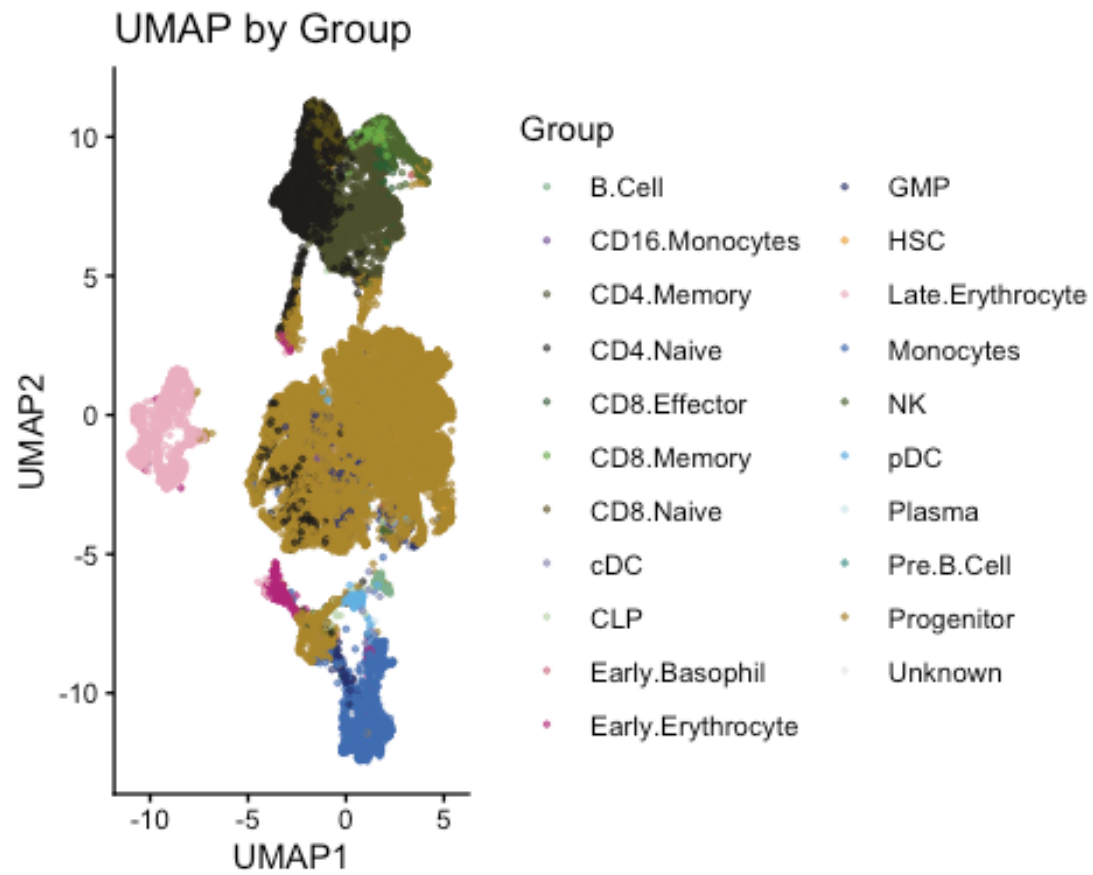
```

```
labs(title = "UMAP by Stage")
```

p_UMAPbyAssay



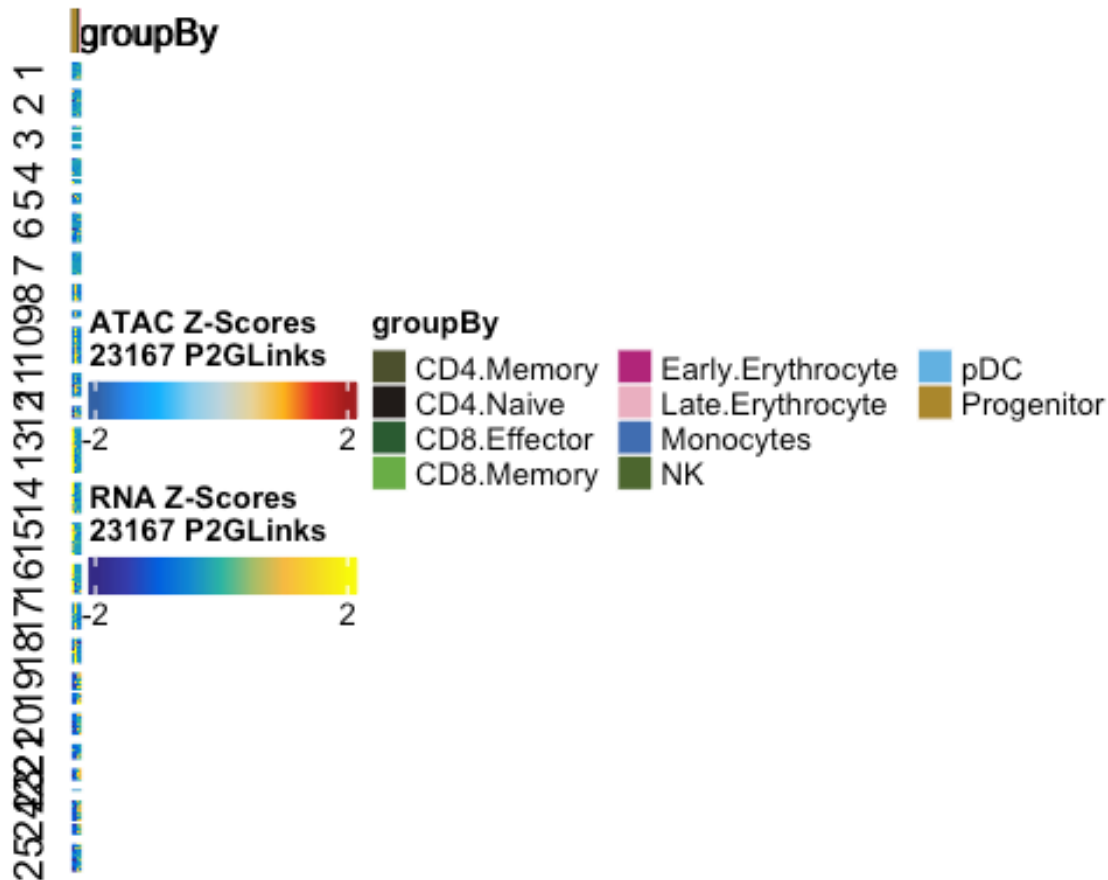
p_UMAPbyGroup



p_UMAPbyStage


```
##  
##  
##  
##  
proj <- addPeak2GeneLinks(scATAC_data, k = 20)  
## ArchR Logging to : ArchRLogs/ArchR-addPeak2GeneLinks-705863579fc8-Date-  
2026-01-27_Time-16-58-24.086862.Log  
## If there is an issue, please report to github with LogFile!  
## 2026-01-27 16:58:24.120077 : Getting Available Matrices, 0.001 mins  
elapsed.  
## 2026-01-27 16:58:24.420792 : Filtered Low Prediction Score Cells (336 of  
11193, 0.03), 0 mins elapsed.  
## 2026-01-27 16:58:24.526146 : Computing KNN, 0.002 mins elapsed.  
## 2026-01-27 16:58:24.820001 : Identifying Non-Overlapping KNN pairs, 0.007  
mins elapsed.  
## 2026-01-27 16:58:25.075536 : Identified 499 Groupings!, 0.011 mins  
elapsed.  
## 2026-01-27 16:58:25.092213 : Getting Group RNA Matrix, 0.012 mins elapsed.  
## 2026-01-27 16:58:52.611374 : Getting Group ATAC Matrix, 0.47 mins elapsed.  
## 2026-01-27 16:59:28.584735 : Normalizing Group Matrices, 1.07 mins  
elapsed.  
## 2026-01-27 16:59:30.576088 : Finding Peak Gene Pairings, 1.103 mins  
elapsed.  
## 2026-01-27 16:59:30.776685 : Computing Correlations, 1.106 mins elapsed.  
## 2026-01-27 16:59:33.770598 : Completed Peak2Gene Correlations!, 1.156 mins  
elapsed.  
## ArchR Logging successful to : ArchRLogs/ArchR-addPeak2GeneLinks-  
705863579fc8-Date-2026-01-27_Time-16-58-24.086862.Log  
# Get P2G Links  
p2g <- getPeak2GeneLinks(proj)  
p_plotPeak2GeneHeatmap <- plotPeak2GeneHeatmap(proj, groupBy =  
"predictedGroup_Un", palGroup = group_colors)  
## ArchR Logging to : ArchRLogs/ArchR-pPlotPeak2GeneHeatmap-7058dbb366c-Date-  
2026-01-27_Time-16-59-33.840274.Log  
## If there is an issue, please report to github with LogFile!  
## 2026-01-27 16:59:36.369645 : Determining KNN Groups!, 0.042 mins elapsed.  
## 2026-01-27 16:59:37.065583 : Ordering Peak2Gene Links!, 0.054 mins  
elapsed.  
## 2026-01-27 16:59:39.686105 : Constructing ATAC Heatmap!, 0.097 mins  
elapsed.  
## Adding Annotations..  
## Preparing Main Heatmap..  
## 'magick' package is suggested to install to give better rasterization.  
##  
## Set `ht_opt$message = FALSE` to turn off this message.  
## 2026-01-27 16:59:39.872628 : Constructing RNA Heatmap!, 0.101 mins  
elapsed.  
## Adding Annotations..  
## Preparing Main Heatmap..  
## 'magick' package is suggested to install to give better rasterization.
```

```
##
## Set `ht_opt$message = FALSE` to turn off this message.
## ArchR Logging successful to : ArchRLogs/ArchR-plotPeak2GeneHeatmap-
7058dbb366c-Date-2026-01-27_Time-16-59-33.840274.Log
ComplexHeatmap::draw(p_plotPeak2GeneHeatmap, heatmap_legend_side = "right")
```



celltype annotation similarity

```
data_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow =
length(setdiff(cellTypes, "Unknown")),
                                                    ncol =
length(setdiff(cellTypes, "Unknown"))))

rownames(data_cellTypeAnn_similarity) <- setdiff(cellTypes, "Unknown")
colnames(data_cellTypeAnn_similarity) <- setdiff(cellTypes, "Unknown")

for (i in 1:nrow(data_cellTypeAnn_similarity)){
  # row > scRNA
  ct_scRNA <- rownames(data_cellTypeAnn_similarity)[i]
  cell_barcode_scRNA <- AML_scRNA_meta %>%
    mutate(CellTag = rownames(.)) %>%
    dplyr::filter(Classified_Celltype == ct_scRNA) %>%
```



```

pull(CellTag)

for (j in 1:ncol(data_cellTypeAnn_similarity)){
  ct_scATA <- colnames(data_cellTypeAnn_similarity)[j]
  cell_barcode_scATA <- AML_scATA_meta %>%
    dplyr::filter(predictedGroup_Un == ct_scATA) %>%
    pull(predictedCell_RNAQun)

  similarity_score <- length(unique(intersect(cell_barcode_scRNA,
cell_barcode_scATA)))/length(unique(union(cell_barcode_scRNA,
cell_barcode_scATA)))
  data_cellTypeAnn_similarity[i, j] <- similarity_score
}
}

breaks <- seq(min(as.matrix(data_cellTypeAnn_similarity)),
              max(as.matrix(data_cellTypeAnn_similarity)),
              length.out = 4)

col_fun <- colorRamp2(
  breaks = breaks,
  colors = c("#404040", "#bababa", "#f4a582", "#ca0020")
)

row_types <- rownames(as.matrix(data_cellTypeAnn_similarity))
col_types <- colnames(as.matrix(data_cellTypeAnn_similarity))

group_colors <- c("B.Cell" = "#7DB18A", "CD16.Monocytes" = "#6F4A98",
"CD4.Memory" = "#4C502D",
                  "CD4.Naive" = "#201C19", "CD8.Effector" = "#2A5B31",
"CD8.Memory" = "#69AD46",
                  "CD8.Naive" = "#5B521A", "cDC" = "#8587B1", "CLP" =
"#B9D5AB", "Early.Basophil" = "#CD6D82",
                  "Early.Erythrocyte" = "#B12579", "GMP" = "#25316E", "HSC" =
"#EAA63A", "Late.Erythrocyte" = "#EAAFC0",
                  "Monocytes" = "#406DB1", "NK" = "#4C662E", "pDC" =
"#62B1E1", "Plasma" = "#D2E7EB",
                  "Pre.B.Cell" = "#45948E", "Progenitor" = "#AA872C",
"Unknown" = "#EAEAEA")

order_annotation <- c("HSC", "Progenitor",
                     "GMP", "Monocytes", "CD16.Monocytes", "cDC", "pDC",
                     "CLP", "Pre.B.Cell", "B.Cell", "CD4.Naive",
"CD4.Memory", "CD8.Naive", "CD8.Memory", "CD8.Effector", "NK",
                     "Early.Erythrocyte", "Late.Erythrocyte",
"Early.Basophil", "Plasma")

mat_ord <- as.matrix(data_cellTypeAnn_similarity)[order_annotation,
order_annotation]

```

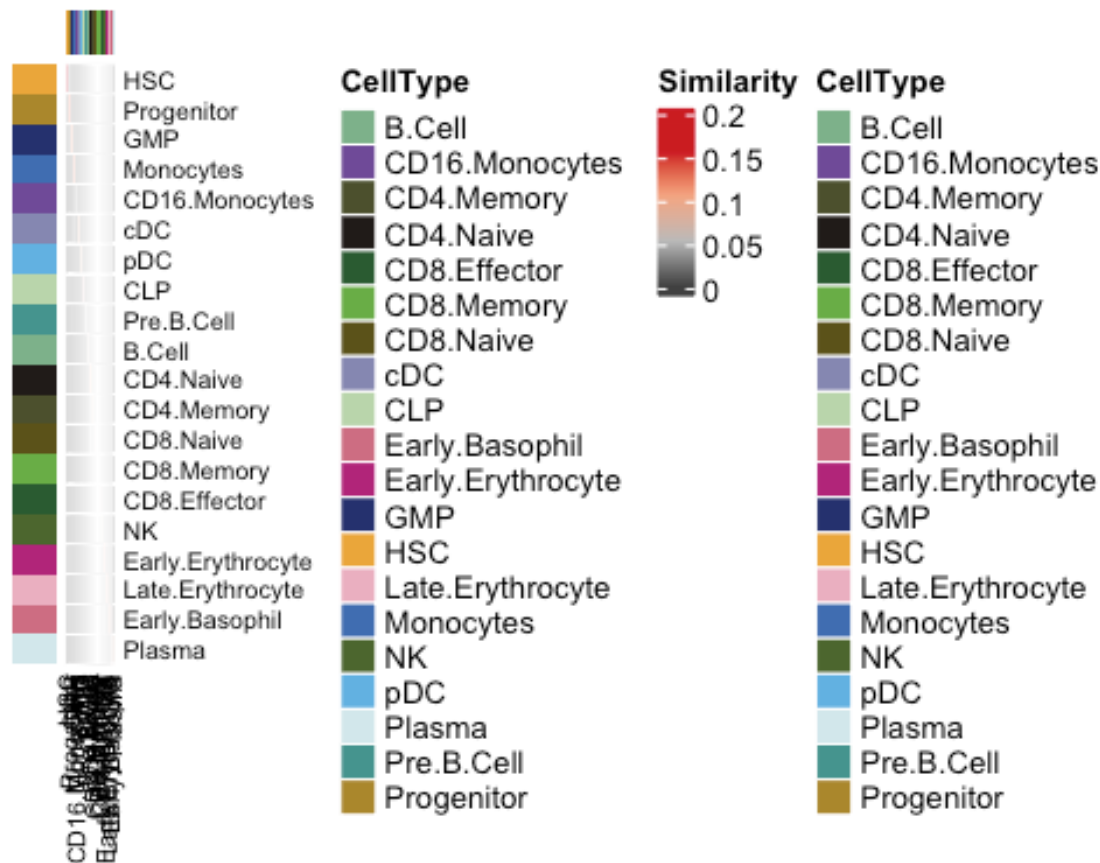
```
row_types_ord <- row_types[match(order_annotation, row_types)]
col_types_ord <- col_types[match(order_annotation, col_types)]
```

```
row_ha <- rowAnnotation(  
  CellType = row_types_ord,  
  col = list(CellType = group_colors),  
  show_annotation_name = FALSE,  
  annotation_width = unit(0.5, "cm")  
)
```

```
col_ha <- HeatmapAnnotation(  
  CellType = col_types_ord,  
  col = list(CellType = group_colors),  
  show_annotation_name = FALSE,  
  annotation_height = unit(0.5, "cm")  
)
```

```
p_cellType_similarity <- Heatmap(  
  mat_ord,  
  name = "Similarity",  
  col = col_fun,  
  cluster_rows = FALSE,  
  cluster_columns = FALSE,  
  show_row_names = TRUE,  
  show_column_names = TRUE,  
  row_names_gp = gpar(fontsize = 8),  
  column_names_gp = gpar(fontsize = 8),  
  row_title = NULL,  
  column_title = NULL,  
  top_annotation = col_ha,  
  left_annotation = row_ha,  
  rect_gp = gpar(col = "white", lwd = 1)  
)
```

```
p_cellType_similarity
```



sample stage mutation similarity

```
# may spent 2hrs
# generate data_commonBed_similarity_all.csv and
data_commonBed_similarity_noCensus.csv
# first using 0_pipelineFunctions.R > # pipelines for generating sample stage
mutation similarity matrix
```

```
sample_order_keep <- c(
  "AML1_DX", "AML1_REM",
  "AML2_DX", "AML2_REM", "AML2_REL",
  "AML3_DX", "AML3_REM",
  "AML4_DX", "AML4_REL",
  "AML5_DX", "AML5_REM", "AML5_REL",
  "AML6_DX", "AML6_REL",
  "AML7_REM", "AML7_REL",
  "AML8_DX", "AML8_REM", "AML8_REL",
  "AML10_DX", "AML10_REM", "AML10_REL",
  "AML11_DX", "AML11_REM", "AML11_REL",
  "AML12_DX", "AML12_REL",
  "AML13_DX", "AML13_REL",
  "AML15_DX", "AML15_REM", "AML15_REL",
```

```

      "AML16_DX", "AML16_REM", "AML16_REL",
      "AML17_DX", "AML17_REL",
      "AML18_DX", "AML18_REL",
      "AML19_DX", "AML19_REL",
      "AML20_DX", "AML20_REM",
      "AML21_DX", "AML21_REM", "AML21_REL",
      "AML22_DX", "AML22_REM", "AML22_REL",
      "AML24_DX", "AML24_REM", "AML24_REL",
      "AML26_DX", "AML26_REM", "AML26_REL",
      "AML27_DX", "AML27_REM", "AML27_REL")
data_commonBed_similarity_lenient <-
read.csv("../Data/AML/data_commonBed_similarity_all.csv", row.names = 1)
data_commonBed_similarity_lenient <-
data_commonBed_similarity_lenient[sample_order_keep, sample_order_keep]

data_commonBed_similarity_stringent <-
read.csv("../Data/AML/data_commonBed_similarity_noCensus.csv", row.names = 1)
data_commonBed_similarity_stringent <-
data_commonBed_similarity_stringent[sample_order_keep, sample_order_keep]

row_prefix <- sapply(rownames(data_commonBed_similarity_lenient), function(x)
strsplit(x, "_")[[1]][1])
row_suffix <- sapply(rownames(data_commonBed_similarity_lenient), function(x)
strsplit(x, "_")[[1]][2])
col_prefix <- sapply(colnames(data_commonBed_similarity_lenient), function(x)
strsplit(x, "_")[[1]][1])
col_suffix <- sapply(colnames(data_commonBed_similarity_lenient), function(x)
strsplit(x, "_")[[1]][2])

col_anno <- HeatmapAnnotation(
  Sample = anno_text(col_prefix, gp = gpar(fontsize = 8)),
  scATACStage = col_suffix,
  col = list(scATACStage = c("DX" = "#d9d9d9", "REM" = "#969696", "REL" =
"#000000"))
)

row_anno <- rowAnnotation(
  Sample = anno_text(row_prefix, gp = gpar(fontsize = 8)),
  scRNAStage = row_suffix,
  col = list(scRNAStage = c("DX" = "#d9d9d9", "REM" = "#969696", "REL" =
"#000000")))

col_fun <- colorRamp2(
  breaks = c(0, 0.33, 0.66, 1),
  colors = c("#404040", "#bababa", "#f4a582", "#ca0020")
)

Heatmap(data_commonBed_similarity_lenient,
  top_annotation = col_anno,

```

```

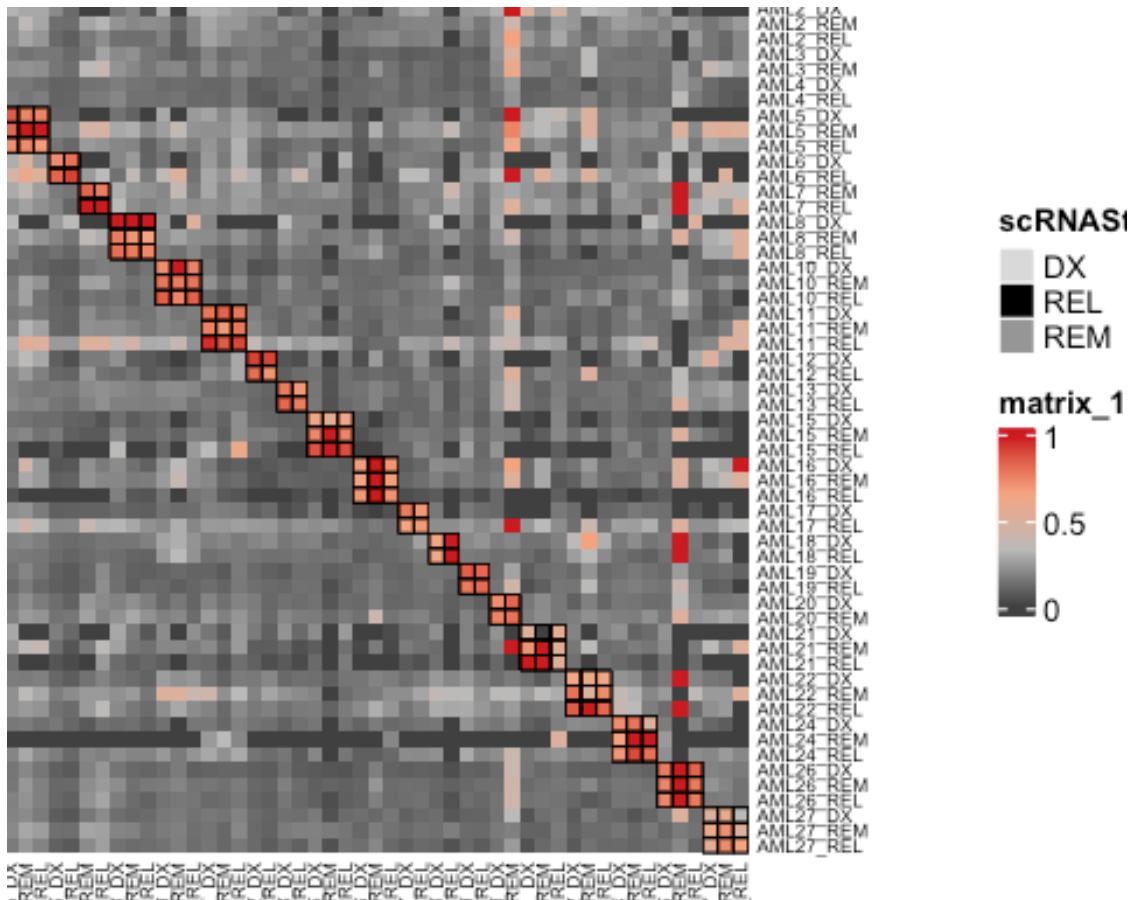
left_annotation = row_anno,
col = col_fun,
cluster_rows = FALSE,
cluster_columns = FALSE,
row_names_gp = gpar(fontsize = 6),
column_names_gp = gpar(fontsize = 6),
width = unit(10, "cm"), height = unit(10, "cm"),
layer_fun = function(j, i, x, y, width, height, fill) {

  grid.rect(x = x, y = y, width = width, height = height,
            gp = gpar(col = NA, fill = fill))

  match_prefix <- row_prefix[i] == col_prefix[j]
  if (any(match_prefix)) {
    grid.rect(x = x[match_prefix], y = y[match_prefix],
              width = width, height = height,
              gp = gpar(col = "#000000", lwd = 0.1, fill = NA))
  }
}
})

```

Warning: The input is a data frame-like object, convert it to a matrix.



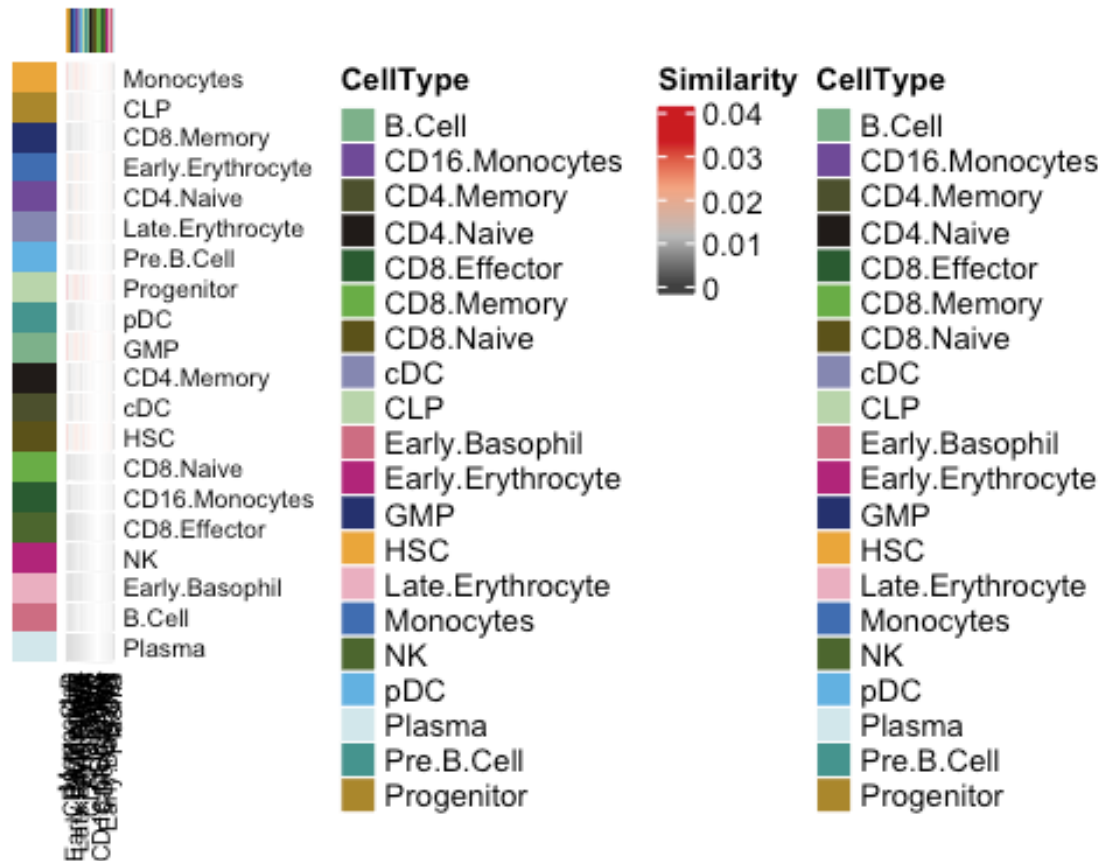
Heatmap(data_commonBed_similarity_stringent,

```
## Warning: The input is a data frame-like object, convert it to a matrix.
```



mutation Celltype similarity (scRNA vs scATAC)

```
data_mutationCelltype_similarity_scRNA_scATAC <-  
readRDS("../Data/AML/data_mutationCelltype_similarity_scRNA_scATAC.rds")  
similarity_sum_scRNA_scATAC <- Reduce("+",  
data_mutationCelltype_similarity_scRNA_scATAC)  
similarity_mean_scRNA_scATAC <- similarity_sum_scRNA_scATAC /  
length(data_mutationCelltype_similarity_scRNA_scATAC)  
  
breaks <- seq(min(as.matrix(similarity_mean_scRNA_scATAC)),  
              max(as.matrix(similarity_mean_scRNA_scATAC)),  
              length.out = 4)  
  
col_fun <- colorRamp2(  
  breaks = breaks,  
  colors = c("#404040", "#bababa", "#f4a582", "#ca0020")  
)  
  
p_cellType_similarity_Mutation_scRNA_scATAC <- Heatmap(  
  as.matrix(similarity_mean_scRNA_scATAC),  
  name = "Similarity",  
  col = col_fun,  
  cluster_rows = FALSE,  
  cluster_columns = FALSE,  
  show_row_names = TRUE,  
  show_column_names = TRUE,  
  row_names_gp = gpar(fontsize = 8),  
  column_names_gp = gpar(fontsize = 8),  
  row_title = NULL,  
  column_title = NULL,  
  top_annotation = col_ha,  
  left_annotation = row_ha,  
  rect_gp = gpar(col = "white", lwd = 1)  
)  
  
p_cellType_similarity_Mutation_scRNA_scATAC
```



mutation among celltype similarity merge all stages (scRNA)

```
data_mutationCelltype_similarity_scRNA <-
readRDS("../Data/AML/data_mutationCelltype_similarity_scRNA.rds")
similarity_sum_scRNA <- Reduce("+", data_mutationCelltype_similarity_scRNA)
similarity_mean_scRNA <- similarity_sum_scRNA /
length(data_mutationCelltype_similarity_scRNA)

breaks <- seq(min(as.matrix(similarity_mean_scRNA)),
              max(as.matrix(similarity_mean_scRNA)),
              length.out = 4)

col_fun <- colorRamp2(
  breaks = breaks,
  colors = c("#404040", "#bababa", "#f4a582", "#ca0020")
)

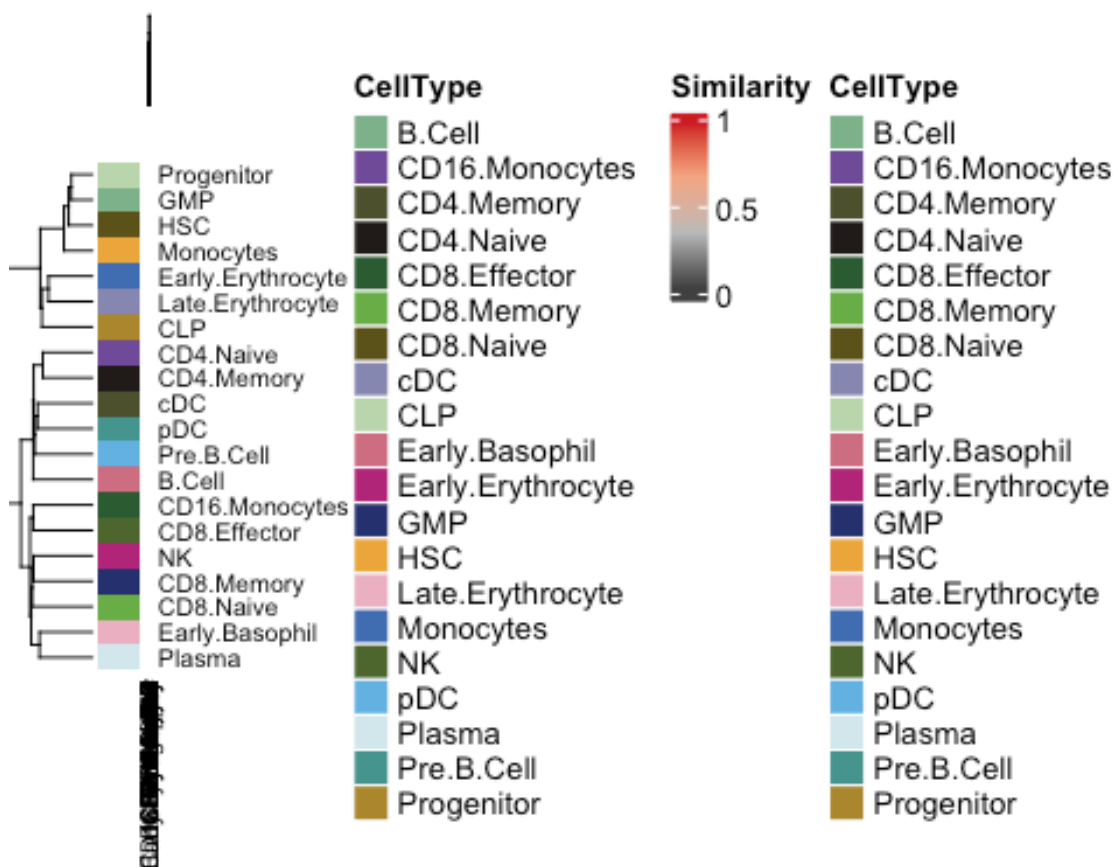
Heatmap(
  as.matrix(similarity_mean_scRNA),
  name = "Similarity",
```



```

col = col_fun,
cluster_rows = TRUE,
cluster_columns = TRUE,
show_row_names = TRUE,
show_column_names = TRUE,
row_names_gp = gpar(fontsize = 8),
column_names_gp = gpar(fontsize = 8),
row_title = NULL,
column_title = NULL,
top_annotation = col_ha,
left_annotation = row_ha,
rect_gp = gpar(col = "white", lwd = 1)
)

```



mutation among celltype similarity merge all stages (scATA)

```

data_mutationCelltype_similarity_scATA <-
readRDS("../Data/AML/data_mutationCelltype_similarity_scATA.rds")
similarity_sum_scATA <- Reduce("+", data_mutationCelltype_similarity_scATA)
similarity_mean_scATA <- similarity_sum_scATA /

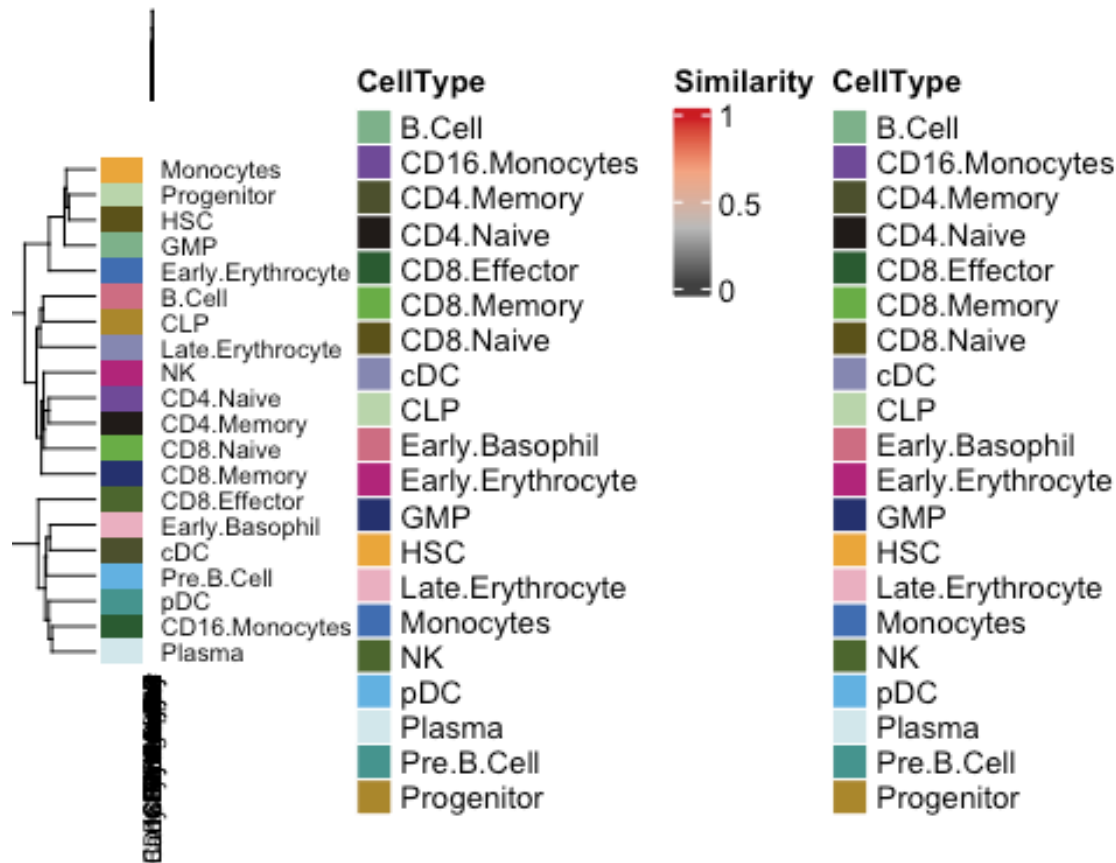
```

```

length(data_mutationCelltype_similarity_scATA)
breaks <- seq(min(as.matrix(similarity_mean_scATA)),
              max(as.matrix(similarity_mean_scATA)),
              length.out = 4)

col_fun <- colorRamp2(
  breaks = breaks,
  colors = c("#404040", "#bababa", "#f4a582", "#ca0020")
)
Heatmap(
  as.matrix(similarity_mean_scATA),
  name = "Similarity",
  col = col_fun,
  cluster_rows = TRUE,
  cluster_columns = TRUE,
  show_row_names = TRUE,
  show_column_names = TRUE,
  row_names_gp = gpar(fontsize = 8),
  column_names_gp = gpar(fontsize = 8),
  row_title = NULL,
  column_title = NULL,
  top_annotation = col_ha,
  left_annotation = row_ha,
  rect_gp = gpar(col = "white", lwd = 1)
)

```

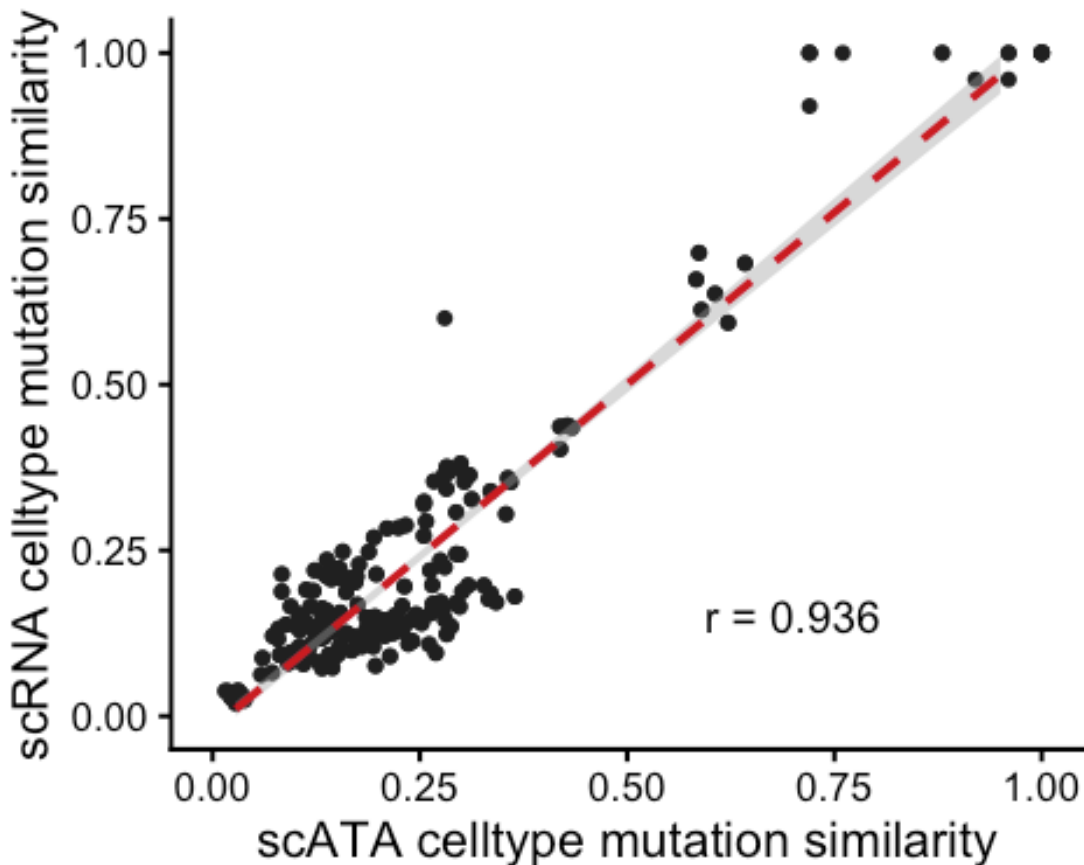


lm between scRNA and scATA mutation celltype similarity

```
mat_long_scATA <- reshape2::melt(as.matrix(similarity_mean_scATA), varnames =
c("Row", "Column"), value.name = "Value")
mat_long_scRNA <- reshape2::melt(as.matrix(similarity_mean_scRNA), varnames =
c("Row", "Column"), value.name = "Value")

p_correlation_celltypeMutationSimilarity <- ggplot(data = NULL, aes(x =
mat_long_scATA$Value, y = mat_long_scRNA$Value)) +
  geom_point(size = 1.5, color = "#202020") +
  geom_smooth(method = "lm", se = TRUE, color = "#ca0020", linetype =
"dashed", lwd = 1.2) +
  theme_classic(base_size = 16) +
  xlab("scATA celltype mutation similarity") +
  ylab("scRNA celltype mutation similarity") +
  xlim(0, 1) + ylim(0, 1) +
  annotate("text",
    x = max(mat_long_scRNA$Value) * 0.7,
    y = max(mat_long_scATA$Value) * 0.15,
    label = paste0("r = ", round(cor(mat_long_scATA$Value,
mat_long_scRNA$Value), 3)),
    size = 5)
```

```
p_correlation_celltypeMutationSimilarity
## `geom_smooth()` using formula = 'y ~ x'
## Warning: Removed 3 rows containing missing values or values outside the
scale range
## (`geom_smooth()`).
```



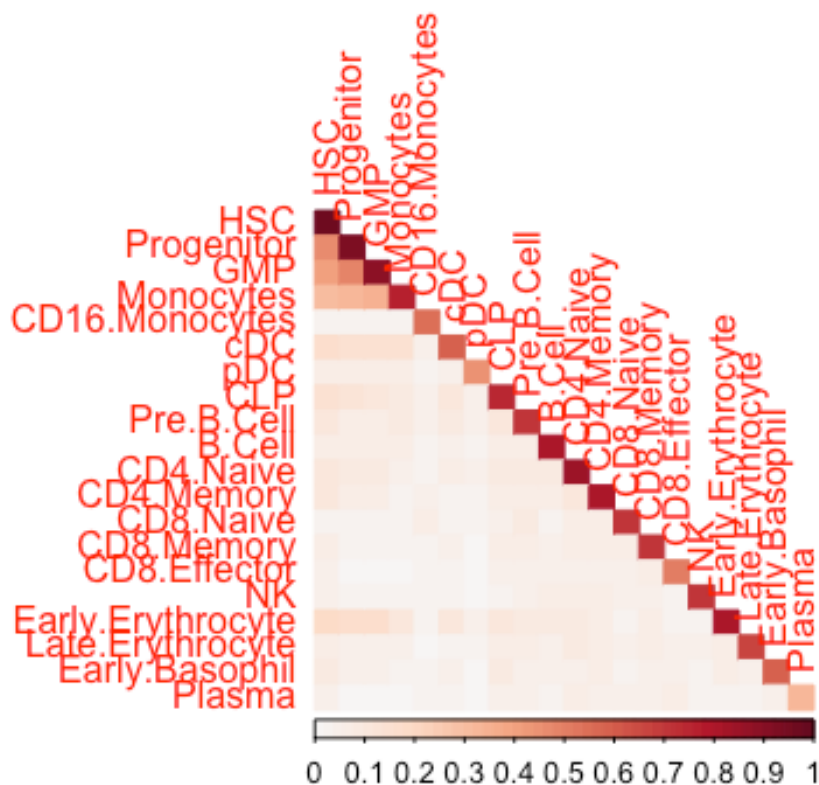
```
cor.test(mat_long_scATA$Value, mat_long_scRNA$Value, method = "pearson")
##
## Pearson's product-moment correlation
##
## data: mat_long_scATA$Value and mat_long_scRNA$Value
## t = 53.212, df = 398, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## 0.9230498 0.9474228
## sample estimates:
## cor
## 0.9363552
```

mutation among celltype similarity by stages (scRNA)

```
res_scRNA_order_manu <- c("HSC", "Progenitor",
                          "GMP", "Monocytes", "CD16.Monocytes", "cDC", "pDC",
                          "CLP", "Pre.B.Cell", "B.Cell", "CD4.Naive",
                          "CD4.Memory", "CD8.Naive", "CD8.Memory", "CD8.Effector", "NK",
                          "Early.Erythrocyte", "Late.Erythrocyte",
                          "Early.Basophil", "Plasma")

cols <- rev(colorRampPalette(brewer.pal(11, "RdBu"))(100))

data_mutationCelltype_similarity_scRNA_DX <-
readRDS("../Data/AML/data_mutationCelltype_similarity_scRNA_DX.rds")
similarity_sum_scRNA_DX <- Reduce("+",
data_mutationCelltype_similarity_scRNA_DX)
similarity_mean_scRNA_DX <- similarity_sum_scRNA_DX /
length(data_mutationCelltype_similarity_scRNA_DX)
corrplot(as.matrix(similarity_mean_scRNA_DX[res_scRNA_order_manu,
res_scRNA_order_manu]), order = "original", method = 'color',
         type = 'lower', col = cols, col.lim = c(0, 1))
```

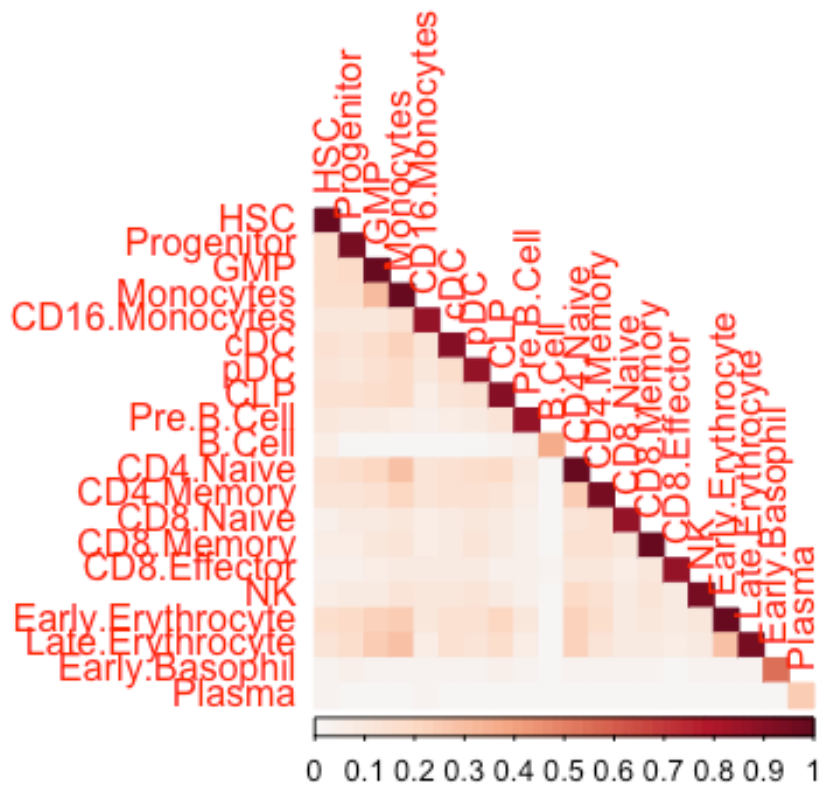


```
data_mutationCelltype_similarity_scRNA_REM <-
```

```

readRDS("../Data/AML/data_mutationCelltype_similarity_scRNA_REM.rds")
similarity_sum_scRNA_REM <- Reduce("+",
data_mutationCelltype_similarity_scRNA_REM)
similarity_mean_scRNA_REM <- similarity_sum_scRNA_REM /
length(data_mutationCelltype_similarity_scRNA_REM)
corrplot(as.matrix(similarity_mean_scRNA_REM[res_scRNA_order_manu,
res_scRNA_order_manu]), order = "original", method = 'color',
type = 'lower', col = cols, col.lim = c(0, 1))

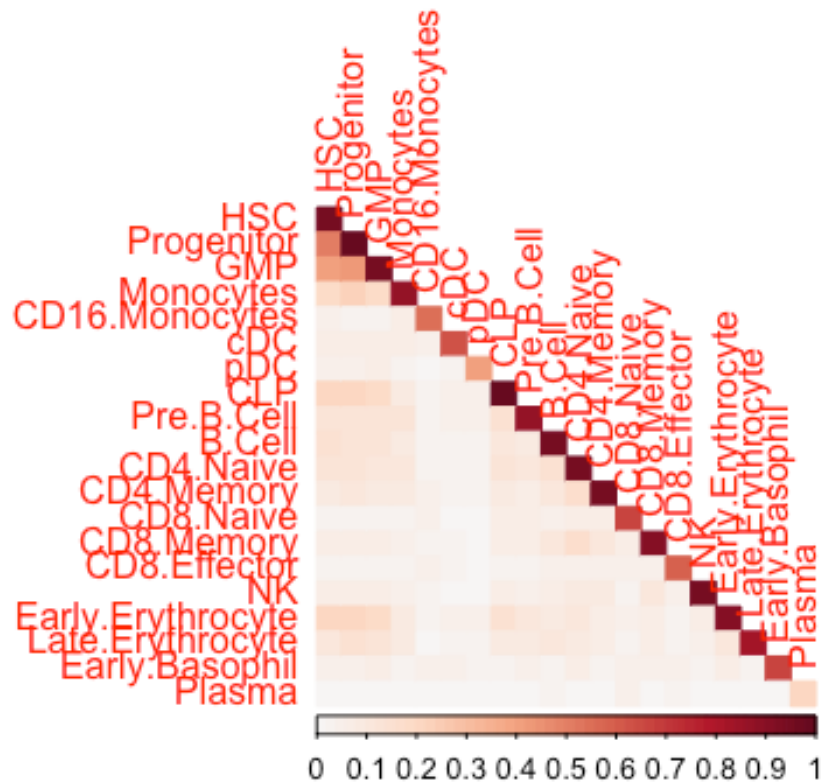
```



```

data_mutationCelltype_similarity_scRNA_REL <-
readRDS("../Data/AML/data_mutationCelltype_similarity_scRNA_REL.rds")
similarity_sum_scRNA_REL <- Reduce("+",
data_mutationCelltype_similarity_scRNA_REL)
similarity_mean_scRNA_REL <- similarity_sum_scRNA_REL /
length(data_mutationCelltype_similarity_scRNA_REL)
corrplot(as.matrix(similarity_mean_scRNA_REL[res_scRNA_order_manu,
res_scRNA_order_manu]), order = "original", method = 'color',
type = 'lower', col = cols, col.lim = c(0, 1))

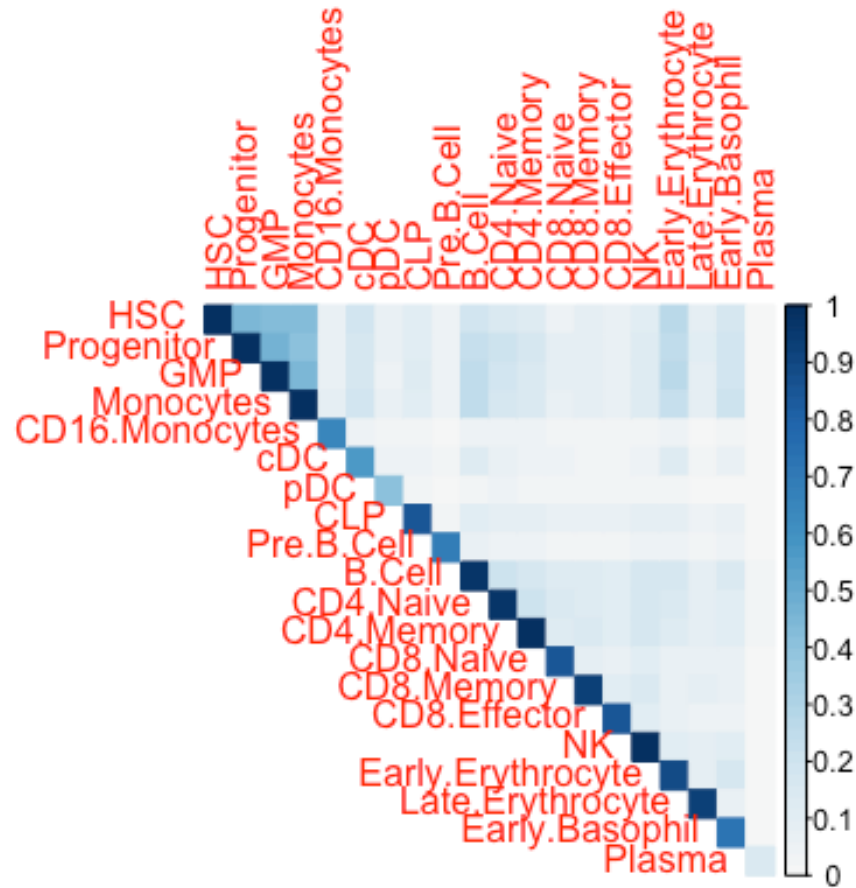
```



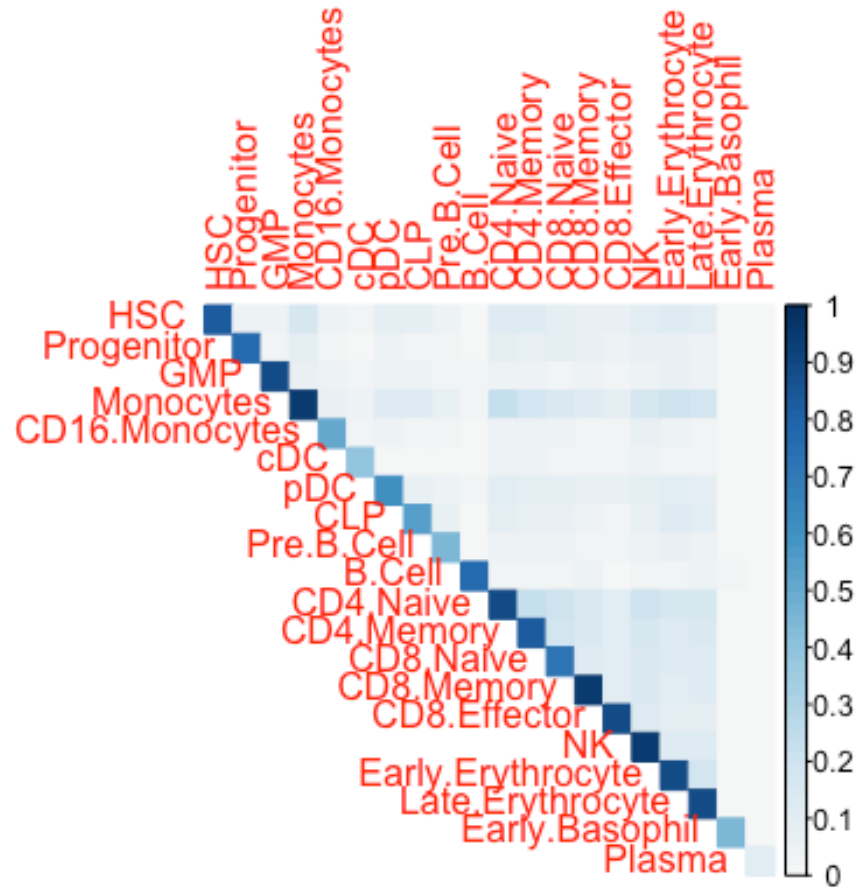
#

mutation among celltype similarity by stages (scATA)

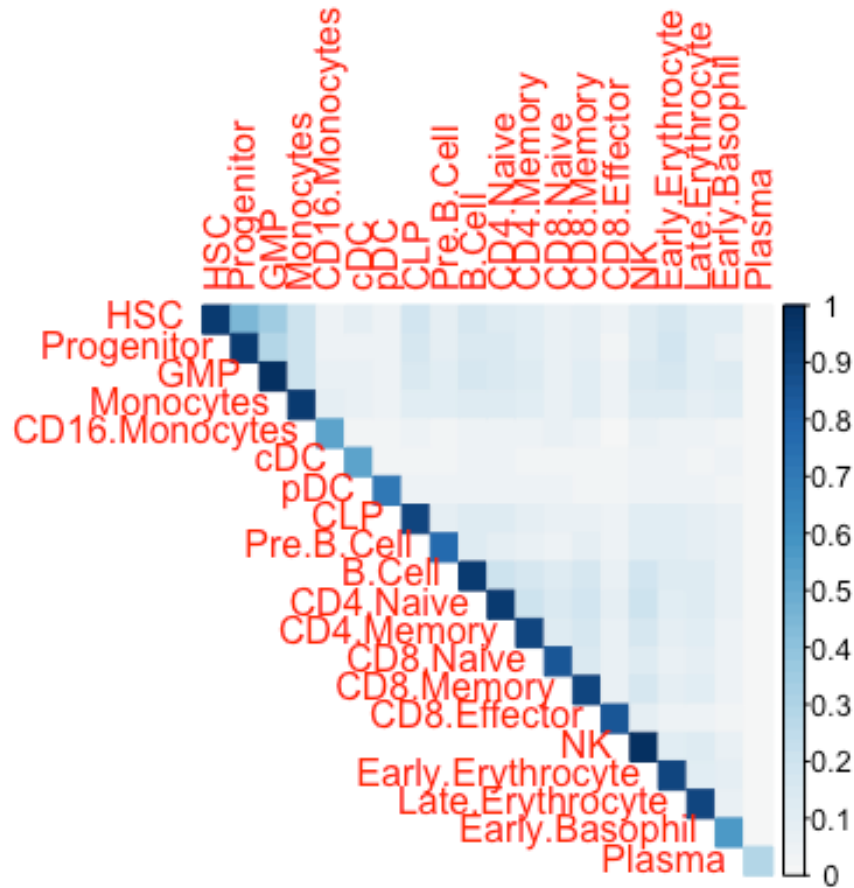
```
cols <- colorRampPalette(brewer.pal(11, "RdBu"))(100)
data_mutationCelltype_similarity_scATA_DX <-
readRDS("../Data/AML/data_mutationCelltype_similarity_scATA_DX.rds")
similarity_sum_scATA_DX <- Reduce("+",
data_mutationCelltype_similarity_scATA_DX)
similarity_mean_scATA_DX <- similarity_sum_scATA_DX /
length(data_mutationCelltype_similarity_scATA_DX)
corrplot(as.matrix(similarity_mean_scATA_DX[res_scRNA_order_manu,
res_scRNA_order_manu]), order = "original", method = 'color',
type = 'upper', col = cols, col.lim = c(0, 1))
```



```
data_mutationCelltype_similarity_scATA_REM <-
readRDS("../Data/AML/data_mutationCelltype_similarity_scATA_REM.rds")
similarity_sum_scATA_REM <- Reduce("+",
data_mutationCelltype_similarity_scATA_REM)
similarity_mean_scATA_REM <- similarity_sum_scATA_REM /
length(data_mutationCelltype_similarity_scATA_REM)
corrplot(as.matrix(similarity_mean_scATA_REM[res_scRNA_order_manu,
res_scRNA_order_manu]), order = "original", method = 'color',
type = 'upper', col = cols, col.lim = c(0, 1))
```

```
data_mutationCelltype_similarity_scATA_REL <-
readRDS("../Data/AML/data_mutationCelltype_similarity_scATA_REL.rds")
similarity_sum_scATA_REL <- Reduce("+",
data_mutationCelltype_similarity_scATA_REL)
similarity_mean_scATA_REL <- similarity_sum_scATA_REL /
length(data_mutationCelltype_similarity_scATA_REL)
corrplot(as.matrix(similarity_mean_scATA_REL[res_scRNA_order_manu,
res_scRNA_order_manu]), order = "original", method = 'color',
type = 'upper', col = cols, col.lim = c(0, 1))
```



stage diff mutations

```

AML_ATA_mutPeakGene <-
readRDS("../Results/AML/data_AML_scATA_mutpeakgene.rds")
AML_RNA_mutExprGene <-
readRDS("../Results/AML/data_AML_scRNA_mutgeneExp.rds")

HSPC = c("HSC", "Progenitor")
Myeloid = c("GMP", "Monocytes", "CD16.Monocytes", "cDC", "pDC")
Lymphoid = c("CLP", "Pre.B.Cell", "B.Cell", "CD4.Memory", "CD4.Naive",
"CD8.Effector", "CD8.Memory", "CD8.Naive", "NK")
Eryth = c("Early.Erythrocyte", "Late.Erythrocyte")

groups <- list(HSPC = HSPC, Myeloid = Myeloid, Lymphoid = Lymphoid, Eryth =
Eryth)

AML_ATA_mutPeakGene$mainCelltype <-
sapply(AML_ATA_mutPeakGene$Classified_Celltype, function(ct) {
  mainCelltype <- "Other"
  for (group_name in names(groups)) {
    if (ct %in% groups[[group_name]]) {
      mainCelltype <- group_name
    }
  }
})

```

```

        break
      }
    }
    return(mainCelltype)
  })

AML_RNA_mutExprGene$mainCelltype <-
sapply(AML_RNA_mutExprGene$Classified_Celltype, function(ct) {
  mainCelltype <- "Other"
  for (group_name in names(groups)) {
    if (ct %in% groups[[group_name]]) {
      mainCelltype <- group_name
      break
    }
  }
  return(mainCelltype)
})

run_fisher_stage <- function(df) {

  # pivot_wider generate wide format mut_status_col
  df_wide <- df %>%
    tidyr::pivot_wider(
      names_from = mut_status_col,
      values_from = cell_count,
      values_fill = 0
    )

  # exclude stage_id_col and mut_id_col
  status_cols <- setdiff(colnames(df_wide), c("stage_id_col", "mut_id_col"))

  # add missing column if only one status present
  if (length(status_cols) == 1) {
    missing_col <- ifelse(status_cols == "Mut", "WT", "Mut")
    df_wide[[missing_col]] <- 0
    status_cols <- c(status_cols, missing_col)
  }

  # rename status columns to "Mut" and "WT"
  names(df_wide)[match(status_cols, names(df_wide))] <- c("Mut", "WT")

  stages <- unique(df_wide$stage_id_col)

  if (length(stages) < 2) {
    return(tibble(
      stage1 = character(0),
      stage2 = character(0),
      p.value = numeric(0),
      log2_OR = numeric(0),

```

```

    estimate = numeric(0),
    table = character(0)
  ))
}

purrr::map_df(combn(stages, 2, simplify = FALSE), function(stg) {

  d1 <- dplyr::filter(df_wide, stage_id_col == stg[1])
  d2 <- dplyr::filter(df_wide, stage_id_col == stg[2])

  mut1 <- ifelse(length(d1$Mut) == 0, 0, d1$Mut)
  wt1 <- ifelse(length(d1$WT) == 0, 0, d1$WT)
  mut2 <- ifelse(length(d2$Mut) == 0, 0, d2$Mut)
  wt2 <- ifelse(length(d2$WT) == 0, 0, d2$WT)

  frac1 <- ifelse((mut1 + wt1) == 0, NA, mut1 / (mut1 + wt1))
  frac2 <- ifelse((mut2 + wt2) == 0, NA, mut2 / (mut2 + wt2))

  tab <- matrix(c(mut1, wt1, mut2, wt2), nrow = 2, byrow = TRUE)

  # if all zero, return NA
  if (sum(tab) == 0) {
    return(tibble(
      stage1 = stg[1],
      stage2 = stg[2],
      p.value = NA,
      log2_OR = NA,
      estimate = NA,
      frac_stage1 = NA,
      frac_stage2 = NA,
      table = paste(tab, collapse = ",")
    ))
  }

  ft <- fisher.test(tab)

  tibble(
    stage1 = stg[1],
    stage2 = stg[2],
    p.value = ft$p.value,
    log2_OR = log2(ft$estimate),
    estimate = ft$estimate,
    frac_stage1 = frac1,
    frac_stage2 = frac2,
    table = paste(tab, collapse = ",")
  )
})
}

```

```

AML_scrNA_stageMutCount <- AML_RNA_mutExprGene %>%
  dplyr::group_by(mut_id_col, stage_id_col, mut_status_col, mainCelltype) %>%
  dplyr::summarise(cell_count = n(), .groups = "drop")

AML_scATA_stageMutCount <- AML_ATA_mutPeakGene %>%
  dplyr::group_by(mut_id_col, stage_id_col, mut_status_col, mainCelltype) %>%
  dplyr::summarise(cell_count = n(), .groups = "drop")

res_scrNA <- AML_scrNA_stageMutCount %>%
  dplyr::group_by(mut_id_col, mainCelltype) %>%
  dplyr::group_modify(~ run_fisher_stage(.x)) %>%
  dplyr::ungroup() %>%
  dplyr::mutate(
    Tag = paste0(
      gsub(".*_", "", stage1),
      "_vs_",
      gsub(".*_", "", stage2)
    )
  )

res_scATA <- AML_scATA_stageMutCount %>%
  dplyr::group_by(mut_id_col, mainCelltype) %>%
  dplyr::group_modify(~ run_fisher_stage(.x)) %>%
  dplyr::ungroup() %>%
  dplyr::mutate(
    Tag = paste0(
      gsub(".*_", "", stage1),
      "_vs_",
      gsub(".*_", "", stage2)
    )
  )

# GO analysis
# Dx_vs_REM
mutscrNA_sig_DX_vs_REM <- res_scrNA %>% dplyr::filter(Tag %in%
  c("DX_vs_REM") & p.value <= 0.05) %>% dplyr::pull(mut_id_col) %>% unique()
genescrNA_sig_DX_vs_REM <- AML_RNA_mutExprGene %>%
  dplyr::filter(mut_id_col %in% mutscrNA_sig_DX_vs_REM) %>%
  dplyr::pull(geneSymbolMut) %>%
  unique()

mutscATA_sig_DX_vs_REM <- res_scATA %>% dplyr::filter(Tag %in%
  c("DX_vs_REM") & p.value <= 0.05) %>% dplyr::pull(mut_id_col) %>% unique()
genescATA_sig_DX_vs_REM <- AML_ATA_mutPeakGene %>%
  dplyr::filter(mut_id_col %in% mutscATA_sig_DX_vs_REM) %>%
  dplyr::pull(geneSymbol) %>%
  unique()

GO_RNA_enrich_DX_vs_REM <- enrichGO(gene = genescrNA_sig_DX_vs_REM,

```

```

        OrgDb = org.Hs.eg.db,
        keyType = "SYMBOL",
        ont = "BP",
        pAdjustMethod = "BH",
        pvalueCutoff = 0.5)
GO_RNA_enrich_DX_vs_REM <- as.data.frame(GO_RNA_enrich_DX_vs_REM)

GO_ATA_enrich_DX_vs_REM <- enrichGO(gene = gene_scATA_sig_DX_vs_REM,
        OrgDb = org.Hs.eg.db,
        keyType = "SYMBOL",
        ont = "BP",
        pAdjustMethod = "BH",
        pvalueCutoff = 0.5)
GO_ATA_enrich_DX_vs_REM <- as.data.frame(GO_ATA_enrich_DX_vs_REM)

# REM_vs_REL
mutscRNA_sig_REM_vs_REL <- res_scRNA %>% dplyr::filter(Tag %in%
c("REL_vs_REL") & p.value <= 0.05) %>% dplyr::pull(mut_id_col) %>% unique()
genescRNA_sig_REM_vs_REL <- AML_RNA_mutExprGene %>%
    dplyr::filter(mut_id_col %in% mutscRNA_sig_REM_vs_REL) %>%
    dplyr::pull(geneSymbolMut) %>%
    unique()

mutscATA_sig_REM_vs_REL <- res_scATA %>% dplyr::filter(Tag %in%
c("REL_vs_REL") & p.value <= 0.05) %>% dplyr::pull(mut_id_col) %>% unique()
genescATA_sig_REM_vs_REL <- AML_ATA_mutPeakGene %>%
    dplyr::filter(mut_id_col %in% mutscATA_sig_REM_vs_REL) %>%
    dplyr::pull(geneSymbol) %>%
    unique()

GO_RNA_enrich_REM_vs_REL <- enrichGO(gene = genescRNA_sig_REM_vs_REL,
        OrgDb = org.Hs.eg.db,
        keyType = "SYMBOL",
        ont = "BP",
        pAdjustMethod = "BH",
        pvalueCutoff = 0.5)
GO_RNA_enrich_REM_vs_REL <- as.data.frame(GO_RNA_enrich_REM_vs_REL)

GO_ATA_enrich_REM_vs_REL <- enrichGO(gene = genescATA_sig_REM_vs_REL,
        OrgDb = org.Hs.eg.db,
        keyType = "SYMBOL",
        ont = "BP",
        pAdjustMethod = "BH",
        pvalueCutoff = 0.5)
GO_ATA_enrich_REM_vs_REL <- as.data.frame(GO_ATA_enrich_REM_vs_REL)

# Dx_vs_REL
mutscRNA_sig_DX_vs_REL <- res_scRNA %>% dplyr::filter(Tag %in%
c("DX_vs_REL") & p.value <= 0.05) %>% dplyr::pull(mut_id_col) %>% unique()

```

```

gene_scRNA_sig_DX_vs_REL <- AML_RNA_mutExprGene %>%
  dplyr::filter(mut_id_col %in% muts_scRNA_sig_DX_vs_REL) %>%
  dplyr::pull(geneSymbolMut) %>%
  unique()

muts_scATA_sig_DX_vs_REL <- res_scATA %>% dplyr::filter(Tag %in%
c("DX_vs_REL") & p.value <= 0.05) %>% dplyr::pull(mut_id_col) %>% unique()
gene_scATA_sig_DX_vs_REL <- AML_ATA_mutPeakGene %>%
  dplyr::filter(mut_id_col %in% muts_scATA_sig_DX_vs_REL) %>%
  dplyr::pull(geneSymbol) %>%
  unique()

GO_RNA_enrich_DX_vs_REL <- enrichGO(gene = gene_scRNA_sig_DX_vs_REL,
  OrgDb = org.Hs.eg.db,
  keyType = "SYMBOL",
  ont = "BP",
  pAdjustMethod = "BH",
  pvalueCutoff = 0.5)
GO_RNA_enrich_DX_vs_REL <- as.data.frame(GO_RNA_enrich_DX_vs_REL)

GO_ATA_enrich_DX_vs_REL <- enrichGO(gene = gene_scATA_sig_DX_vs_REL,
  OrgDb = org.Hs.eg.db,
  keyType = "SYMBOL",
  ont = "BP",
  pAdjustMethod = "BH",
  pvalueCutoff = 0.5)
GO_ATA_enrich_DX_vs_REL <- as.data.frame(GO_ATA_enrich_DX_vs_REL)

# all
muts_scRNA_sig_ALL <- res_scRNA %>% dplyr::filter(p.value <= 0.05) %>%
dplyr::pull(mut_id_col) %>% unique()
gene_scRNA_sig_ALL <- AML_RNA_mutExprGene %>%
  dplyr::filter(mut_id_col %in% muts_scRNA_sig_ALL) %>%
  dplyr::pull(geneSymbolMut) %>%
  unique()

muts_scATA_sig_ALL <- res_scATA %>% dplyr::filter(p.value <= 0.05) %>%
dplyr::pull(mut_id_col) %>% unique()
gene_scATA_sig_ALL <- AML_ATA_mutPeakGene %>%
  dplyr::filter(mut_id_col %in% muts_scATA_sig_ALL) %>%
  dplyr::pull(geneSymbol) %>%
  unique()

GO_RNA_enrich_ALL <- enrichGO(gene = gene_scRNA_sig_ALL,
  OrgDb = org.Hs.eg.db,
  keyType = "SYMBOL",
  ont = "BP",
  pAdjustMethod = "BH",
  pvalueCutoff = 0.5)

```

```

GO_RNA_enrich_ALL <- as.data.frame(GO_RNA_enrich_ALL)

GO_ATA_enrich_ALL <- enrichGO(gene = gene_scATA_sig_ALL,
                              OrgDb = org.Hs.eg.db,
                              keyType = "SYMBOL",
                              ont = "BP",
                              pAdjustMethod = "BH",
                              pvalueCutoff = 0.5)
GO_ATA_enrich_ALL <- as.data.frame(GO_ATA_enrich_ALL)

# combine
GO_RNA_enrich_DX_vs_REM$Comparison <- "DX_vs_REM"
GO_RNA_enrich_REM_vs_REL$Comparison <- "REM_vs_REL"
GO_RNA_enrich_DX_vs_REL$Comparison <- "DX_vs_REL"
GO_RNA_enrich_ALL$Comparison <- "ALL"
GO_RNA_enrich <- rbind(GO_RNA_enrich_DX_vs_REM, GO_RNA_enrich_REM_vs_REL,
GO_RNA_enrich_DX_vs_REL, GO_RNA_enrich_ALL)
GO_ATA_enrich_DX_vs_REM$Comparison <- "DX_vs_REM"
GO_ATA_enrich_REM_vs_REL$Comparison <- "REM_vs_REL"
GO_ATA_enrich_DX_vs_REL$Comparison <- "DX_vs_REL"
GO_ATA_enrich_ALL$Comparison <- "ALL"
GO_ATA_enrich <- rbind(GO_ATA_enrich_DX_vs_REM, GO_ATA_enrich_REM_vs_REL,
GO_ATA_enrich_DX_vs_REL, GO_ATA_enrich_ALL)

```

GO for stage diff mutations

```

GO_RNA_enrich <- read.csv("../Data/AML/GO_RNA_enrich.csv")
GO_ATA_enrich <- read.csv("../Data/AML/GO_ATA_enrich.csv")
# remove duplicated GO terms
GO_ID_RNA_ex <- c("GO:0030099", "GO:0051056")
GO_ID_ATA_ex <- c("GO:0015669", "GO:0015670", "GO:0042744", "GO:0098869",
"GO:0019755",
                  "GO:0042743", "GO:0044403")

GO_RNA_enrich_selectAll <- GO_RNA_enrich %>%
  dplyr::filter(Comparison == "ALL") %>%
  dplyr::filter(!ID %in% GO_ID_RNA_ex) %>%
  dplyr::arrange(pvalue) %>%
  dplyr::slice_head(n = 10) %>%
  dplyr::mutate(
    nlog2p = -log2(pvalue),
    Description = factor(Description, levels = Description[order(nlog2p)])
  )

p_GO_RNA_enrich_all_top10 <- ggplot(GO_RNA_enrich_selectAll, aes(x = nlog2p,
y = Description)) +
  geom_col(fill = "#F8B191") +
  theme_classic() +
  labs(x = "-log2(p-value)", y = "GO Term") +

```



```

  theme(axis.text.x = element_text(angle = 0, hjust = 0.5))

GO_ATA_enrich_selectAll <- GO_ATA_enrich %>%
  dplyr::filter(Comparison == "ALL") %>%
  dplyr::filter(!ID %in% GO_ID_ATA_ex) %>%
  dplyr::arrange(pvalue) %>%
  dplyr::slice_head(n = 10) %>%
  dplyr::mutate(
    nlog2p = -log2(pvalue),
    Description = factor(Description, levels = Description[order(nlog2p)])
  )

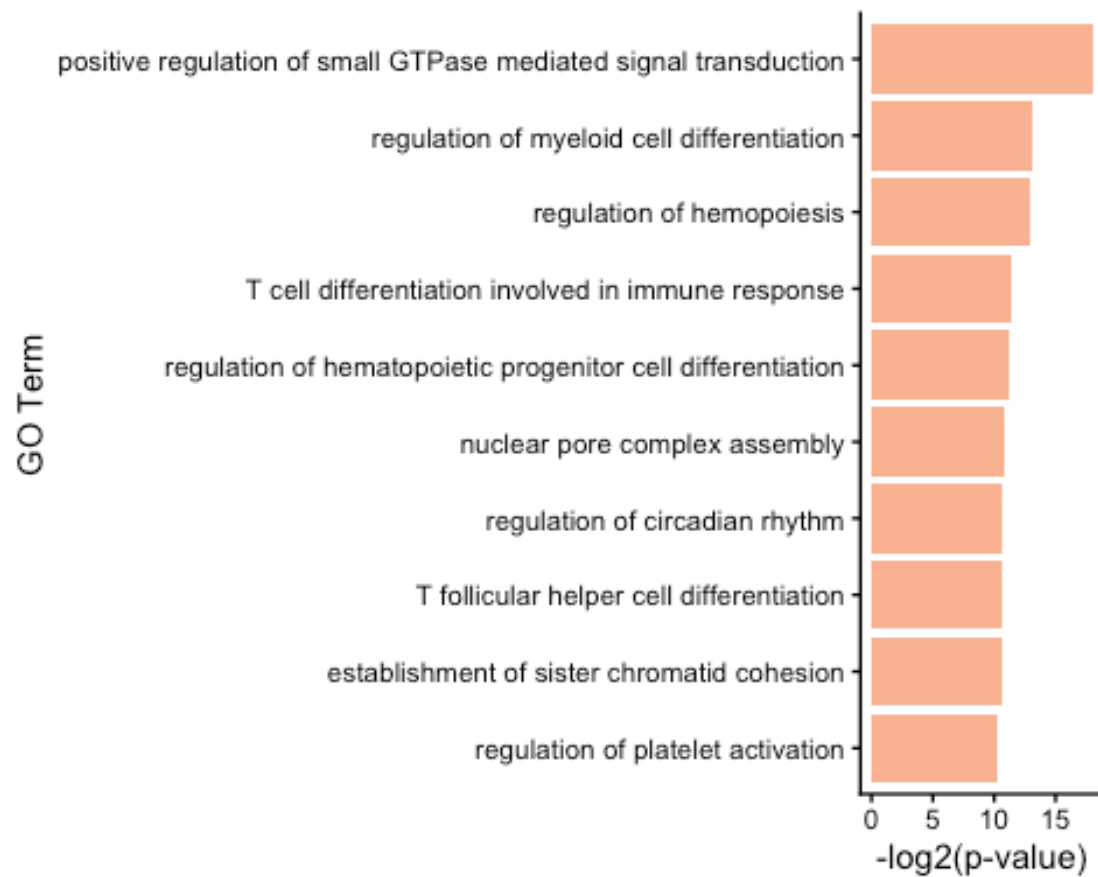
p_GO_ATA_enrich_all_top10 <- ggplot(GO_ATA_enrich_selectAll, aes(x = nlog2p, y
= Description)) +
  geom_col(fill = "#B9D4E2") +
  theme_classic() +
  labs(x = "-log2(p-value)", y = "GO Term") +
  theme(axis.text.x = element_text(angle = 0, hjust = 0.5))

# combine plot
GO_ATA_enrich_selectAll$Type <- "scATAC"
GO_RNA_enrich_selectAll$Type <- "scRNA"
GO_enrich_selectAll_combined <- rbind(GO_RNA_enrich_selectAll,
GO_ATA_enrich_selectAll)
GO_enrich_selectAll_combined <- GO_enrich_selectAll_combined %>%
  arrange(Type, nlog2p) %>%
  group_by(Type) %>%
  mutate(
    Description = factor(Description, levels =
unique(Description[order(nlog2p)]))
  )

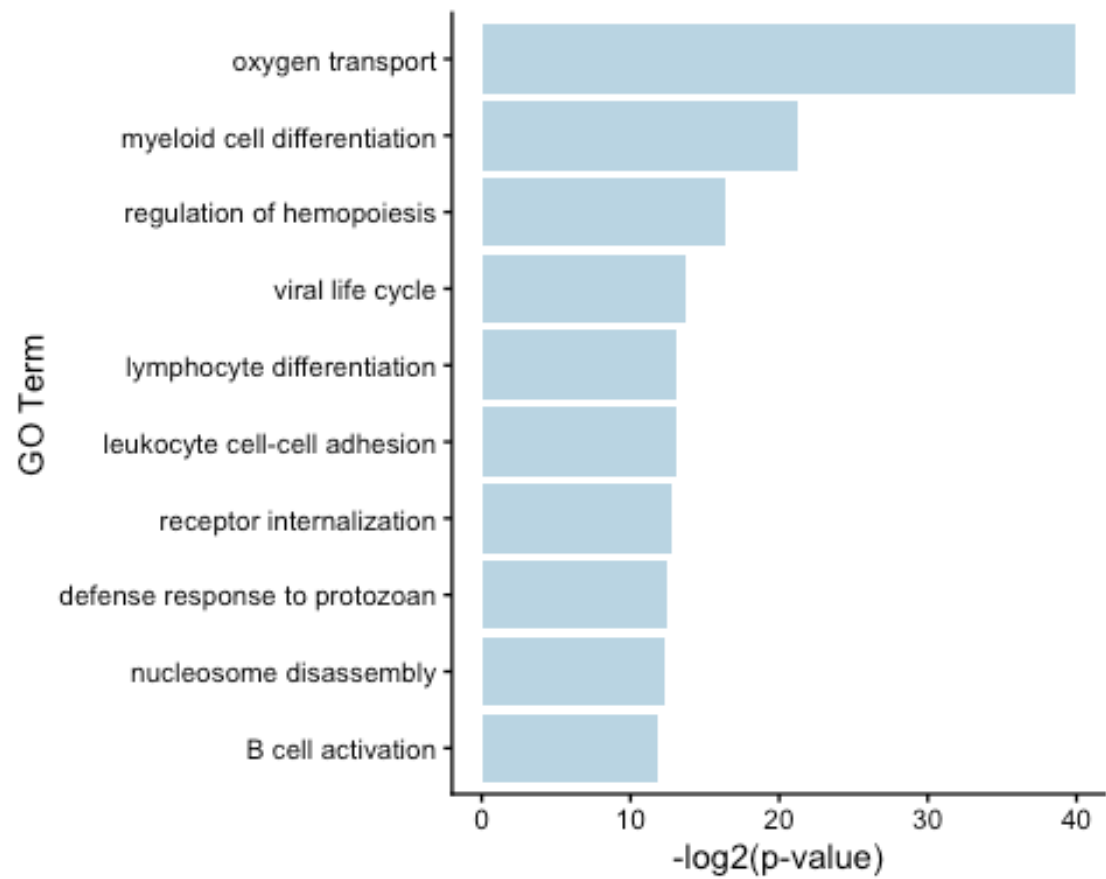
p_GO_combined <- ggplot(GO_enrich_selectAll_combined,
  aes(x = nlog2p, y = Description, fill = Type)) +
  geom_col() +
  facet_wrap(~ Type, ncol = 1, scales = "free_y") +
  theme_classic() +
  scale_fill_manual(values = c("scRNA" = "#F8B191", "scATAC" = "#B9D4E2")) +
  labs(x = "-log2(p-value)", y = "GO Term") +
  theme(
    axis.text.x = element_text(angle = 0, hjust = 0.5),
    legend.position = "none"
  )

p_GO_RNA_enrich_all_top10

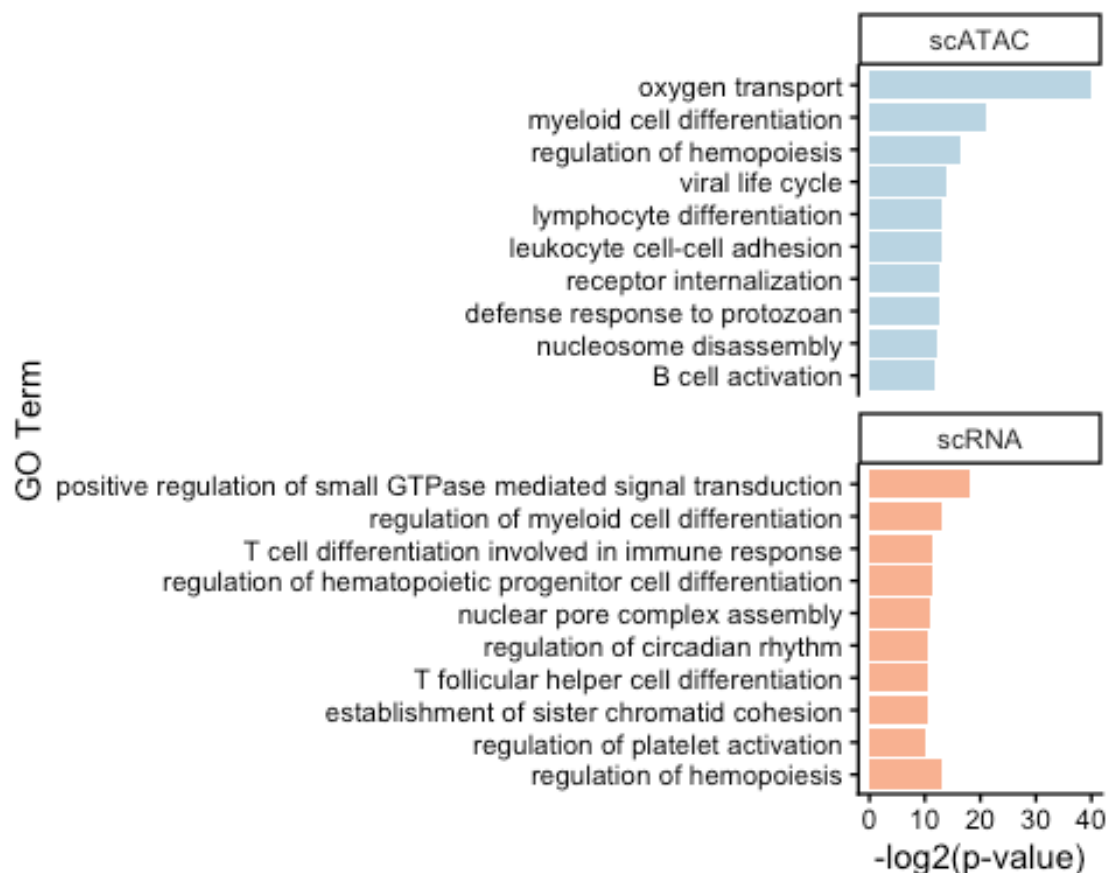
```



p_GO_ATA_enrich_all_top10



p_GO_combined



percentage celltype per sample across stages

```
AML_RNA_meta_merge <- AML_scRNA_meta
AML_ATA_meta_merge <- AML_scATA_meta

AML_ATA_meta_merge$SampleID <- sub("_.*", "", AML_ATA_meta_merge$Sample)

rownames(AML_ATA_meta_merge) <- sub("^AML[0-9]+\\. ", "",
rownames(AML_ATA_meta_merge))

colnames(AML_ATA_meta_merge) <- c("Library_ID", "TSSEnrichment",
"ReadsInTSS", "ReadsInPromoter",
"ReadsInBlacklist", "PromoterRatio", "PassQC",
"NucleosomeRatio",
"nMultiFrag", "nMonoFrag", "nFrag", "nDiFrag",
"BlacklistRatio",
"DoubletScore", "DoubletEnrichment", "Clusters",
"ReadsInPeaks",
"FRIP", "predictedCell_Un", "Classified_Celltype",
"predictedScore_Un", "predictedCell_RNAQun",
"Lambo_et_al_ID")
```

```

# ATA
AML_ATA_celltype_percentage <- AML_ATA_meta_merge %>%
  dplyr::count(Library_ID, Classified_Celltype, name = "cell_count") %>%
  dplyr::group_by(Library_ID) %>%
  dplyr::mutate(
    total_cells = sum(cell_count),
    percentage = cell_count / total_cells * 100
  ) %>%
  dplyr::ungroup()

AML_RNA_celltype_percentage <- AML_RNA_meta_merge %>%
  dplyr::count(Library_ID, Classified_Celltype, name = "cell_count") %>%
  dplyr::group_by(Library_ID) %>%
  dplyr::mutate(
    total_cells = sum(cell_count),
    percentage = cell_count / total_cells * 100
  ) %>%
  dplyr::ungroup()

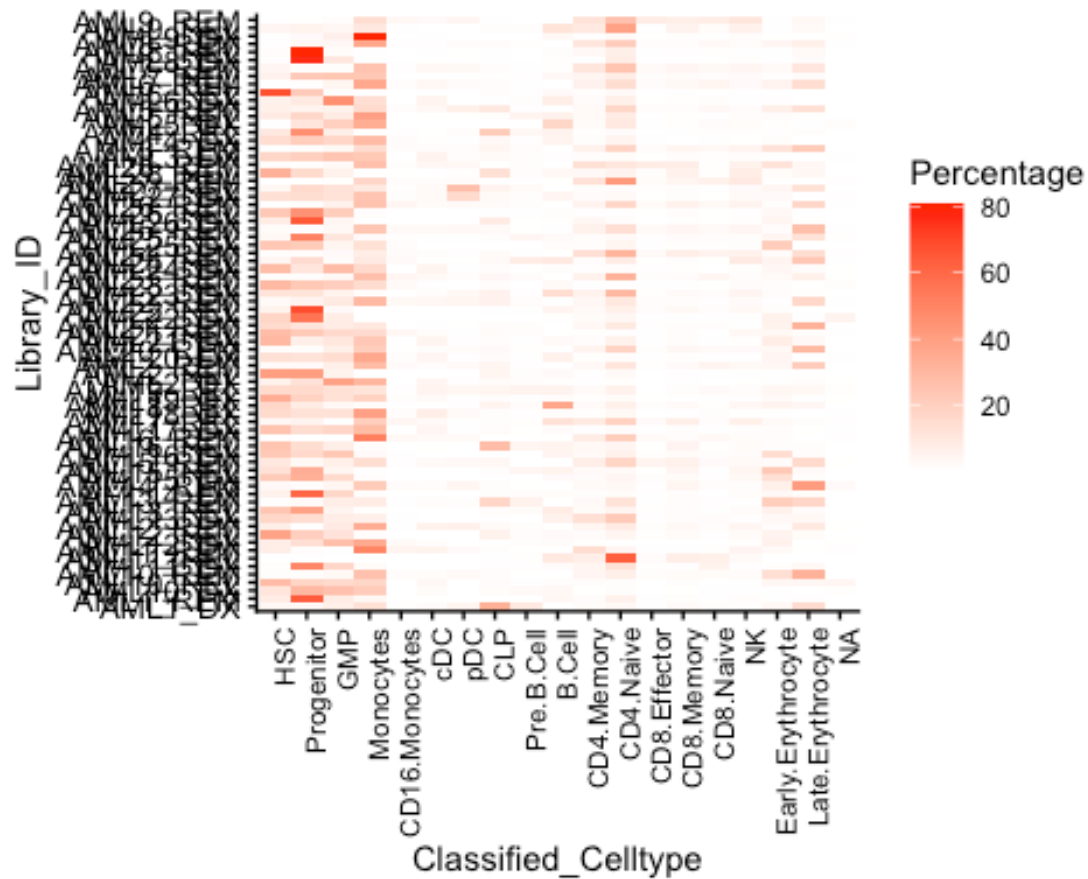
cellTypes_order <- c(
  "HSC", "Progenitor", "GMP", "Monocytes", "CD16.Monocytes", "cDC", "pDC",
  "CLP", "Pre.B.Cell", "B.Cell", "CD4.Memory", "CD4.Naive", "CD8.Effector",
  "CD8.Memory", "CD8.Naive", "NK",
  "Early.Erythrocyte", "Late.Erythrocyte", "Unknown"
)

AML_RNA_celltype_percentage$Classified_Celltype <- factor(
  AML_RNA_celltype_percentage$Classified_Celltype,
  levels = cellTypes_order
)

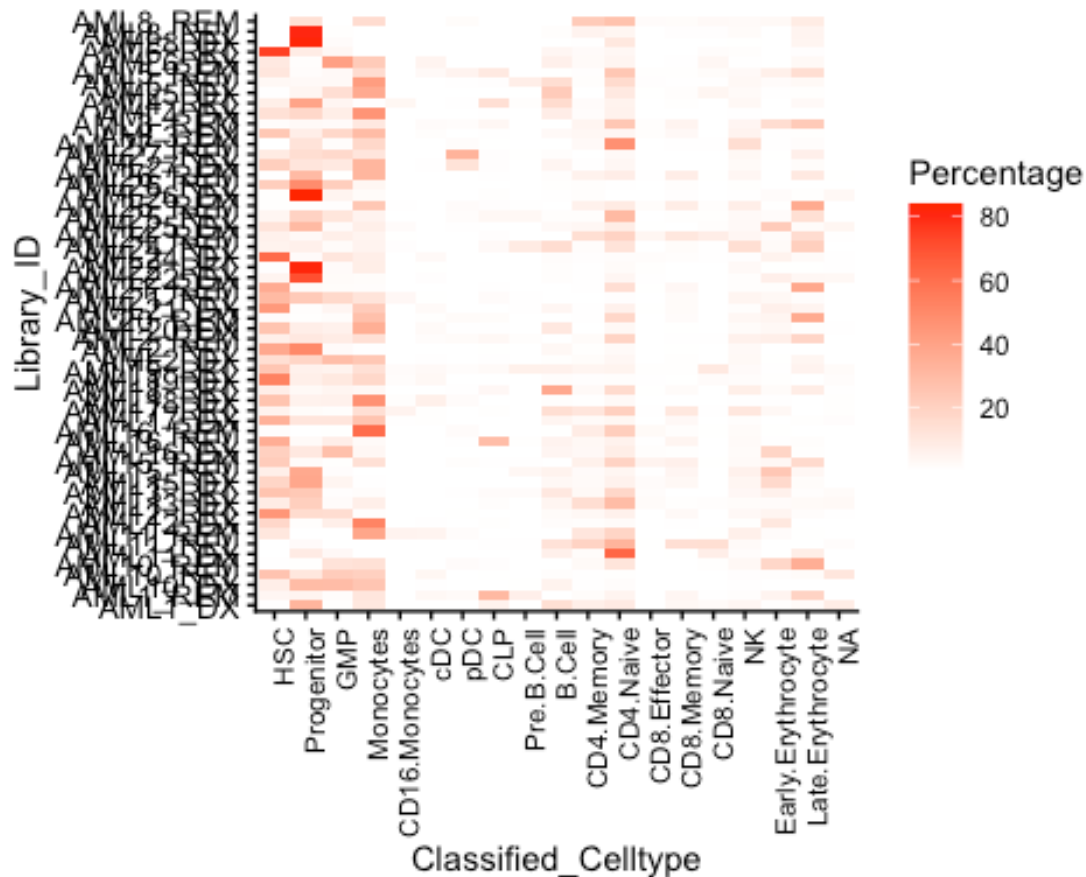
AML_ATA_celltype_percentage$Classified_Celltype <- factor(
  AML_ATA_celltype_percentage$Classified_Celltype,
  levels = cellTypes_order
)

ggplot(AML_RNA_celltype_percentage, aes(x = Classified_Celltype, y =
Library_ID, fill = percentage)) +
  geom_tile() +
  scale_fill_gradient(name = "Percentage", low = "white", high = "red") +
  theme_classic() +
  theme(
    axis.text.x = element_text(angle = 90, hjust = 1),
    axis.text.y = element_text(size = 10)
  )

```



```
ggplot(AML_ATA_celltype_percentage, aes(x = Classified_Celltype, y =
Library_ID, fill = percentage)) +
  geom_tile() +
  scale_fill_gradient(name = "Percentage", low = "white", high = "red") +
  theme_classic() +
  theme(
    axis.text.x = element_text(angle = 90, hjust = 1),
    axis.text.y = element_text(size = 10)
  )
```



```
# main

HSPC = c("HSC", "Progenitor")
Myeloid = c("GMP", "Monocytes", "CD16.Monocytes", "cDC", "pDC")
Lymphoid = c("CLP", "Pre.B.Cell", "B.Cell", "CD4.Memory", "CD4.Naive",
"CD8.Effector", "CD8.Memory", "CD8.Naive", "NK")
Eryth = c("Early.Erythrocyte", "Late.Erythrocyte")

AML_ATA_meta_merge_main <- AML_ATA_meta_merge %>%
  mutate(
    Main_celltype = case_when(
      Classified_Celltype %in% HSPC ~ "HSPC",
      Classified_Celltype %in% Myeloid ~ "Myeloid",
      Classified_Celltype %in% Lymphoid ~ "Lymphoid",
      Classified_Celltype %in% Eryth ~ "Eryth",
      TRUE ~ "Other" # 如果有未分类的细胞类型
    )
  )
```

```

AML_RNA_meta_merge_main <- AML_RNA_meta_merge %>%
  mutate(
    Main_celltype = case_when(
      Classified_Celltype %in% HSPC ~ "HSPC",
      Classified_Celltype %in% Myeloid ~ "Myeloid",
      Classified_Celltype %in% Lymphoid ~ "Lymphoid",
      Classified_Celltype %in% Eryth ~ "Eryth",
      TRUE ~ "Other"
    )
  )

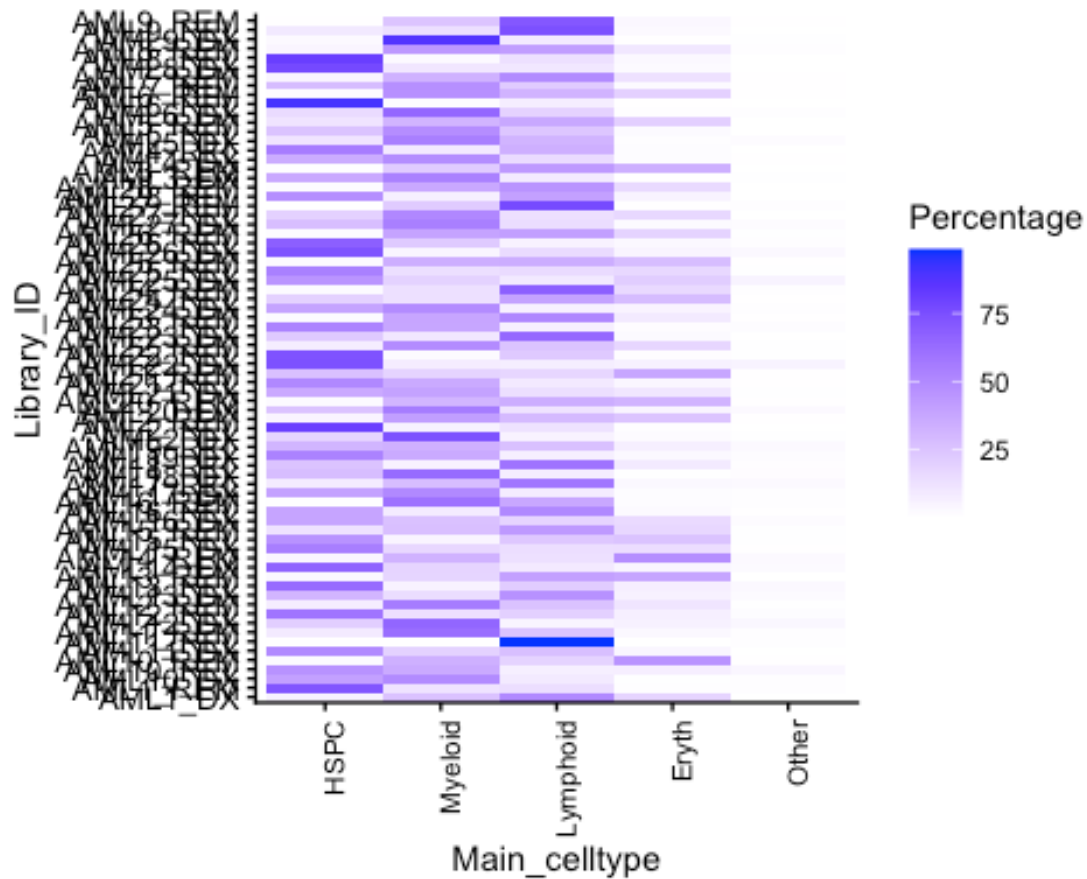
AML_ATA_Maincelltype_percentage <- AML_ATA_meta_merge_main %>%
  dplyr::count(Library_ID, Main_celltype, name = "cell_count") %>%
  dplyr::group_by(Library_ID) %>%
  dplyr::mutate(
    total_cells = sum(cell_count),
    percentage = cell_count / total_cells * 100
  ) %>%
  dplyr::ungroup()

AML_RNA_Maincelltype_percentage <- AML_RNA_meta_merge_main %>%
  dplyr::count(Library_ID, Main_celltype, name = "cell_count") %>%
  dplyr::group_by(Library_ID) %>%
  dplyr::mutate(
    total_cells = sum(cell_count),
    percentage = cell_count / total_cells * 100
  ) %>%
  dplyr::ungroup()

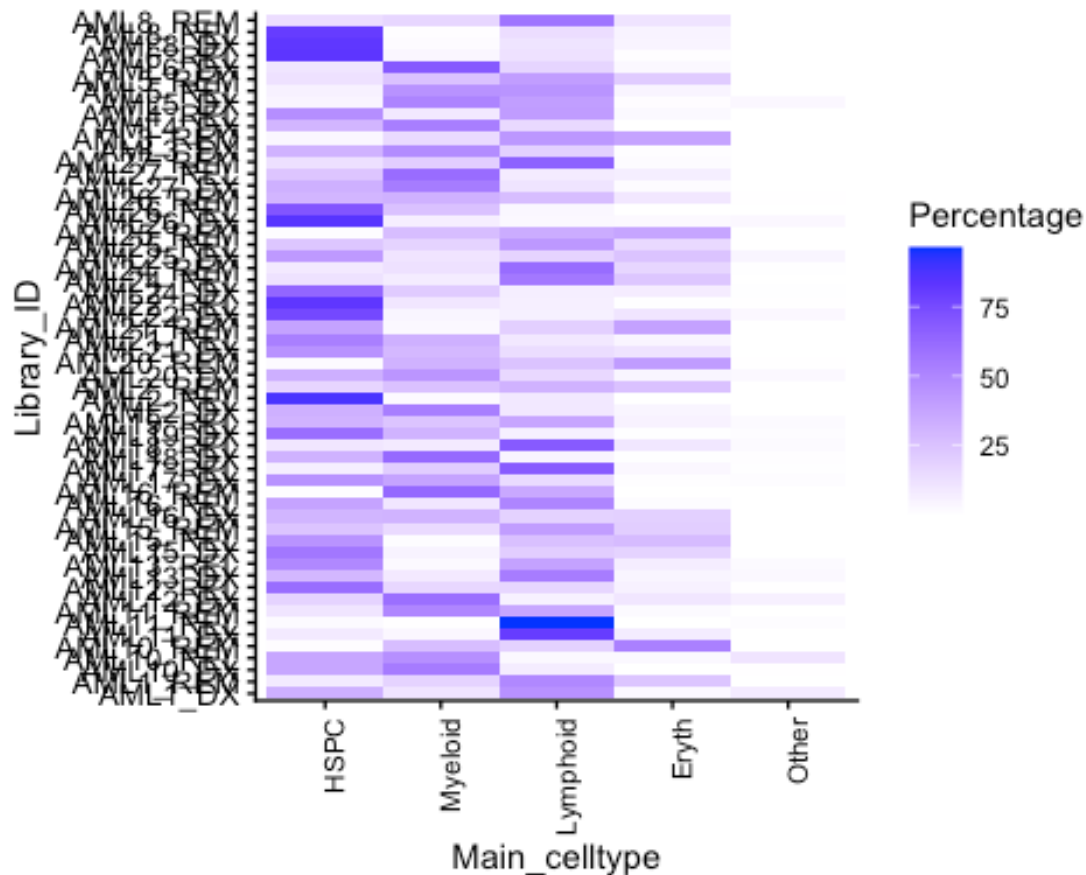
mainCellTypes_order <- c("HSPC", "Myeloid", "Lymphoid", "Eryth", "Other")
AML_RNA_Maincelltype_percentage$Main_celltype <- factor(
  AML_RNA_Maincelltype_percentage$Main_celltype,
  levels = mainCellTypes_order
)
AML_ATA_Maincelltype_percentage$Main_celltype <- factor(
  AML_ATA_Maincelltype_percentage$Main_celltype,
  levels = mainCellTypes_order
)

ggplot(AML_RNA_Maincelltype_percentage, aes(x = Main_celltype, y =
Library_ID, fill = percentage)) +
  geom_tile() +
  scale_fill_gradient(name = "Percentage", low = "white", high = "blue") +
  theme_classic() +
  theme(
    axis.text.x = element_text(angle = 90, hjust = 1),
    axis.text.y = element_text(size = 10)
  )

```

```
ggplot(AML_ATA_Maincelltype_percentage, aes(x = Main_celltype, y =
Library_ID, fill = percentage)) +
  geom_tile() +
  scale_fill_gradient(name = "Percentage", low = "white", high = "blue") +
  theme_classic() +
  theme(
    axis.text.x = element_text(angle = 90, hjust = 1),
    axis.text.y = element_text(size = 10)
  )
```



```
# Main cell type proportion barplot
```

```
AML_RNA_Maincelltype_percentage_data <- AML_RNA_Maincelltype_percentage %>%
mutate(Main_celltype = ifelse(is.na(Main_celltype), "Unknown",
as.character(Main_celltype)))
AML_ATA_Maincelltype_percentage_data <- AML_ATA_Maincelltype_percentage %>%
mutate(Main_celltype = ifelse(is.na(Main_celltype), "Unknown",
as.character(Main_celltype)))
```

```
# keep DX and REL pair samples only
```

```
# RNA
```

```
sample_pair_RNA <- c("AML10", "AML11", "AML12", "AML13", "AML15", "AML16",
"AML17", "AML18", "AML19", "AML21", "AML22", "AML23",
"AML24", "AML25", "AML26", "AML27", "AML2", "AML4",
"AML5", "AML6", "AML8", "AML9")
```

```
sample_pair_ATA <- c("AML10", "AML11", "AML12", "AML13", "AML15", "AML16",
"AML17", "AML18",
"AML19", "AML21", "AML22", "AML24", "AML25", "AML26",
"AML27", "AML2", "AML4", "AML5", "AML6", "AML8")
```

```
sample_pair_RNA_ATA <- intersect(sample_pair_RNA, sample_pair_ATA)
```

```

AML_RNA_Maincelltype_percentage_data_filtered <-
AML_RNA_Maincelltype_percentage_data %>%
  dplyr::filter(gsub("_.*", "", Library_ID) %in% sample_pair_RNA_ATA) %>%
  dplyr::filter(Main_celltype != "Other") %>%
  tidyr::separate(col = Library_ID, into = c("Sample", "Stage"), sep = "_",
remove = FALSE) %>%
  dplyr::filter(Stage %in% c("DX", "REL"))

AML_RNA_Maincelltype_percentage_data_filtered$Stage <- factor(
  AML_RNA_Maincelltype_percentage_data_filtered$Stage,
  levels = c("DX", "REL")
)

# ATA
sample_pair_ATA <- c("AML10", "AML11", "AML12", "AML13", "AML15", "AML16",
"AML17", "AML18",
                    "AML19", "AML21", "AML22", "AML24", "AML25", "AML26",
                    "AML27", "AML2", "AML4", "AML5", "AML6", "AML8")

AML_ATA_Maincelltype_percentage_data_filtered <-
AML_ATA_Maincelltype_percentage_data %>%
  dplyr::filter(gsub("_.*", "", Library_ID) %in% sample_pair_RNA_ATA) %>%
  dplyr::filter(Main_celltype != "Other") %>%
  tidyr::separate(col = Library_ID, into = c("Sample", "Stage"), sep = "_",
remove = FALSE) %>%
  dplyr::filter(Stage %in% c("DX", "REL"))

AML_ATA_Maincelltype_percentage_data_filtered$Stage <- factor(
  AML_ATA_Maincelltype_percentage_data_filtered$Stage,
  levels = c("DX", "REL")
)

AML_RNA_Maincelltype_percentage_data_filtered$Main_celltype <- factor(
  AML_RNA_Maincelltype_percentage_data_filtered$Main_celltype,
  levels = c("HSPC", "Myeloid", "Lymphoid", "Eryth")
)

RNA_sample_level <- c("AML4", "AML16", "AML11", "AML17", "AML18", "AML24",
"AML10", "AML12", "AML13", "AML15", "AML19", "AML21", "AML22", "AML25",
"AML26", "AML27", "AML2", "AML5", "AML6", "AML8")

AML_RNA_Maincelltype_percentage_data_filtered$Sample <- factor(
  AML_RNA_Maincelltype_percentage_data_filtered$Sample,
  levels = RNA_sample_level
)

```

```

AML_ATA_Maincelltype_percentage_data_filtered <-
  AML_ATA_Maincelltype_percentage_data_filtered %>%
  dplyr::add_row(
    Library_ID = "AML11_REL",
    Sample      = "AML11",
    Stage       = "REL",
    Main_celltype = "Eryth",
    cell_count   = 0L,
    total_cells  = 3686L,
    percentage   = 0
  )

AML_ATA_Maincelltype_percentage_data_filtered$Main_celltype <- factor(
  AML_ATA_Maincelltype_percentage_data_filtered$Main_celltype,
  levels = c("HSPC", "Myeloid", "Lymphoid", "Eryth")
)

ATA_sample_level <- c("AML4", "AML16", "AML11", "AML17", "AML18", "AML24",
  "AML10", "AML12", "AML13", "AML15", "AML19", "AML21",
  "AML22",
  "AML25", "AML26", "AML27", "AML2", "AML5",
  "AML6", "AML8")

AML_ATA_Maincelltype_percentage_data_filtered$Sample <- factor(
  AML_ATA_Maincelltype_percentage_data_filtered$Sample,
  levels = ATA_sample_level
)

myColors <- colorRampPalette(brewer.pal(11, "RdBu"))(100)

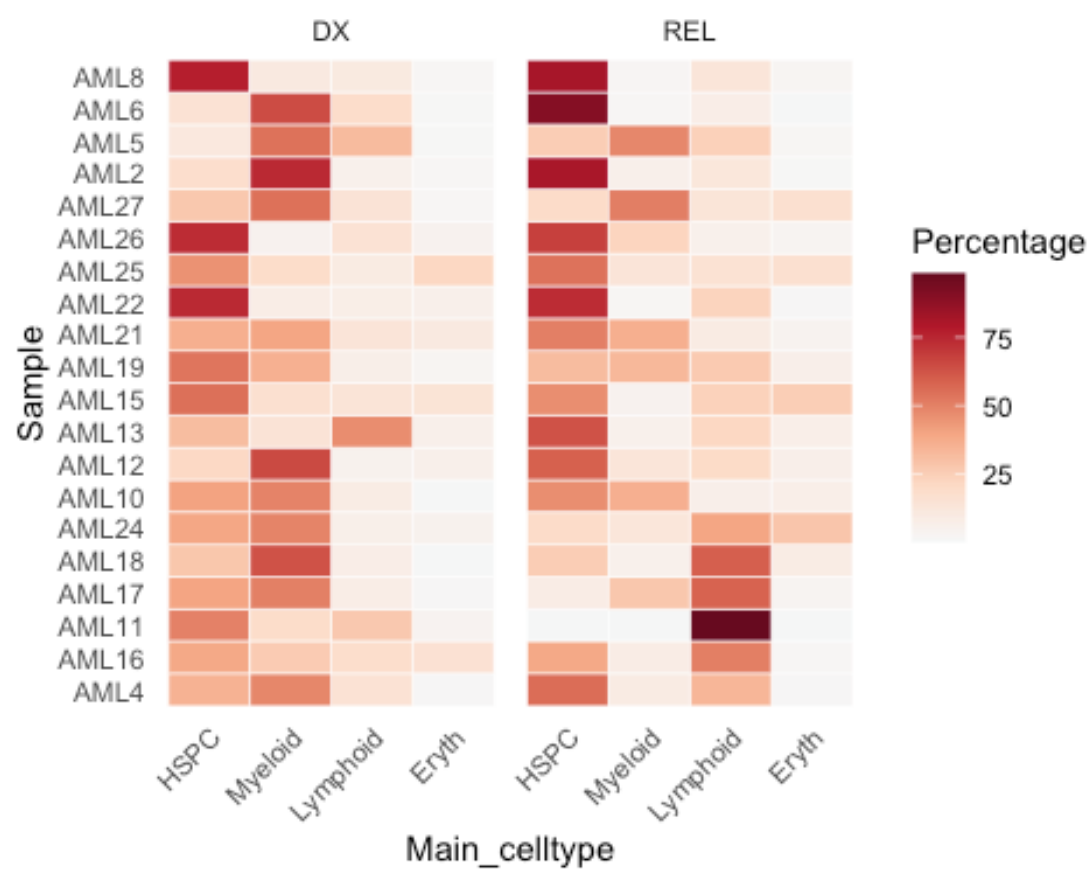
p_RNA_percentage <- ggplot(AML_RNA_Maincelltype_percentage_data_filtered,
  aes(x = Main_celltype, y = Sample, fill = percentage)) +
  geom_tile(color = "white", linewidth = 0.2) +
  facet_wrap(~ Stage) +
  scale_y_discrete(limits = RNA_sample_level) +
  scale_fill_gradientn(colors = rev(myColors)[50:100], name = "Percentage") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid = element_blank())

p_ATA_percentage <- ggplot(AML_ATA_Maincelltype_percentage_data_filtered,
  aes(x = Main_celltype, y = Sample, fill = percentage)) +
  geom_tile(color = "white", linewidth = 0.2) +
  facet_wrap(~ Stage) +
  scale_y_discrete(limits = ATA_sample_level) +
  scale_fill_gradientn(colors = myColors[50:100], name = "Percentage") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),

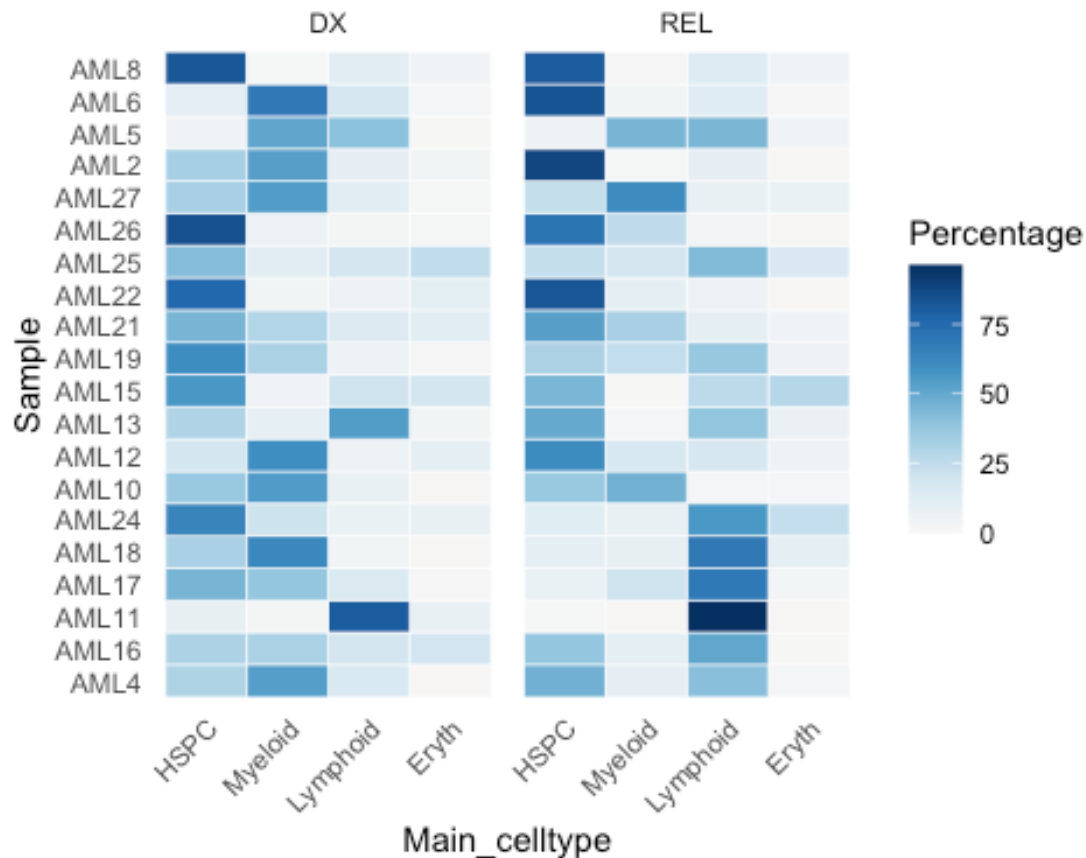
```

```
panel.grid = element_blank()
```

p_RNA_percentage



p_ATA_percentage



```

AML_ATA_Maincelltype_percentage_data_filtered$Tag <-
paste(AML_ATA_Maincelltype_percentage_data_filtered$Library_ID,

AML_ATA_Maincelltype_percentage_data_filtered$Main_celltype,
      sep = "_")

AML_RNA_Maincelltype_percentage_data_filtered$Tag <-
paste(AML_RNA_Maincelltype_percentage_data_filtered$Library_ID,

AML_RNA_Maincelltype_percentage_data_filtered$Main_celltype,
      sep = "_")

# merge
merged_percentage_data <- merge(
  AML_RNA_Maincelltype_percentage_data_filtered,
  AML_ATA_Maincelltype_percentage_data_filtered,
  by = "Tag",
  suffixes = c("_RNA", "_ATA")
)

p_similarity_celltypePercentage <- ggplot(data = merged_percentage_data,

```

```

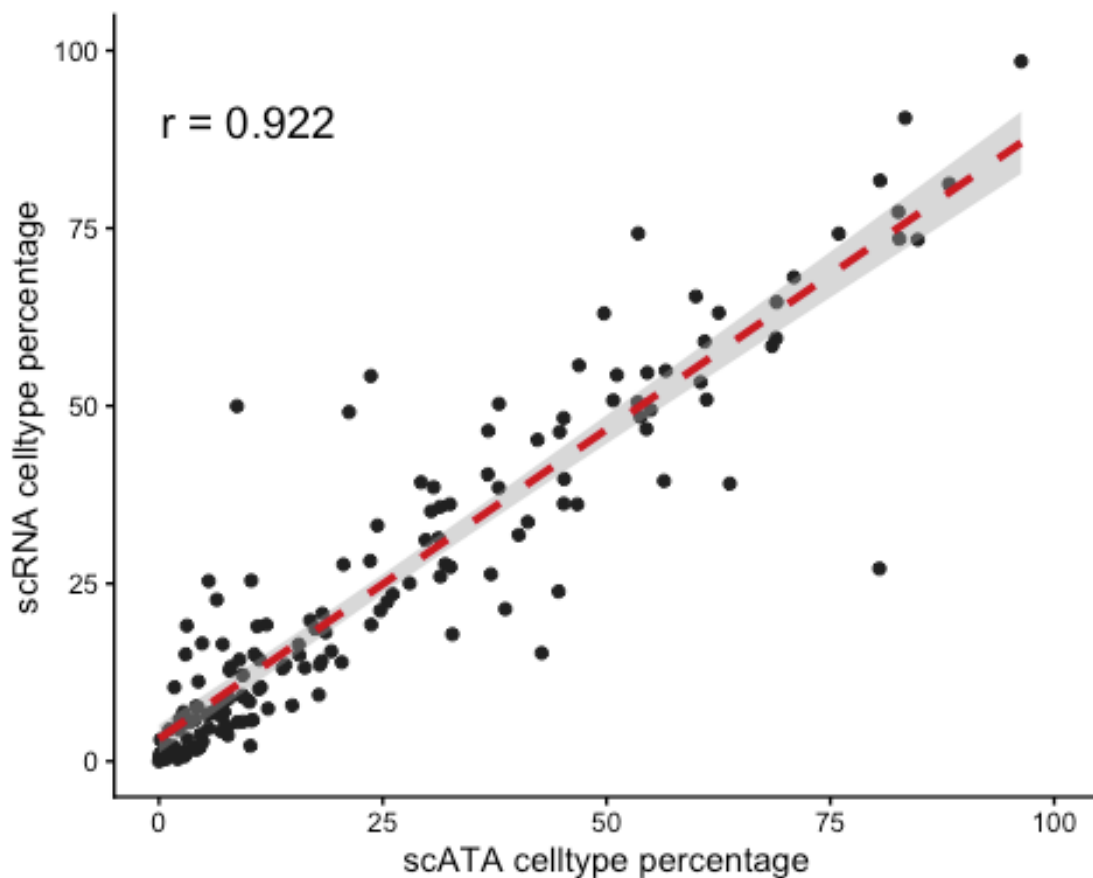
    aes(x = percentage_ATA, y = percentage_RNA)) +
  geom_point(size = 1.5, color = "#202020") +
  geom_smooth(method = "lm", se = TRUE, color = "#ca0020", linetype =
"dashed", size = 1.2) +
  theme_classic() +
  xlab("scATA celltype percentage") +
  ylab("scRNA celltype percentage") +
  xlim(0, 100) + ylim(0, 100) +
  annotate("text", x = 10, y = 90,
    label = paste0("r = ",
round(cor(merged_percentage_data$percentage_ATA,
merged_percentage_data$percentage_RNA), 3)),
    size = 5)

```

```

p_similarity_celltypePercentage
## `geom_smooth()` using formula = 'y ~ x'

```



```

cor.test(merged_percentage_data$percentage_ATA,
merged_percentage_data$percentage_RNA, method = "pearson")
##
##  Pearson's product-moment correlation
##

```

```
## data: merged_percentage_data$percentage_ATA and
merged_percentage_data$percentage_RNA
## t = 29.833, df = 158, p-value < 2.2e-16
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## 0.8942463 0.9420069
## sample estimates:
## cor
## 0.9215411
```

transcription factor lineage analysis

```
plot_list_lineageSwitching <- function(GIMPath, MMPPath, colOrder = NA){
  message("Function to plot lineage switching heatmaps for GIM and MM data.")
  if (all(is.na(colOrder))) {
    colOrder <- seq_len(ncol(GIM_data_mat))
  }

  message("Reading GIM data from: ", GIMPath)
  GIM_data <- readRDS(GIMPath)
  GIM_data_mat <- GIM_data$assayMat
  #GIM_data_mat <- pmin(pmax(GIM_data_mat, -1.5), 1.5)
  rownames(GIM_data_mat) <- sub("chr[0-9XY]+:", "", rownames(GIM_data_mat))
  colnames(GIM_data_mat) <- paste(GIM_data$assayColData$label,
1:length(GIM_data$assayColData$label), sep = "_")
  message("Reading MM data from: ", MMPPath)
  MM_data <- readRDS(MMPPath)
  MM_data_mat <- MM_data$assayMat
  #MM_data_mat <- pmin(pmax(MM_data_mat, -1.5), 1.5)
  rownames(MM_data_mat) <- sub("^z:([A-Za-z0-9]+)_.*$", "\\1",
rownames(MM_data_mat))
  colnames(MM_data_mat) <- paste(MM_data$assayColData$label,
1:length(MM_data$assayColData$label), sep = "_")

  final_row_order <- rownames(GIM_data_mat)[row_order(Heatmap(GIM_data_mat))]

  GIM_data_mat <- GIM_data_mat[, colOrder]
  MM_data_mat <- MM_data_mat[, colOrder]

  col_GIM_stage <- sub("^.*_([_^_]+)_.*$", "\\1", colnames(GIM_data_mat))
  ha_col <- HeatmapAnnotation(
    SampleStage = col_GIM_stage,
    col = list(
      SampleStage = c(
        "DX" = "#33a02c",
        "REL" = "#E31A1C"
      )
    )
  )
}
```



```

ht_GIM_final <- Heatmap(GIM_data_mat[final_row_order,],
  cluster_rows = FALSE,
  cluster_columns = FALSE,
  top_annotation = ha_col,
  col = rev(colorRampPalette(brewer.pal(11,
"RdBu"))(100))[50:100])

ht_MM_final <- Heatmap(MM_data_mat[final_row_order,],
  cluster_rows = FALSE,
  cluster_columns = FALSE,
  top_annotation = ha_col,
  col = colorRampPalette(brewer.pal(11, "RdBu"))(100)[50:100])

return(list(ht_GIM_final = ht_GIM_final,
  ht_MM_final = ht_MM_final,
  GIM_mat = GIM_data_mat,
  MM_mat = MM_data_mat,
  genes = final_row_order))
}

rowMeans_no_outlier <- function(mat) {
  apply(mat, 1, function(x) {
    q <- quantile(x, probs = c(0.25, 0.75), na.rm = TRUE)
    iqr <- q[2] - q[1]
    x[x >= (q[1] - 1.5 * iqr) & x <= (q[2] + 1.5 * iqr)] |>
      mean(na.rm = TRUE)
  })
}

generate_marker_matrix <- function(marker_gene, matMatrix, iteration_times =
100, iteration_size = 50, dx_pattern = "DX", rel_pattern = "REL"){
  matMatrix_marker <- matMatrix[marker_gene, ]
  cells_DX <- grep(dx_pattern, colnames(matMatrix_marker), value = TRUE)
  cells_REL <- grep(rel_pattern, colnames(matMatrix_marker), value = TRUE)

  set.seed(2025)
  iteration_DX <- list()
  iteration_REL <- list()
  for (i in 1:iteration_times) {
    sampled_cells_DX <- sample(cells_DX, size = iteration_size, replace =
FALSE)
    iteration_DX[[i]] <-
rowMeans_no_outlier(matMatrix_marker[,sampled_cells_DX, drop = FALSE])
    sampled_cells_REL <- sample(cells_REL, size = iteration_size, replace =
FALSE)
    iteration_REL[[i]] <-
rowMeans_no_outlier(matMatrix_marker[,sampled_cells_REL, drop = FALSE])

```

```

}

return(list(DX = iteration_DX,
            REL = iteration_REL))
}

range_scale <- function(x, min_val = -1.5, max_val = 1.5) {
  x_min <- min(x, na.rm = TRUE)
  x_max <- max(x, na.rm = TRUE)

  (x - x_min) / (x_max - x_min) * (max_val - min_val) + min_val
}

# paths
GIM_AML4_Path <- "../Data/AML/p8_RDS_GIM2_AML4_M.rds"
MM_AML4_Path <- "../Data/AML/p8_RDS_MM2_AML4_M.rds"
GIM_AML16_Path <- "../Data/AML/p8_RDS_GIM2_AML16_M.rds"
MM_AML16_Path <- "../Data/AML/p8_RDS_MM2_AML16_M.rds"
GIM_AML11_Path <- "../Data/AML/p8_RDS_GIM2_AML18_M.rds"
MM_AML11_Path <- "../Data/AML/p8_RDS_MM2_AML18_M.rds"
GIM_AML17_Path <- "../Data/AML/p8_RDS_GIM2_AML17_M.rds"
MM_AML17_Path <- "../Data/AML/p8_RDS_MM2_AML17_M.rds"
GIM_AML18_Path <- "../Data/AML/p8_RDS_GIM2_AML18_M.rds"
MM_AML18_Path <- "../Data/AML/p8_RDS_MM2_AML18_M.rds"
GIM_AML24_Path <- "../Data/AML/p8_RDS_GIM2_AML24_M.rds"
MM_AML24_Path <- "../Data/AML/p8_RDS_MM2_AML24_M.rds"

colOrderQ <- c(1:100)
lineage_AML4 <- plot_list_lineageSwitching(GIM_AML4_Path, MM_AML4_Path,
colOrderQ)
## Function to plot Lineage switching heatmaps for GIM and MM data.
## Reading GIM data from: ../Data/AML/p8_RDS_GIM2_AML4_M.rds
## Reading MM data from: ../Data/AML/p8_RDS_MM2_AML4_M.rds
## Warning: The input is a data frame-like object, convert it to a matrix.
## Warning: The heatmap has not been initialized. You might have different
results
## if you repeatedly execute this function, e.g. when row_km/column_km was
## set. It is more suggested to do as `ht = draw(ht); row_order(ht)`.
## Warning: The input is a data frame-like object, convert it to a matrix.
## Warning: The input is a data frame-like object, convert it to a matrix.

colOrderQ <- c(1:100)
lineage_AML16 <- plot_list_lineageSwitching(GIM_AML16_Path, MM_AML16_Path,
colOrderQ)
## Function to plot Lineage switching heatmaps for GIM and MM data.
## Reading GIM data from: ../Data/AML/p8_RDS_GIM2_AML16_M.rds
## Reading MM data from: ../Data/AML/p8_RDS_MM2_AML16_M.rds
## Warning: The input is a data frame-like object, convert it to a matrix.

```

```

## Warning: The heatmap has not been initialized. You might have different results
## if you repeatedly execute this function, e.g. when row_km/column_km was
## set. It is more suggested to do as `ht = draw(ht); row_order(ht)`.
## Warning: The input is a data frame-like object, convert it to a matrix.
## Warning: The input is a data frame-like object, convert it to a matrix.

colOrderQ <- c(1:100)
lineage_AML11 <- plot_list_lineageSwitching(GIM_AML11_Path, MM_AML11_Path,
colOrderQ)
## Function to plot lineage switching heatmaps for GIM and MM data.
## Reading GIM data from: ../Data/AML/p8_RDS_GIM2_AML18_M.rds
## Reading MM data from: ../Data/AML/p8_RDS_MM2_AML18_M.rds
## Warning: The input is a data frame-like object, convert it to a matrix.
## Warning: The heatmap has not been initialized. You might have different results
## if you repeatedly execute this function, e.g. when row_km/column_km was
## set. It is more suggested to do as `ht = draw(ht); row_order(ht)`.
## Warning: The input is a data frame-like object, convert it to a matrix.
## Warning: The input is a data frame-like object, convert it to a matrix.

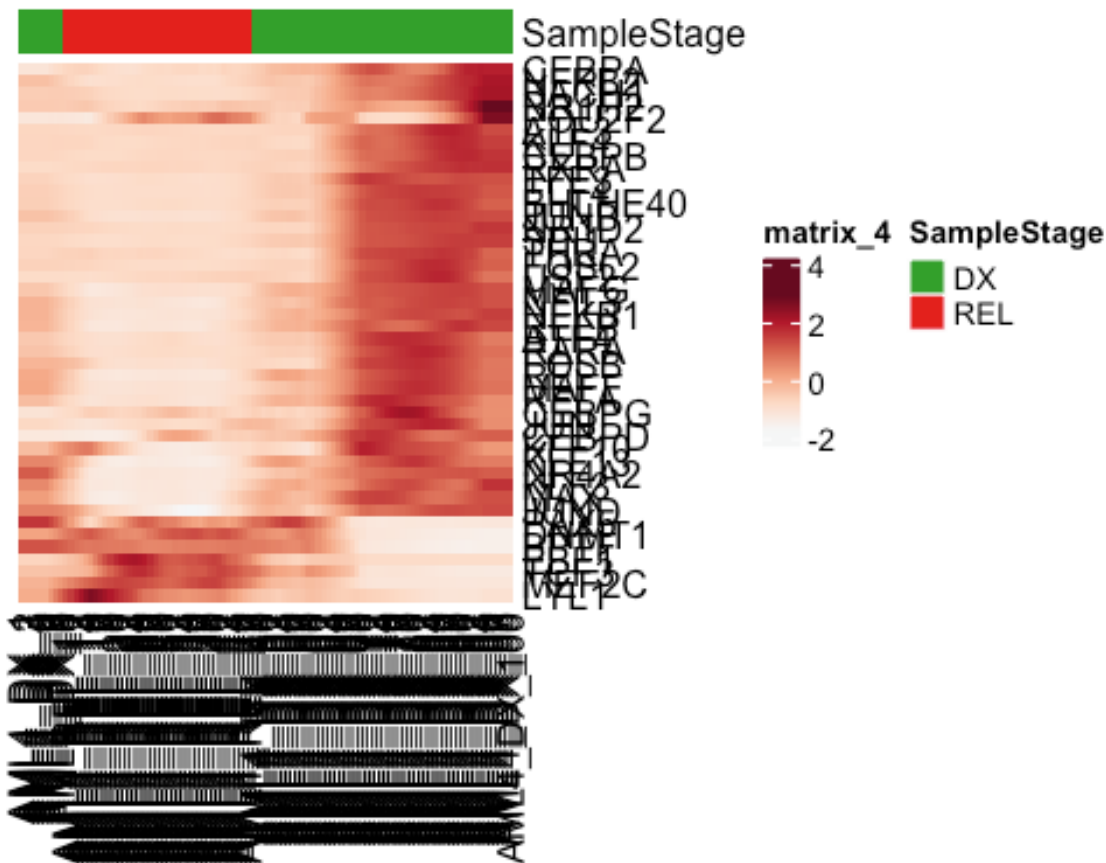
colOrderQ <- c(1:100)
lineage_AML17 <- plot_list_lineageSwitching(GIM_AML17_Path, MM_AML17_Path,
colOrderQ)
## Function to plot lineage switching heatmaps for GIM and MM data.
## Reading GIM data from: ../Data/AML/p8_RDS_GIM2_AML17_M.rds
## Reading MM data from: ../Data/AML/p8_RDS_MM2_AML17_M.rds
## Warning: The input is a data frame-like object, convert it to a matrix.
## Warning: The heatmap has not been initialized. You might have different results
## if you repeatedly execute this function, e.g. when row_km/column_km was
## set. It is more suggested to do as `ht = draw(ht); row_order(ht)`.
## Warning: The input is a data frame-like object, convert it to a matrix.
## Warning: The input is a data frame-like object, convert it to a matrix.

colOrderQ <- c(1:100)
lineage_AML18 <- plot_list_lineageSwitching(GIM_AML18_Path, MM_AML18_Path,
colOrderQ)
## Function to plot Lineage switching heatmaps for GIM and MM data.
## Reading GIM data from: ../Data/AML/p8_RDS_GIM2_AML18_M.rds
## Reading MM data from: ../Data/AML/p8_RDS_MM2_AML18_M.rds
## Warning: The input is a data frame-like object, convert it to a matrix.
## Warning: The heatmap has not been initialized. You might have different results
## if you repeatedly execute this function, e.g. when row_km/column_km was
## set. It is more suggested to do as `ht = draw(ht); row_order(ht)`.
## Warning: The input is a data frame-like object, convert it to a matrix.
## Warning: The input is a data frame-like object, convert it to a matrix.

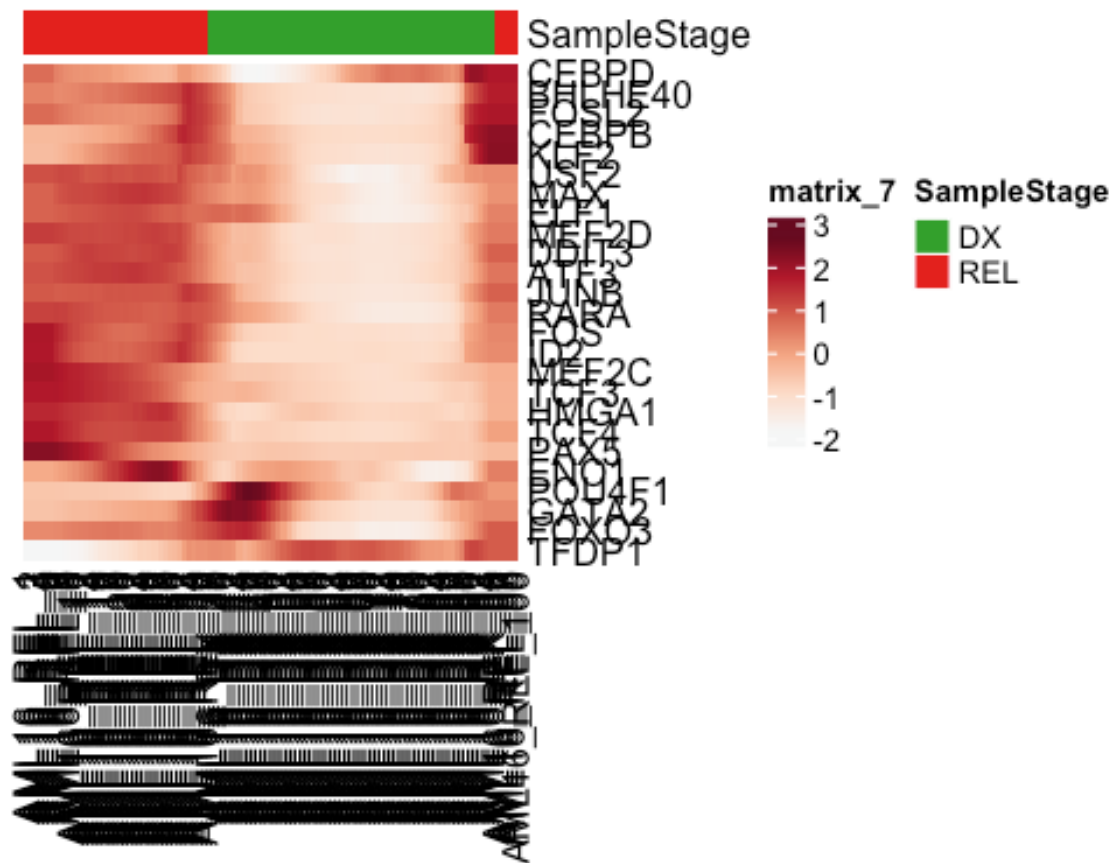
```

```
colOrderQ <- c(1:100)
lineage_AML24 <- plot_list_lineageSwitching(GIM_AML24_Path, MM_AML24_Path,
colOrderQ)
## Function to plot Lineage switching heatmaps for GIM and MM data.
## Reading GIM data from: ../Data/AML/p8_RDS_GIM2_AML24_M.rds
## Reading MM data from: ../Data/AML/p8_RDS_MM2_AML24_M.rds
## Warning: The input is a data frame-like object, convert it to a matrix.
## Warning: The heatmap has not been initialized. You might have different
results
## if you repeatedly execute this function, e.g. when row_km/column_km was
## set. It is more suggested to do as `ht = draw(ht); row_order(ht)`.
## Warning: The input is a data frame-like object, convert it to a matrix.
## Warning: The input is a data frame-like object, convert it to a matrix.

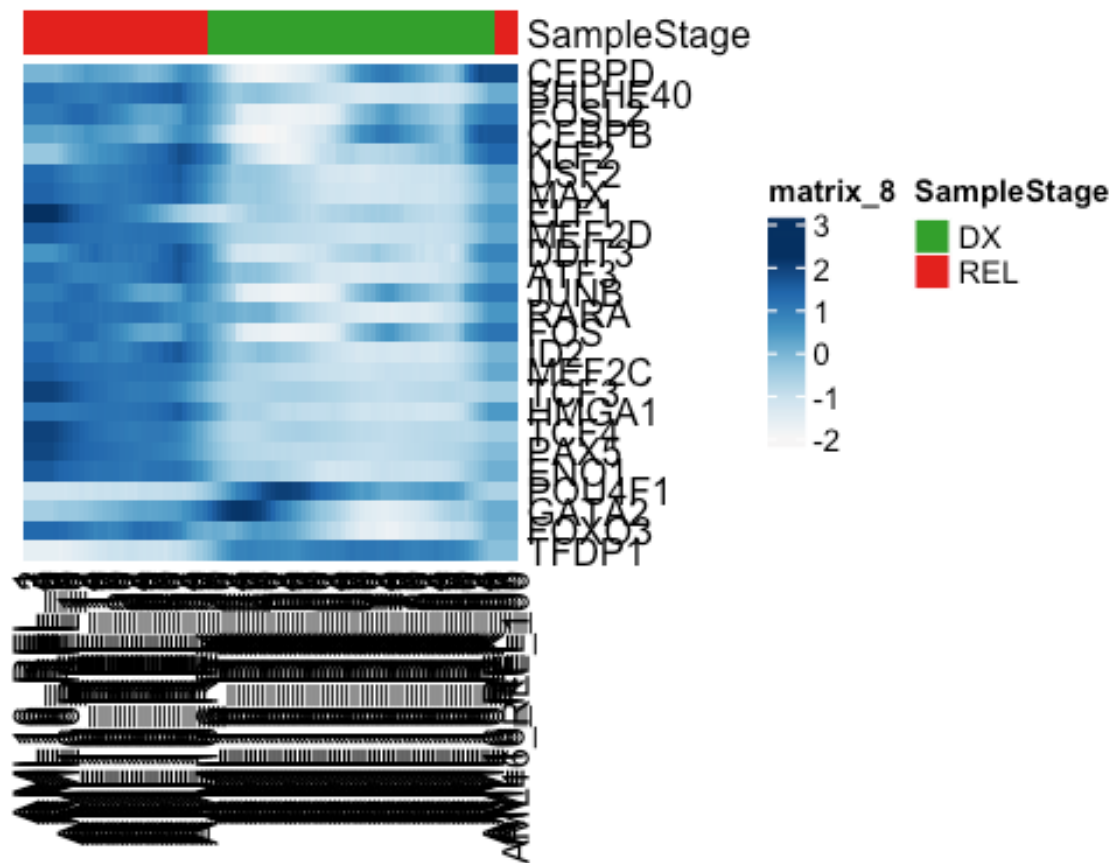
lineage_AML4$ht_GIM_final
```



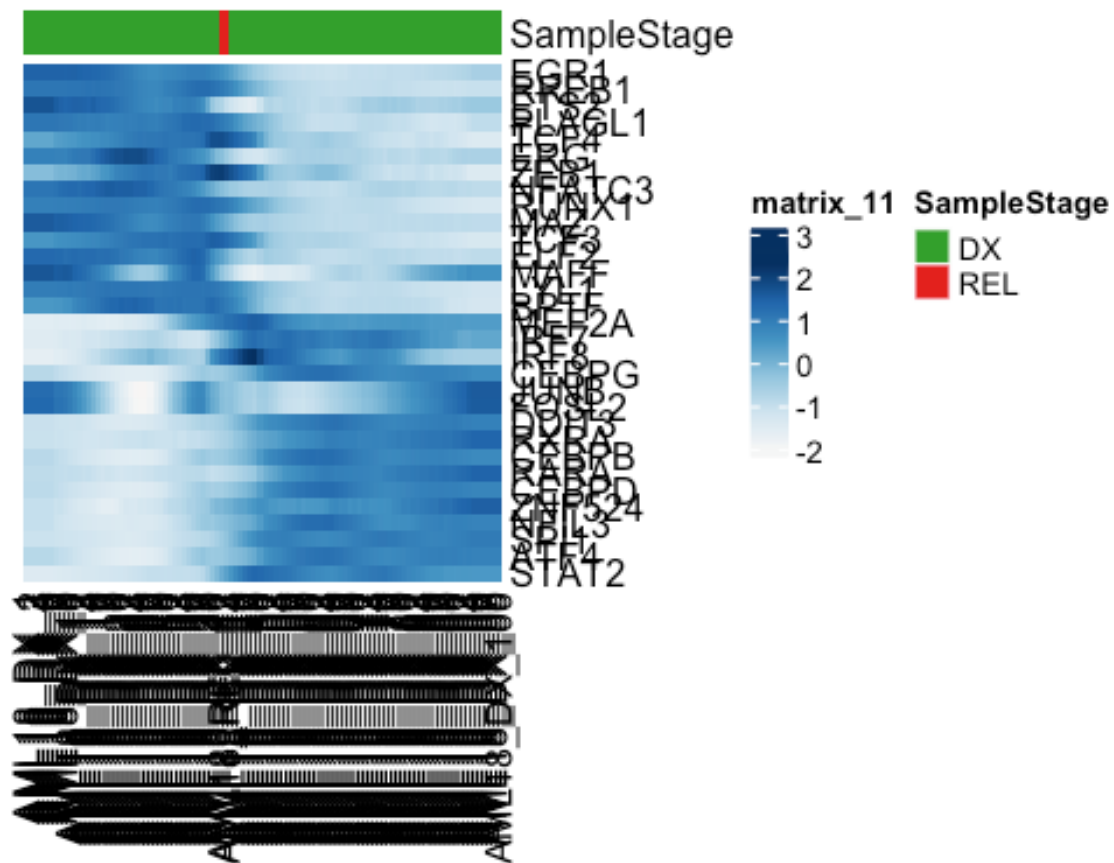
```
lineage_AML4$ht_MM_final
```

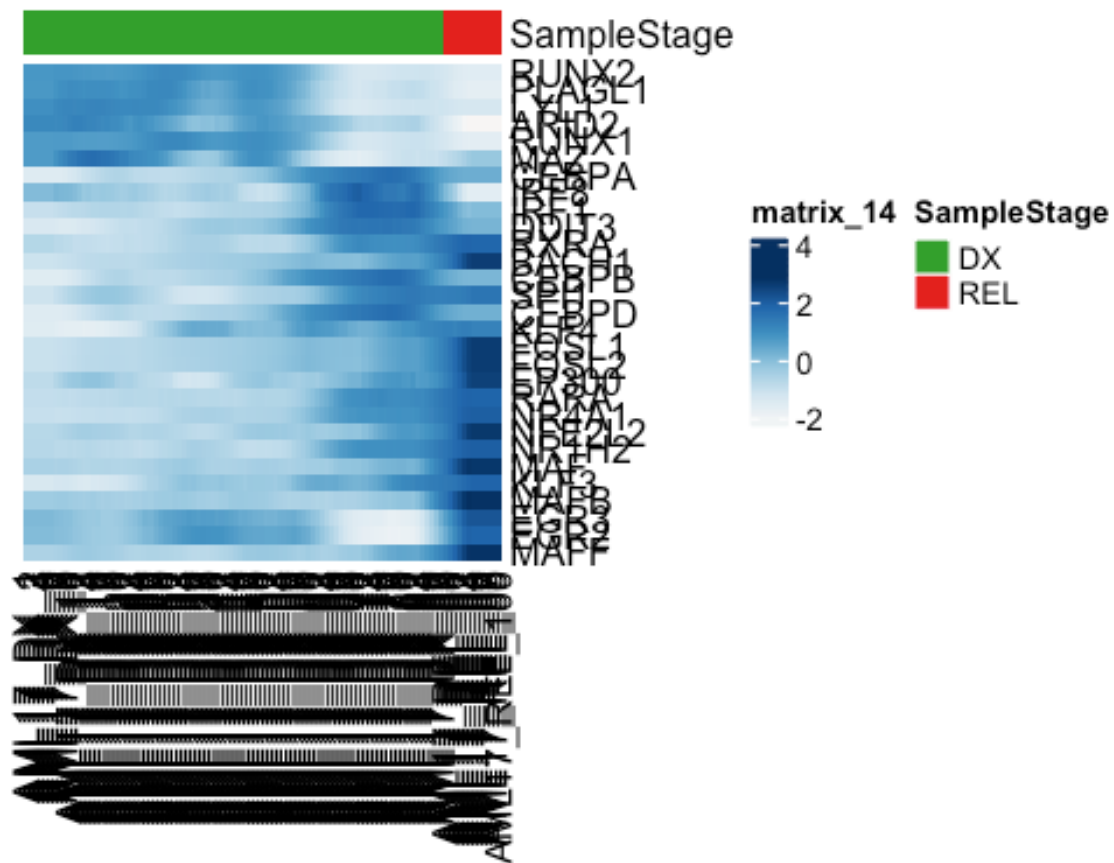
lineage_AML16\$ht_MM_final



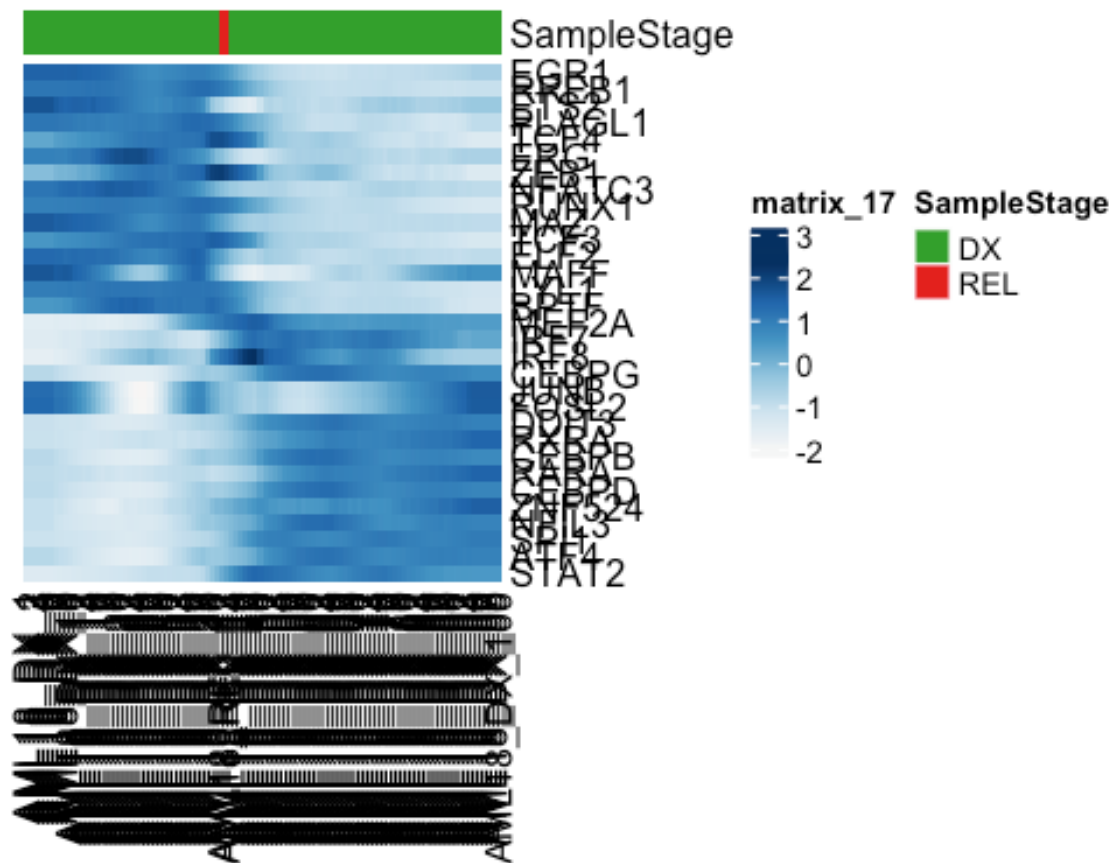
lineage_AML11\$ht_GIM_final



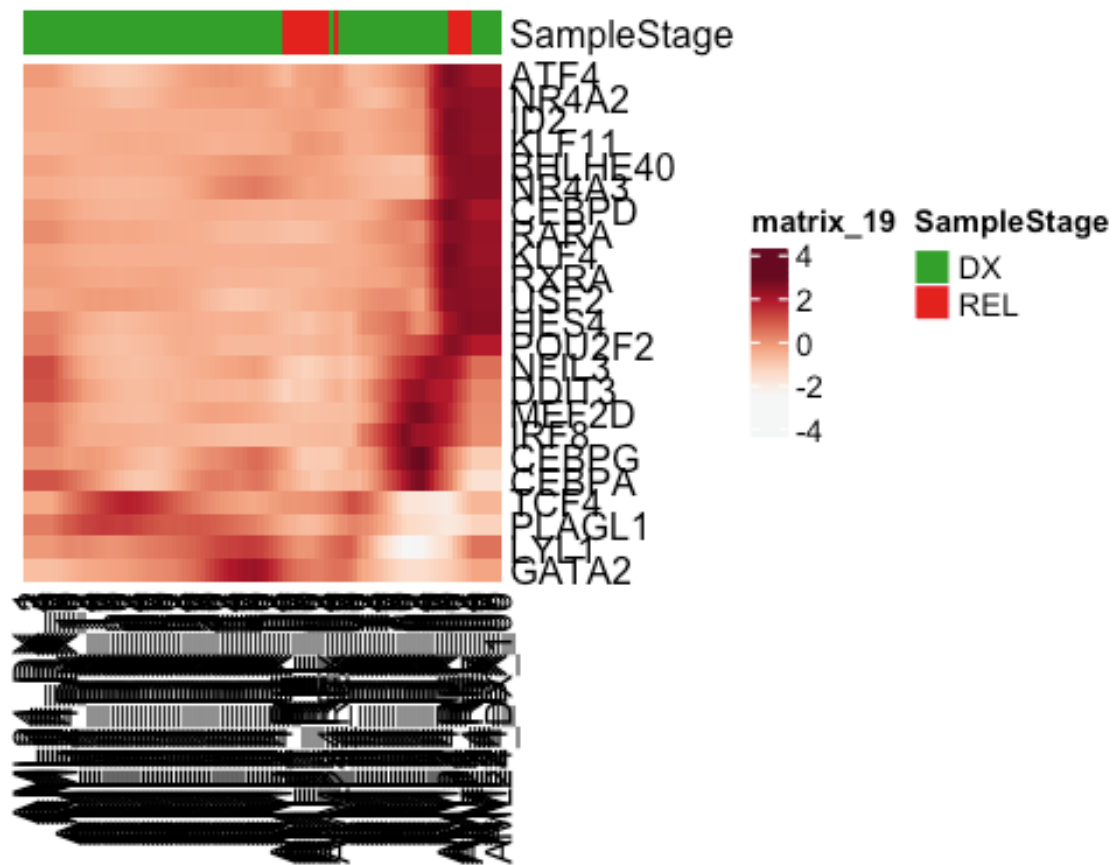
lineage_AML17\$ht_GIM_final

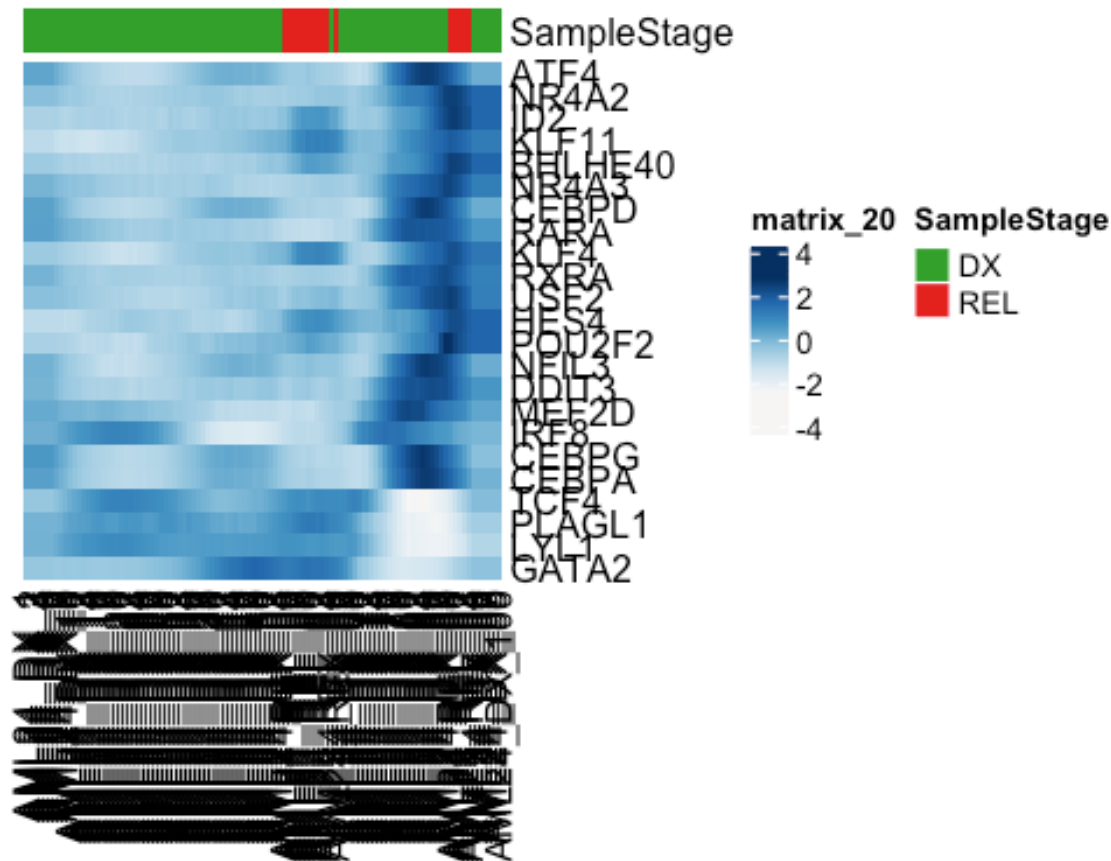


lineage_AML18\$ht_GIM_final



lineage_AML24\$ht_GIM_final





```
# select genes showing lineage switching
lineageSwitching_genes_H <- unique(c("LM02", "RUNX1", "GATA2", "TAL1", "ERG",
"FLI1",
                                "HOXA9"))

lineageSwitching_genes_M <- unique(c("STAT2", "CREM", "SPI1", "KLF2",
"BACH1", "ATF4", "IRF1", "ATF3",
    "JDP2", "NR4A1", "KLF3", "BHLHE40", "CEBPD", "NR4A3", "KLF4", "NR4A2",
"DDIT3", "RXRA", "REL",
    "MAFF", "CEBPA", "NFKB1", "TFEB", "CEBPB", "KLF11", "NR1D2", "JUNB",
"ETV3", "CREB5", "FOSL2",
    "ELF4", "RARA", "FOS", "KLF13", "JUN", "MLX", "NFIL3", "FOSB"))

lineageSwitching_genes_L <- unique(c("USF2", "PAX5", "POU2F1", "IRF7",
"XBP1", "FOXO1", "GFI1", "TCF4",
    "FOXP1", "ETS1", "ID3", "MEF2C", "FOXN3", "IRF4", "EBF1", "TCF7L2",
"IRF2", "TCFL5", "TCF3", "LEF1",
    "SMARCC2", "KLF6", "TCF12", "IRF3", "SP3"))

HSC_marker <- c("HOPX", "SPINK2")
```

```

Progenitor_marker <- c("CD34", "NUCB2", "C1QTNF4", "CDK6", "IGFBP7")
GMP_Myeloid_marker <- c("PRTN3", "ELANE", "MPO", "AZU1", "CTSG")
Monocyte_Myeloid_marker <- c("TYROBP", "FCN1", "VCAN")
B_Lymphoid_marker <- c("CD79A", "CD19", "DNTT", "VPREB3")
T_Lymphoid_marker <- c("IL7R", "LTB", "CD7")

tf_genes <- c(lineageSwitching_genes_H, lineageSwitching_genes_M,
lineageSwitching_genes_L)
marker_genes <- c(HSC_marker, Progenitor_marker, GMP_Myeloid_marker,
Monocyte_Myeloid_marker,
                    B_Lymphoid_marker, T_Lymphoid_marker)

gene_lineage <- data.frame(
  Gene = c(lineageSwitching_genes_H, lineageSwitching_genes_M,
lineageSwitching_genes_L, HSC_marker, Progenitor_marker,
            GMP_Myeloid_marker, Monocyte_Myeloid_marker, B_Lymphoid_marker,
T_Lymphoid_marker),
  Lineage = c(rep("HSPC_TF", length(lineageSwitching_genes_H)),
              rep("Myeloid_TF", length(lineageSwitching_genes_M)),
              rep("Lymphoid_TF", length(lineageSwitching_genes_L)),
              rep("HSC_Marker", length(HSC_marker)),
              rep("Progenitor_Marker", length(Progenitor_marker)),
              rep("Myeloid_Marker_GMP", length(GMP_Myeloid_marker)),
              rep("Myeloid_Marker_Monocyte",
length(Monocyte_Myeloid_marker)),
              rep("Lymphoid_Marker_B", length(B_Lymphoid_marker)),
              rep("Lymphoid_Marker_T", length(T_Lymphoid_marker)))
)

lineageSwitching_genes <- unique(c(lineageSwitching_genes_H,
lineageSwitching_genes_M, lineageSwitching_genes_L,
                                HSC_marker, Progenitor_marker,
GMP_Myeloid_marker,
                                Monocyte_Myeloid_marker, B_Lymphoid_marker, T_Lymphoid_marker))

include_genes <- c(lineageSwitching_genes_H, "NR4A1", "CEBPA", "KLF13",
"NFIL3", "PAX5", "POU2F1", "TCF4",
                  "MEF2C", "EBF1", "TCFL5", "TCF3", "LEF1", "IRF3", "PRTN3",
                  "ELANE", "MPO", "AZU1", "CTSG", "TYROBP", "IGLL1",
"CD79A",
                  "DNTT", "VPREB3", "IL7R", "LTB", marker_genes)

lineageSwitching_genes <- lineageSwitching_genes[which(lineageSwitching_genes
%in% include_genes)]

sample_names_lineage <- c("AML4", "AML16", "AML11", "AML17", "AML18",
"AML24")
modality_names_lineage <- c("RNA", "ATA")

```



```

stage_names_lineage <- c("DX", "REL")
iteration_times <- 10
# Lineage switching gene matrix
lineageSwitching_gene_matrix <- as.data.frame(matrix(NA, nrow =
length(lineageSwitching_genes),
                                ncol = iteration_times *
length(sample_names_lineage) * length(modality_names_lineage) *
length(stage_names_lineage)))
rownames(lineageSwitching_gene_matrix) <- lineageSwitching_genes

colnames_lineageSwitching_matrix <- c(paste("AML4",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2], iteration_times),
1:iteration_times, sep = "_")), "RNA", sep = "_"),
                                paste("AML4",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2], iteration_times),
1:iteration_times, sep = "_")), "ATA", sep = "_"),
                                paste("AML16",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2], iteration_times),
1:iteration_times, sep = "_")), "RNA", sep = "_"),
                                paste("AML16",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2], iteration_times),
1:iteration_times, sep = "_")), "ATA", sep = "_"),
                                paste("AML11",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2], iteration_times),
1:iteration_times, sep = "_")), "RNA", sep = "_"),
                                paste("AML11",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2],
iteration_times), 1:iteration_times, sep = "_")), "ATA", sep = "_"),
                                paste("AML17",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2], iteration_times),
1:iteration_times, sep = "_")), "RNA", sep = "_"),
                                paste("AML17",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2], iteration_times),
1:iteration_times, sep = "_")), "ATA", sep = "_"),
                                paste("AML18",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2], iteration_times),
1:iteration_times, sep = "_")), "RNA", sep = "_"),
                                paste("AML18",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2],
iteration_times), 1:iteration_times, sep = "_")), "ATA", sep = "_"),

```

```

                                paste("AML24",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2], iteration_times),
1:iteration_times, sep = "_")), "RNA", sep = "_"),
                                paste("AML24",
c(paste(rep(stage_names_lineage[1], iteration_times), 1:iteration_times, sep
= "_"), paste(rep(stage_names_lineage[2], iteration_times),
1:iteration_times, sep = "_")), "ATA", sep = "_"))

colnames(lineageSwitching_gene_matrix) <- colnames_lineageSwitching_matrix

for (sample in sample_names_lineage) {
  ATA_motif_path <- paste0("../Data/AML/", sample, "_cellMotif.rds")
  RNA_motif_path <- paste0("../Data/AML/singleCellMatrix/scATAC_geneExp/",
sample, "_geneExp.rds")
  ATA_motif <- readRDS(ATA_motif_path)
  rownames(ATA_motif) <- sub("_.*$", "", rownames(ATA_motif))
  RNA_exp <- readRDS(RNA_motif_path)
  for (gene in lineageSwitching_genes) {
    message("Processing gene: ", gene, " in sample: ", sample)
    # ATA
    message(" Processing ATA data...")
    res_ATA <- generate_marker_matrix(gene, ATA_motif, iteration_times =
iteration_times, iteration_size = 1000, dx_pattern = "DX", rel_pattern =
"REL")
    col_ATA <- paste(sample, c(paste(rep(stage_names_lineage[1],
iteration_times), 1:iteration_times, sep = "_"),
paste(rep(stage_names_lineage[2], iteration_times), 1:iteration_times, sep =
"_")), "ATA", sep = "_")
    col_ATA_DX <- grep("DX", col_ATA, value = TRUE)
    col_ATA_REL <- grep("REL", col_ATA, value = TRUE)
    ATA_value <- c(unlist(res_ATA$DX), unlist(res_ATA$REL))
    ATA_value_scaled <- range_scale(ATA_value, min_val = -1.5, max_val = 1.5)
    lineageSwitching_gene_matrix[gene, col_ATA_DX] <-
ATA_value_scaled[1:iteration_times]
    lineageSwitching_gene_matrix[gene, col_ATA_REL] <-
ATA_value_scaled[(iteration_times + 1):(2 * iteration_times)]
    # RNA
    message(" Processing RNA data...")
    res_RNA <- generate_marker_matrix(gene, RNA_exp, iteration_times =
iteration_times, iteration_size = 1000, dx_pattern = "DX", rel_pattern =
"REL")
    col_RNA <- paste(sample, c(paste(rep(stage_names_lineage[1],
iteration_times), 1:iteration_times, sep = "_"),
paste(rep(stage_names_lineage[2], iteration_times), 1:iteration_times, sep =
"_")), "RNA", sep = "_")
    col_RNA_DX <- grep("DX", col_RNA, value = TRUE)
    col_RNA_REL <- grep("REL", col_RNA, value = TRUE)
    RNA_value <- c(unlist(res_RNA$DX), unlist(res_RNA$REL))
    RNA_value_scaled <- range_scale(RNA_value, min_val = -1.5, max_val = 1.5)

```

```

    lineageSwitching_gene_matrix[gene, col_RNA_DX] <-
RNA_value_scaled[1:iteration_times]
    lineageSwitching_gene_matrix[gene, col_RNA_REL] <-
RNA_value_scaled[(iteration_times + 1):(2 * iteration_times)]
  }
}

## Processing gene: LMO2 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: RUNX1 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: GATA2 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: TAL1 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: ERG in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: FLI1 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: HOXA9 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: NR4A1 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: CEBPA in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: KLF13 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: NFIL3 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: PAX5 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: POU2F1 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: TCF4 in sample: AML4
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: MEF2C in sample: AML4
##   Processing ATA data...

```

```
## Processing RNA data...
## Processing gene: EBF1 in sample: AML4
## Processing ATA data...
## Processing RNA data...
## Processing gene: TCFL5 in sample: AML4
## Processing ATA data...
## Processing RNA data...
## Processing gene: TCF3 in sample: AML4
## Processing ATA data...
## Processing RNA data...
## Processing gene: LEF1 in sample: AML4
## Processing ATA data...
## Processing RNA data...
## Processing gene: IRF3 in sample: AML4
## Processing ATA data...
## Processing RNA data...
## Processing gene: HOPX in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in max(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: SPINK2 in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD34 in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: NUCB2 in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: C1QTNF4 in sample: AML4
## Processing ATA data...
```

```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CDK6 in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: IGFBP7 in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: PRTN3 in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: ELANE in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: MPO in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: AZU1 in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
```

```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CTSG in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: TYROBP in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: FCN1 in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: VCAN in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD79A in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD19 in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
```

```
## -Inf
## Processing RNA data...
## Processing gene: DNTT in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: VPREB3 in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: IL7R in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: LTB in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD7 in sample: AML4
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: LM02 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: RUNX1 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: GATA2 in sample: AML16
## Processing ATA data...
```



```
## Processing RNA data...
## Processing gene: TAL1 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: ERG in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: FLI1 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: HOXA9 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: NR4A1 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: CEBPA in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: KLF13 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: NFIL3 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: PAX5 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: POU2F1 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: TCF4 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: MEF2C in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: EBF1 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: TCFL5 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: TCF3 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: LEF1 in sample: AML16
## Processing ATA data...
## Processing RNA data...
## Processing gene: IRF3 in sample: AML16
```



```
## Processing ATA data...
## Processing RNA data...
## Processing gene: HOPX in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: SPINK2 in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD34 in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: NUCB2 in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: C1QTNF4 in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CDK6 in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
```

```
## Processing gene: IGFBP7 in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: PRTN3 in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: ELANE in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: MPO in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: AZU1 in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CTSG in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: TYROBP in sample: AML16
## Processing ATA data...
```

```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: FCN1 in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: VCAN in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD79A in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD19 in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: DNTT in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: VPREB3 in sample: AML16
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
```

```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: IL7R in sample: AML16  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: LTB in sample: AML16  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: CD7 in sample: AML16  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: LMO2 in sample: AML11  
## Processing ATA data...  
## Processing RNA data...  
## Processing gene: RUNX1 in sample: AML11  
## Processing ATA data...  
## Processing RNA data...  
## Processing gene: GATA2 in sample: AML11  
## Processing ATA data...  
## Processing RNA data...  
## Processing gene: TAL1 in sample: AML11  
## Processing ATA data...  
## Processing RNA data...  
## Processing gene: ERG in sample: AML11  
## Processing ATA data...  
## Processing RNA data...  
## Processing gene: FLI1 in sample: AML11  
## Processing ATA data...  
## Processing RNA data...  
## Processing gene: HOXA9 in sample: AML11  
## Processing ATA data...  
## Processing RNA data...  
## Processing gene: NR4A1 in sample: AML11
```

```
## Processing ATA data...
## Processing RNA data...
## Processing gene: CEBPA in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: KLF13 in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: NFIL3 in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: PAX5 in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: POU2F1 in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: TCF4 in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: MEF2C in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: EBF1 in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: TCFL5 in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: TCF3 in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: LEF1 in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: IRF3 in sample: AML11
## Processing ATA data...
## Processing RNA data...
## Processing gene: HOPX in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: SPINK2 in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
```

```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: CD34 in sample: AML11  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: NUCB2 in sample: AML11  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: C1QTNF4 in sample: AML11  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: CDK6 in sample: AML11  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: IGFBP7 in sample: AML11  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: PRTN3 in sample: AML11  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning
```

```
## -Inf
## Processing RNA data...
## Processing gene: ELANE in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: MPO in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: AZU1 in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CTSG in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: TYROBP in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: FCN1 in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
```

```
## Processing gene: VCAN in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD79A in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD19 in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: DNTT in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: VPREB3 in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: IL7R in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: LTB in sample: AML11
## Processing ATA data...
```



```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD7 in sample: AML11
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: LMO2 in sample: AML17
## Processing ATA data...
## Processing RNA data...
## Processing gene: RUNX1 in sample: AML17
## Processing ATA data...
## Processing RNA data...
## Processing gene: GATA2 in sample: AML17
## Processing ATA data...
## Processing RNA data...
## Processing gene: TAL1 in sample: AML17
## Processing ATA data...
## Processing RNA data...
## Processing gene: ERG in sample: AML17
## Processing ATA data...
## Processing RNA data...
## Processing gene: FLI1 in sample: AML17
## Processing ATA data...
## Processing RNA data...
## Processing gene: HOXA9 in sample: AML17
## Processing ATA data...
## Processing RNA data...
## Processing gene: NR4A1 in sample: AML17
## Processing ATA data...
## Processing RNA data...
## Processing gene: CEBPA in sample: AML17
## Processing ATA data...
## Processing RNA data...
## Processing gene: KLF13 in sample: AML17
## Processing ATA data...
## Processing RNA data...
## Processing gene: NFIL3 in sample: AML17
## Processing ATA data...
## Processing RNA data...
## Processing gene: PAX5 in sample: AML17
## Processing ATA data...
## Processing RNA data...
```

```
## Processing gene: POU2F1 in sample: AML17
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: TCF4 in sample: AML17
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: MEF2C in sample: AML17
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: EBF1 in sample: AML17
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: TCFL5 in sample: AML17
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: TCF3 in sample: AML17
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: LEF1 in sample: AML17
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: IRF3 in sample: AML17
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: HOPX in sample: AML17
##   Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
##   Processing RNA data...
## Processing gene: SPINK2 in sample: AML17
##   Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
##   Processing RNA data...
## Processing gene: CD34 in sample: AML17
##   Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
##   Processing RNA data...
## Processing gene: NUCB2 in sample: AML17
##   Processing ATA data...
```

```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: C1QTNF4 in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CDK6 in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: IGFBP7 in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: PRTN3 in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: ELANE in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: MPO in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
```

```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: AZU1 in sample: AML17  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: CTSG in sample: AML17  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: TYROBP in sample: AML17  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: FCN1 in sample: AML17  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: VCAN in sample: AML17  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: CD79A in sample: AML17  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning
```

```
## -Inf
## Processing RNA data...
## Processing gene: CD19 in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: DNTT in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: VPREB3 in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: IL7R in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: LTB in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD7 in sample: AML17
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
```

```
## Processing gene: LM02 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: RUNX1 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: GATA2 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: TAL1 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: ERG in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: FLI1 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: HOXA9 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: NR4A1 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: CEBPA in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: KLF13 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: NFIL3 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: PAX5 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: POU2F1 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: TCF4 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: MEF2C in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: EBF1 in sample: AML18
##   Processing ATA data...
##   Processing RNA data...
## Processing gene: TCFL5 in sample: AML18
##   Processing ATA data...
```

```
## Processing RNA data...
## Processing gene: TCF3 in sample: AML18
## Processing ATA data...
## Processing RNA data...
## Processing gene: LEF1 in sample: AML18
## Processing ATA data...
## Processing RNA data...
## Processing gene: IRF3 in sample: AML18
## Processing ATA data...
## Processing RNA data...
## Processing gene: HOPX in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: SPINK2 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD34 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: NUCB2 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: C1QTNF4 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
```

```
## Processing gene: CDK6 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: IGFBP7 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: PRTN3 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: ELANE in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: MPO in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: AZU1 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CTSB in sample: AML18
## Processing ATA data...
```



```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: TYROBP in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: FCN1 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: VCAN in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD79A in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD19 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: DNTT in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
```

```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: VPREB3 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: IL7R in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: LTB in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD7 in sample: AML18
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: LMO2 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: RUNX1 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: GATA2 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: TAL1 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: ERG in sample: AML24
## Processing ATA data...
```

```
## Processing RNA data...
## Processing gene: FLI1 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: HOXA9 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: NR4A1 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: CEBPA in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: KLF13 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: NFIL3 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: PAX5 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: POU2F1 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: TCF4 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: MEF2C in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: EBF1 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: TCFL5 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: TCF3 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: LEF1 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: IRF3 in sample: AML24
## Processing ATA data...
## Processing RNA data...
## Processing gene: HOPX in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
```

```
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: SPINK2 in sample: AML24  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: CD34 in sample: AML24  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: NUCB2 in sample: AML24  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: C1QTNF4 in sample: AML24  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: CDK6 in sample: AML24  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning  
## -Inf  
## Processing RNA data...  
## Processing gene: IGFBP7 in sample: AML24  
## Processing ATA data...  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;  
returning Inf  
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;  
returning
```

```
## -Inf
## Processing RNA data...
## Processing gene: PRTN3 in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: ELANE in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: MPO in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: AZU1 in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CTSG in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: TYROBP in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
```

```
## Processing gene: FCN1 in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: VCAN in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD79A in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD19 in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: DNTT in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: VPREB3 in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: IL7R in sample: AML24
## Processing ATA data...
```

```

## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: LTB in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...
## Processing gene: CD7 in sample: AML24
## Processing ATA data...
## Warning in min(x, na.rm = TRUE): no non-missing arguments to min;
returning Inf
## Warning in min(x, na.rm = TRUE): no non-missing arguments to max;
returning
## -Inf
## Processing RNA data...

lineageSwitching_gene_matrix_rmna <- na.omit(lineageSwitching_gene_matrix)

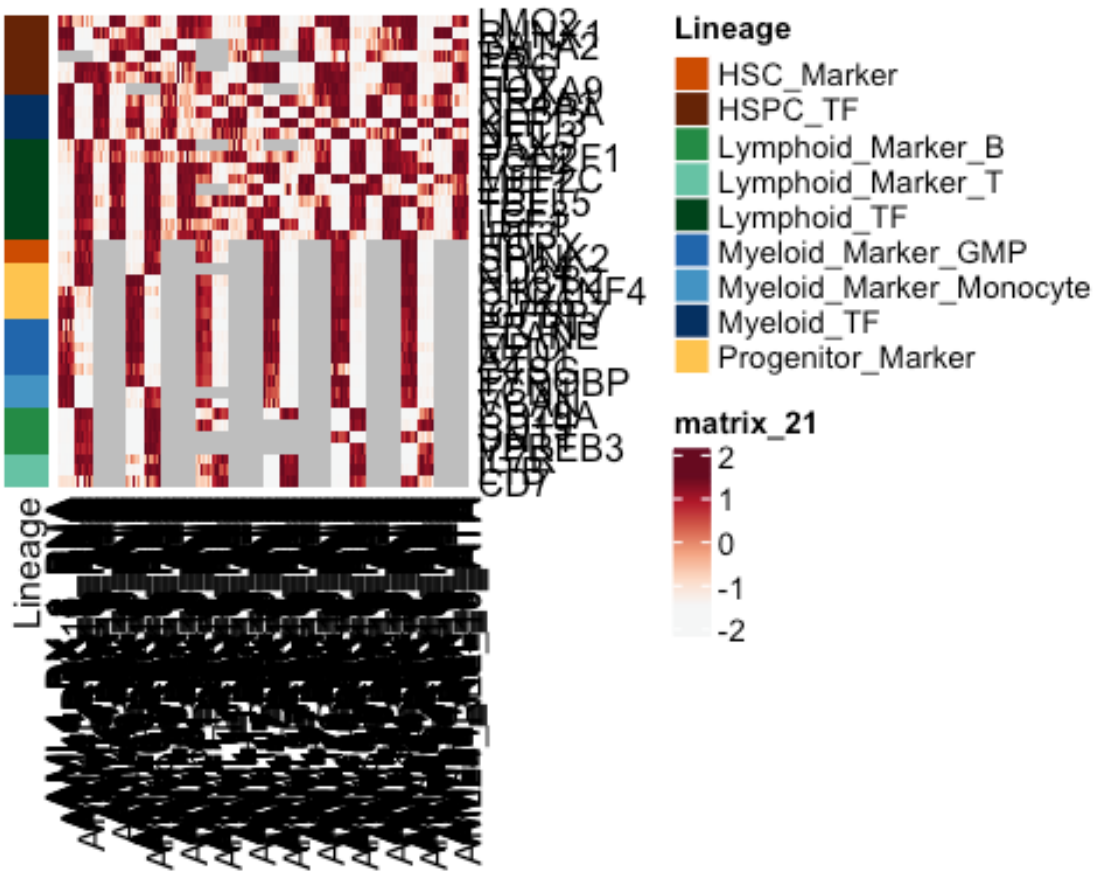
gene_lineage <- gene_lineage[gene_lineage$Gene %in% include_genes, ]

ha_row <- rowAnnotation(
  Lineage = gene_lineage$Lineage,
  col = list(
    Lineage = c(
      HSPC_TF = "#662506",
      HSC_Marker = "#cc4c02",
      Progenitor_Marker = "#fec44f",
      Myeloid_TF = "#053061",
      Myeloid_Marker_GMP = "#2166AC",
      Myeloid_Marker_Monocyte = "#4393C3",
      Lymphoid_TF = "#00441b",
      Lymphoid_Marker_B = "#238b45",
      Lymphoid_Marker_T = "#66c2a4"
    )
  )
)

Heatmap(lineageSwitching_gene_matrix,
  cluster_rows = FALSE,
  cluster_columns = FALSE,
  left_annotation = ha_row,

```

```
col = rev(colorRampPalette(brewer.pal(11, "RdBu"))(100))[50:100])
## Warning: The input is a data frame-like object, convert it to a matrix.
```



```
#saveRDS(LineageSwitching_gene_matrix,
"../Data/AML/AML_lineageSwitching_gene_matrix.rds")
#saveRDS(gene_lineage, "../Data/AML/AML_lineageSwitching_gene_meta.rds")
```

transcription factor lineage plots

```
lineageSwitching_gene_matrix <-
readRDS("../Data/AML/AML_lineageSwitching_gene_matrix.rds")
gene_lineage <- readRDS("../Data/AML/AML_lineageSwitching_gene_meta.rds")
gene_lineage_tf <- gene_lineage[gene_lineage$Lineage %in% c("Myeloid_TF",
"Lymphoid_TF"),]
gene_lineage_marker <- gene_lineage[gene_lineage$Lineage %in% c("HSC_Marker",
"Progenitor_Marker",
"Myeloid_Marker_GMP", "Myeloid_Marker_Monocyte",
"Lymphoid_Marker_B", "Lymphoid_Marker_T"),]
```



```

gene_tf <- gene_lineage$Gene[gene_lineage$Lineage %in% c("Myeloid_TF",
"Lymphoid_TF")]
gene_marker <- gene_lineage$Gene[gene_lineage$Lineage %in% c("HSC_Marker",
"Progenitor_Marker",

"Myeloid_Marker_GMP", "Myeloid_Marker_Monocyte",

"Lymphoid_Marker_B", "Lymphoid_Marker_T")]

lineageSwitching_gene_matrix_tf <-
lineageSwitching_gene_matrix[rownames(lineageSwitching_gene_matrix) %in%
gene_tf, ]
lineageSwitching_gene_matrix_tf_RNA <- lineageSwitching_gene_matrix_tf[,
grep("RNA", colnames(lineageSwitching_gene_matrix_tf))]
lineageSwitching_gene_matrix_tf_ATA <- lineageSwitching_gene_matrix_tf[,
grep("ATA", colnames(lineageSwitching_gene_matrix_tf))]

lineageSwitching_gene_matrix_marker <-
lineageSwitching_gene_matrix[rownames(lineageSwitching_gene_matrix) %in%
gene_marker, ]
lineageSwitching_gene_matrix_marker_RNA <-
lineageSwitching_gene_matrix_marker[, grep("RNA",
colnames(lineageSwitching_gene_matrix_marker))]

ha_row_tf <- rowAnnotation(
  Lineage = gene_lineage_tf$Lineage,
  col = list(
    Lineage = c(
      Myeloid_TF = "#053061",
      Lymphoid_TF = "#00441b"
    )
  )
)

ha_row_marker <- rowAnnotation(
  Lineage = gene_lineage_marker$Lineage,
  col = list(
    Lineage = c(
      HSC_Marker = "#7f2704",
      Progenitor_Marker = "#d94801",
      Myeloid_Marker_GMP = "#2166AC",
      Myeloid_Marker_Monocyte = "#4393C3",
      Lymphoid_Marker_B = "#238b45",
      Lymphoid_Marker_T = "#66c2a4"
    )
  )
)

TF_motif_order_byletter <- c("CEBPA", "KLF13", "NFIL3", "NR4A1", "EBF1",

```

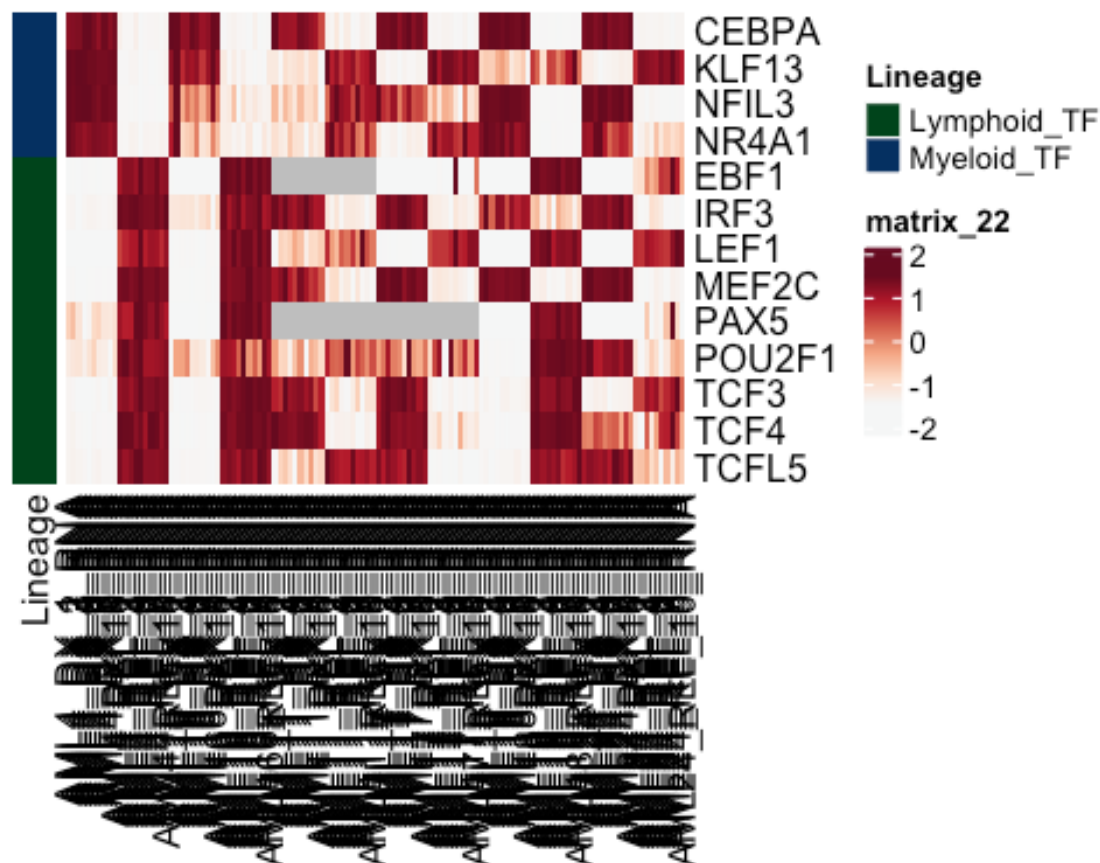
```

"IRF3",
                                "LEF1", "MEF2C", "PAX5", "POU2F1", "TCF3",
"TCF4", "TCFL5")

marker_gene_order_byletter <- c("C1QTNF4", "CD34", "CDK6", "HOPX", "IGFBP7",
"NUCB2", "SPINK2",
                                "AZU1", "CTSG", "ELANE", "FCN1", "MPO",
"PRTN3", "TYROBP", "VCAN",
                                "CD19", "CD79A", "DNTT", "IL7R", "LTB",
"VPREB3", "CD7")

# TF_EXP
Heatmap(lineageSwitching_gene_matrix_tf_RNA[TF_motif_order_byletter,],
        cluster_rows = FALSE,
        cluster_columns = FALSE,
        left_annotation = ha_row_tf,
        col = rev(colorRampPalette(brewer.pal(11, "RdBu"))(100))[50:100])
## Warning: The input is a data frame-like object, convert it to a matrix.

```



```

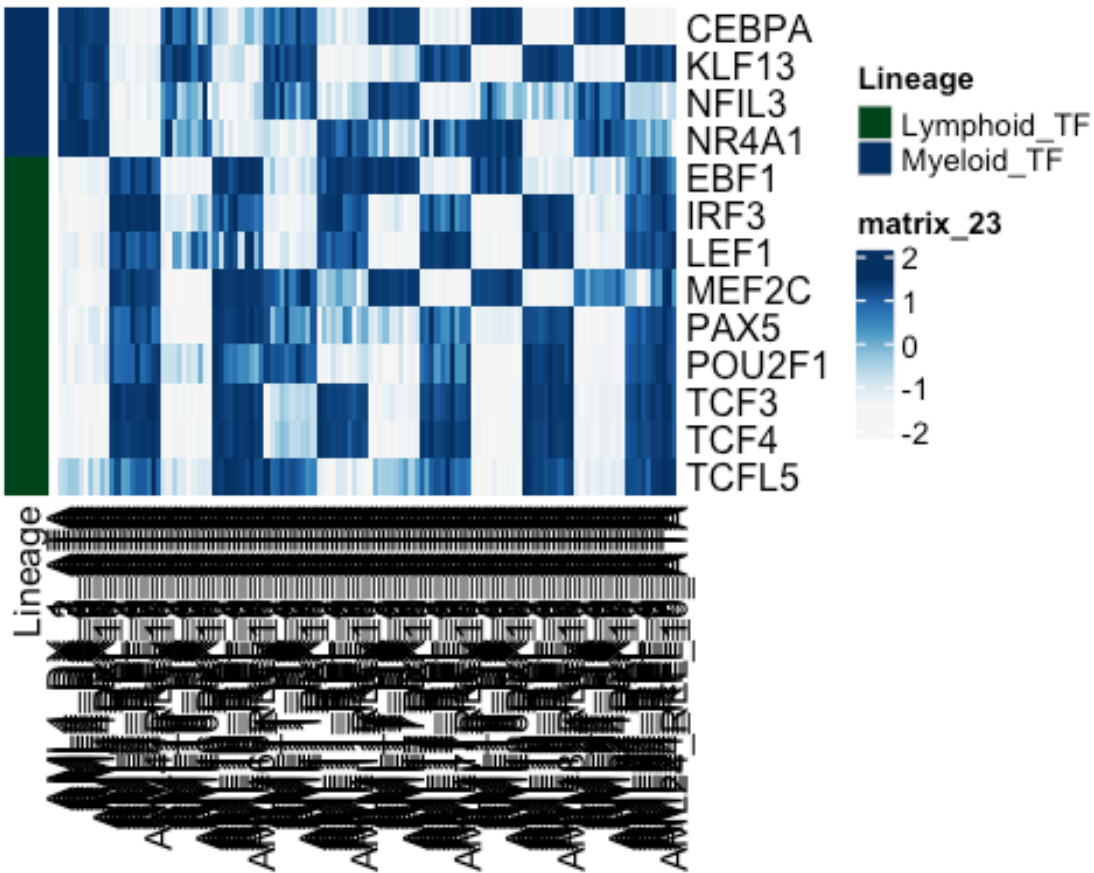
# TF_ATA
Heatmap(lineageSwitching_gene_matrix_tf_ATA[TF_motif_order_byletter, ],
        cluster_rows = FALSE,

```

```

cluster_columns = FALSE,
left_annotation = ha_row_tf,
col = colorRampPalette(brewer.pal(11, "RdBu"))(100)[50:100])
## Warning: The input is a data frame-like object, convert it to a matrix.

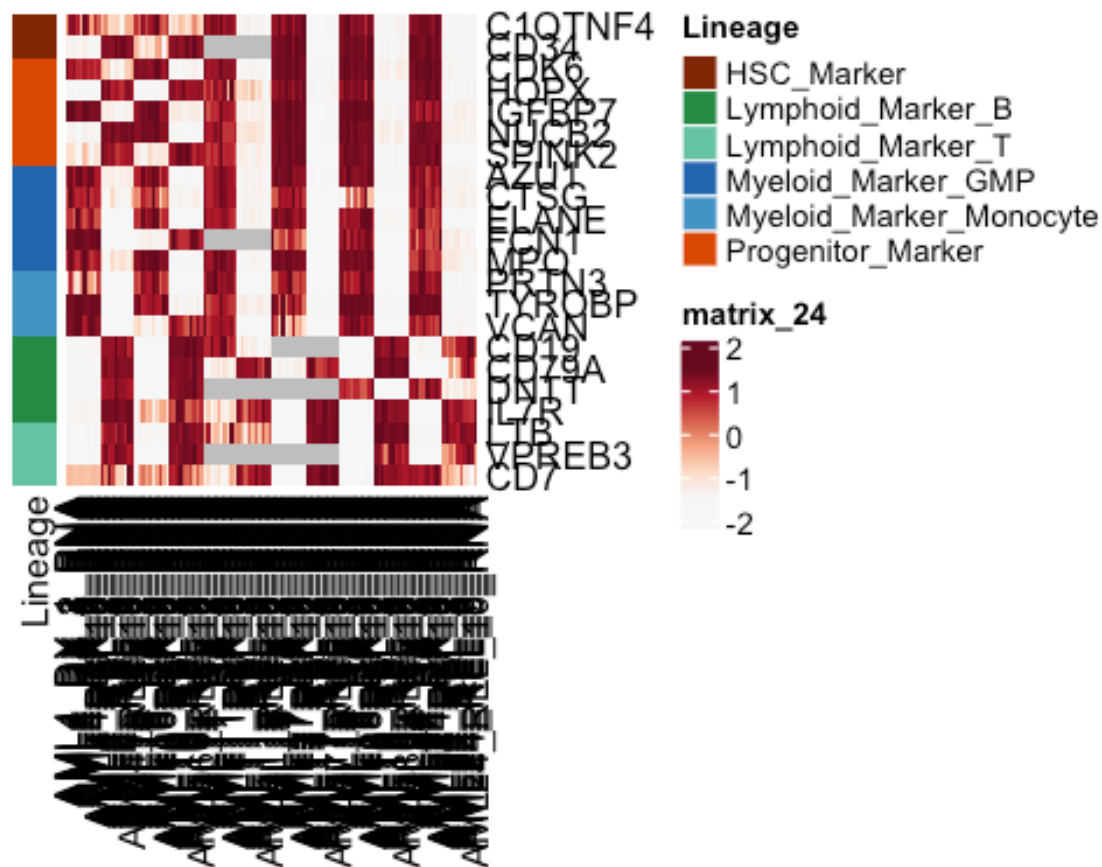
```



```

# Marker_EXP
Heatmap(lineageSwitching_gene_matrix_marker_RNA[marker_gene_order_byletter,
],
cluster_rows = FALSE,
cluster_columns = FALSE,
left_annotation = ha_row_marker,
col = rev(colorRampPalette(brewer.pal(11, "RdBu"))(100))[50:100])
## Warning: The input is a data frame-like object, convert it to a matrix.

```



mutation Similarity DXREL for lineage switching

```
get_scrna_mutSites_byStages <- function(CSM_list, meta_data){
  common_sample_pairs <- unique(names(CSM_list))
  ct_forComparision <- unique(meta_data$Classified_Celltype)
  ct_forComparision <- setdiff(ct_forComparision, "Unknown")
  print(ct_forComparision)
  data_mutationCelltype_sites_list <- list()
  for (sample in common_sample_pairs){
    message(sample)
    csm_tmp <- CSM_list[[sample]]
    for (i in ct_forComparision){
      ct_i <- i
      cell_barcode_meta_i <- meta_data %>%
        mutate(CellTag = rownames(.)) %>%
        dplyr::filter(Classified_Celltype == ct_i & Library_ID == sample) %>%
        pull(CellTag)
      cell_barcode_csm_format_all_i <- rownames(csm_tmp) %>%
        sub("_L[0-9]+_", "_", .) %>%
        sub("_(.*)_", "_\\1.", .) %>%
        paste0(., "-1") %>%
        sub("\\.", "#", x = .)
    }
  }
}
```

```

    cell_barcode_csm_selected_i <-
rownames(csm_tmp)[cell_barcode_csm_format_all_i %in% cell_barcode_meta_i]
    mut_site_i <- retriveLocsByCells(csm_tmp,
cell_barcode_csm_selected_i)$Mut
    if (length(cell_barcode_meta_i) <= 10){
        mut_site_i <- "Empty"
    }
    data_mutationCelltype_sites_list[[sample]][[ct_i]] <- mut_site_i
}
}
return(data_mutationCelltype_sites_list)
}

get_scATA_mutSites_byStages <- function(CSM_list, meta_data){
    common_sample_pairs <- unique(names(CSM_list))
    ct_forComparision <- unique(meta_data$Classified_Celltype)
    ct_forComparision <- setdiff(ct_forComparision, "Unknown")
    print(ct_forComparision)
    data_mutationCelltype_sites_list <- list()
    for (sample in common_sample_pairs){
        message(sample)
        csm_tmp <- CSM_list[[sample]]
        for (i in ct_forComparision){
            ct_i <- i
            cell_barcode_meta_i <- meta_data %>%
                mutate(CellTag = rownames(.)) %>%
                dplyr::filter(Classified_Celltype == ct_i & Sample == sample) %>%
                pull(CellTag)
            cell_barcode_csm_format_all_i <- rownames(csm_tmp) %>%
                sub("_L[0-9]+_", "_", .) %>%
                sub("_(.*)_ ", "_\\1.", .) %>%
                paste0(., "-1") %>%
                sub("\\.", "#", x = .)
            cell_barcode_csm_selected_i <-
rownames(csm_tmp)[cell_barcode_csm_format_all_i %in% cell_barcode_meta_i]
            mut_site_i <- retriveLocsByCells(csm_tmp,
cell_barcode_csm_selected_i)$Mut
            if (length(cell_barcode_meta_i) <= 10){
                mut_site_i <- "Empty"
            }
            data_mutationCelltype_sites_list[[sample]][[ct_i]] <- mut_site_i
        }
    }
    return(data_mutationCelltype_sites_list)
}

AML_scRNA_CSM_list <- AML_RNA_CSM_list
AML_scATA_CSM_list <- AML_ATA_CSM_list

```

```

# scRNA
# AML_CSM_DX_List
AML_scRNA_CSM_DX_list <- lapply(AML_scRNA_CSM_list, function(df) {
  df[grep("DX", rownames(df)), , drop = FALSE]
})
AML_scRNA_CSM_DX_list <- Filter(function(x) nrow(x) > 0,
AML_scRNA_CSM_DX_list)

# AML_CSM_REM_List
AML_scRNA_CSM_REM_list <- lapply(AML_scRNA_CSM_list, function(df) {
  df[grep("REM", rownames(df)), , drop = FALSE]
})
AML_scRNA_CSM_REM_list <- Filter(function(x) nrow(x) > 0,
AML_scRNA_CSM_REM_list)

# AML_CSM_REL_List
AML_scRNA_CSM_REL_list <- lapply(AML_scRNA_CSM_list, function(df) {
  df[grep("REL", rownames(df)), , drop = FALSE]
})
AML_scRNA_CSM_REL_list <- Filter(function(x) nrow(x) > 0,
AML_scRNA_CSM_REL_list)

# scATA
# AML_CSM_DX_List
AML_scATA_CSM_DX_list <- lapply(AML_scATA_CSM_list, function(df) {
  df[grep("DX", rownames(df)), , drop = FALSE]
})
AML_scATA_CSM_DX_list <- Filter(function(x) nrow(x) > 0,
AML_scATA_CSM_DX_list)

# AML_CSM_REM_List
AML_scATA_CSM_REM_list <- lapply(AML_scATA_CSM_list, function(df) {
  df[grep("REM", rownames(df)), , drop = FALSE]
})
AML_scATA_CSM_REM_list <- Filter(function(x) nrow(x) > 0,
AML_scATA_CSM_REM_list)

# AML_CSM_REL_List
AML_scATA_CSM_REL_list <- lapply(AML_scATA_CSM_list, function(df) {
  df[grep("REL", rownames(df)), , drop = FALSE]
})
AML_scATA_CSM_REL_list <- Filter(function(x) nrow(x) > 0,
AML_scATA_CSM_REL_list)

HSPC <- c("HSC", "Progenitor")
Myeloid <- c("GMP", "Monocytes", "CD16.Monocytes", "cDC", "pDC")
Lymphoid <- c("CLP", "Pre.B.Cell", "B.Cell", "CD4.Naive", "CD4.Memory",
"CD8.Naive", "CD8.Memory", "CD8.Effector", "NK")
Erythroid_Basophil <- c("Early.Erythrocyte", "Late.Erythrocyte")

```

```

Other <- c("Early.Basophil", "Plasma")

AML_scrNA_meta <- AML_scrNA_meta %>%
  dplyr::mutate(
    maincelltype = case_when(
      Classified_Celltype %in% HSPC ~ "HSPC",
      Classified_Celltype %in% Myeloid ~ "Myeloid",
      Classified_Celltype %in% Lymphoid ~ "Lymphoid",
      Classified_Celltype %in% Erythroid_Basophil ~ "Erythroid",
      Classified_Celltype %in% Other ~ "Other",
      TRUE ~ NA_character_
    )
  )

AML_scATA_meta <- AML_scATA_meta %>%
  dplyr::mutate(
    maincelltype = case_when(
      predictedGroup_Un %in% HSPC ~ "HSPC",
      predictedGroup_Un %in% Myeloid ~ "Myeloid",
      predictedGroup_Un %in% Lymphoid ~ "Lymphoid",
      predictedGroup_Un %in% Erythroid_Basophil ~ "Erythroid",
      predictedGroup_Un %in% Other ~ "Other",
      TRUE ~ NA_character_
    )
  )

AML_scATA_meta$Classified_Celltype <- AML_scATA_meta$predictedGroup_Un

maincelltypes <- c("HSPC", "Myeloid", "Lymphoid", "Erythroid", "Other")
subcelltypes <- c(HSPC, Myeloid, Lymphoid, Erythroid_Basophil, Other)

AML_scrNA_meta_DX <- AML_scrNA_meta %>%
  dplyr::filter(grepl("DX", Library_ID))

AML_scrNA_meta_REL <- AML_scrNA_meta %>%
  dplyr::filter(grepl("REL", Library_ID))

AML_scATA_meta_DX <- AML_scATA_meta %>%
  dplyr::filter(grepl("DX", Sample))

AML_scATA_meta_REL <- AML_scATA_meta %>%
  dplyr::filter(grepl("REL", Sample))

AML_scrNA_meta_DX$Classified_CelltypeQ <-
  paste(AML_scrNA_meta_DX$Classified_Celltype, "DX", sep = "_")
AML_scrNA_meta_REL$Classified_CelltypeQ <-
  paste(AML_scrNA_meta_REL$Classified_Celltype, "REL", sep = "_")
AML_scrNA_meta_DX$maincelltypeQ <- paste(AML_scrNA_meta_DX$maincelltype,
  "DX", sep = "_")

```

```

AML_scrRNA_meta_REL$maincelltypeQ <- paste(AML_scrRNA_meta_REL$maincelltype,
"REL", sep = "_")

AML_scATA_meta_DX$Classified_CelltypeQ <-
paste(AML_scATA_meta_DX$Classified_Celltype, "DX", sep = "_")
AML_scATA_meta_REL$Classified_CelltypeQ <-
paste(AML_scATA_meta_REL$Classified_Celltype, "REL", sep = "_")
AML_scATA_meta_DX$maincelltypeQ <- paste(AML_scATA_meta_DX$maincelltype,
"DX", sep = "_")
AML_scATA_meta_REL$maincelltypeQ <- paste(AML_scATA_meta_REL$maincelltype,
"REL", sep = "_")

AML_scrRNA_meta_DX_REL <- rbind(AML_scrRNA_meta_DX, AML_scrRNA_meta_REL)
AML_scATA_meta_DX_REL <- rbind(AML_scATA_meta_DX, AML_scATA_meta_REL)

# samples for lineage switching
selected_samples_id <- c("AML4", "AML11", "AML16", "AML17", "AML18", "AML24")
# scRNA
AML_scrRNA_CSM_DX_select_list <-
AML_scrRNA_CSM_DX_list[names(AML_scrRNA_CSM_DX_list) %in% selected_samples_id]
names(AML_scrRNA_CSM_DX_select_list) <- paste(selected_samples_id, "DX", sep =
"_")
AML_scrRNA_CSM_REL_select_list <-
AML_scrRNA_CSM_REL_list[names(AML_scrRNA_CSM_REL_list) %in%
selected_samples_id]
names(AML_scrRNA_CSM_REL_select_list) <- paste(selected_samples_id, "REL", sep
= "_")
AML_scrRNA_CSM_DX_REL_select_list <- c(AML_scrRNA_CSM_DX_select_list,
AML_scrRNA_CSM_REL_select_list)

# scATA
AML_scATA_CSM_DX_select_list <-
AML_scATA_CSM_DX_list[names(AML_scATA_CSM_DX_list) %in% selected_samples_id]
names(AML_scATA_CSM_DX_select_list) <- paste(selected_samples_id, "DX", sep =
"_")
AML_scATA_CSM_REL_select_list <-
AML_scATA_CSM_REL_list[names(AML_scATA_CSM_REL_list) %in%
selected_samples_id]
names(AML_scATA_CSM_REL_select_list) <- paste(selected_samples_id, "REL", sep
= "_")
AML_scATA_CSM_DX_REL_select_list <- c(AML_scATA_CSM_DX_select_list,
AML_scATA_CSM_REL_select_list)

data_mutationCelltype_mutSites_scrRNA_byStage <-
get_scrRNA_mutSites_byStages(AML_scrRNA_CSM_DX_REL_select_list,
AML_scrRNA_meta_DX_REL)
## [1] "Monocytes" "CLP" "CD8.Memory"
## [4] "Early.Erythrocyte" "CD4.Naive" "Late.Erythrocyte"
## [7] "Pre.B.Cell" "Progenitor" "pDC"

```



```

## [10] "GMP"           "CD4.Memory"      "cDC"
## [13] "HSC"           "CD8.Naive"       "CD16.Monocytes"
## [16] "CD8.Effector"  "NK"              "Early.Basophil"
## [19] "B.Cell"        "Plasma"
## AML4_DX
## AML11_DX
## AML16_DX
## AML17_DX
## AML18_DX
## AML24_DX
## AML4_REL
## AML11_REL
## AML16_REL
## AML17_REL
## AML18_REL
## AML24_REL
data_mutationCelltype_mutSites_scATA_byStage <-
get_scATA_mutSites_byStages(AML_scATA_CSM_DX_REL_select_list,
AML_scATA_meta_DX_REL)
## [1] "Progenitor"      "B.Cell"          "Early.Erythrocyte"
## [4] "CD4.Memory"      "GMP"             "Early.Basophil"
## [7] "NK"              "CD8.Memory"      "Monocytes"
## [10] "HSC"             "CD8.Naive"       "Late.Erythrocyte"
## [13] "CLP"             "Pre.B.Cell"      "CD4.Naive"
## [16] "CD8.Effector"    "pDC"             "cDC"
## [19] "Plasma"          "CD16.Monocytes"
## AML4_DX
## AML11_DX
## AML16_DX
## AML17_DX
## AML18_DX
## AML24_DX
## AML4_REL
## AML11_REL
## AML16_REL
## AML17_REL
## AML18_REL
## AML24_REL

data_mutationCelltype_similarity_scRNA_byStage_list <- list()
data_mutationCelltype_similarity_scATA_byStage_list <- list()
common_sample_pairs <- selected_samples_id
common_sample_stages <- c("DX", "REL")
ct_forComparision <- subcelltypes

for (sample in common_sample_pairs){
  message(sample)
  row_col_data_Names <- c(paste(ct_forComparision, "DX", sep = "_"),
paste(ct_forComparision, "REL", sep = "_"))
  mut_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow =

```

```

length(row_col_data_Names),
                                ncol =
length(row_col_data_Names)))
  rownames(mut_cellTypeAnn_similarity) <- row_col_data_Names
  colnames(mut_cellTypeAnn_similarity) <- row_col_data_Names
  for (i in 1:nrow(mut_cellTypeAnn_similarity)){
    ct_stage_i <- rownames(mut_cellTypeAnn_similarity)[i]
    ct_i <- strsplit(ct_stage_i, "_")[[1]][1]
    stage_i <- strsplit(ct_stage_i, "_")[[1]][2]
    mut_site_i <- data_mutationCelltype_mutSites_scRNA_byStage[[paste(sample,
stage_i, sep = "_")]][[ct_i]]
    for (j in 1:ncol(mut_cellTypeAnn_similarity)){
      ct_stage_j <- colnames(mut_cellTypeAnn_similarity)[j]
      ct_j <- strsplit(ct_stage_j, "_")[[1]][1]
      stage_j <- strsplit(ct_stage_j, "_")[[1]][2]
      mut_site_j <-
data_mutationCelltype_mutSites_scRNA_byStage[[paste(sample, stage_j, sep =
"_")]][[ct_j]]
      similarity_score <- ifelse(
        length(unique(c(mut_site_i, mut_site_j))) == 0,
        0,
        length(unique(intersect(mut_site_i, mut_site_j))) /
length(unique(c(mut_site_i, mut_site_j)))
      )
      if (length(mut_site_i) == 1 | length(mut_site_j) == 1){
        similarity_score <- 0
      }

      mut_cellTypeAnn_similarity[i, j] <- similarity_score
    }
  }
  data_mutationCelltype_similarity_scRNA_byStage_list[[sample]] <-
mut_cellTypeAnn_similarity
}
## AML4
## AML11
## AML16
## AML17
## AML18
## AML24

for (sample in common_sample_pairs){
  message(sample)
  row_col_data_Names <- c(paste(ct_forComparision, "DX", sep = "_"),
paste(ct_forComparision, "REL", sep = "_"))
  mut_cellTypeAnn_similarity <- as.data.frame(matrix(data = 0, nrow =
length(row_col_data_Names),
                                ncol =
length(row_col_data_Names)))
  rownames(mut_cellTypeAnn_similarity) <- row_col_data_Names

```

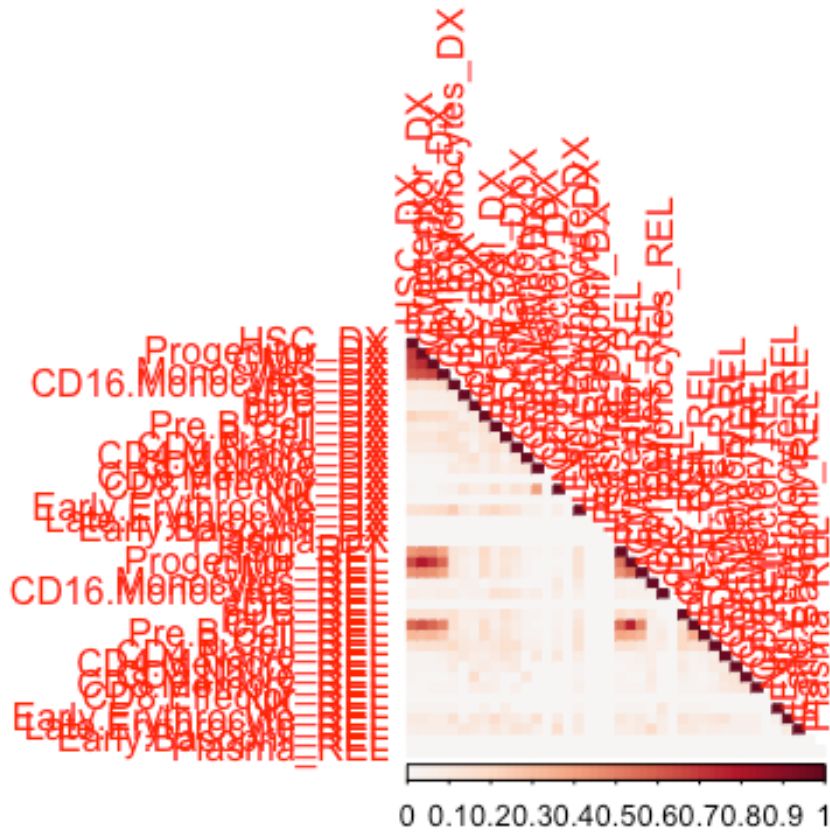
```

colnames(mut_cellTypeAnn_similarity) <- row_col_data_Names
for (i in 1:nrow(mut_cellTypeAnn_similarity)){
  ct_stage_i <- rownames(mut_cellTypeAnn_similarity)[i]
  ct_i <- strsplit(ct_stage_i, "_")[[1]][1]
  stage_i <- strsplit(ct_stage_i, "_")[[1]][2]
  mut_site_i <- data_mutationCelltype_mutSites_scATA_byStage[[paste(sample,
stage_i, sep = "_")]][[ct_i]]
  for (j in 1:ncol(mut_cellTypeAnn_similarity)){
    ct_stage_j <- colnames(mut_cellTypeAnn_similarity)[j]
    ct_j <- strsplit(ct_stage_j, "_")[[1]][1]
    stage_j <- strsplit(ct_stage_j, "_")[[1]][2]
    mut_site_j <-
data_mutationCelltype_mutSites_scATA_byStage[[paste(sample, stage_j, sep =
"_")]][[ct_j]]
    similarity_score <- ifelse(
      length(unique(c(mut_site_i, mut_site_j))) == 0,
      0,
      length(unique(intersect(mut_site_i, mut_site_j))) /
length(unique(c(mut_site_i, mut_site_j)))
    )
    if (length(mut_site_i) == 1 | length(mut_site_j) == 1){
      similarity_score <- 0
    }
    mut_cellTypeAnn_similarity[i, j] <- similarity_score
  }
}
data_mutationCelltype_similarity_scATA_byStage_list[[sample]] <-
mut_cellTypeAnn_similarity
}
## AML4
## AML11
## AML16
## AML17
## AML18
## AML24

corrplot(as.matrix(data_mutationCelltype_similarity_scRNA_byStage_list$AML4),
order = "original", type = 'lower',
method = 'color', col = rev(colorRampPalette(brewer.pal(11,
"RdBu"))(100)), col.lim = c(0, 1), title = "AML4")

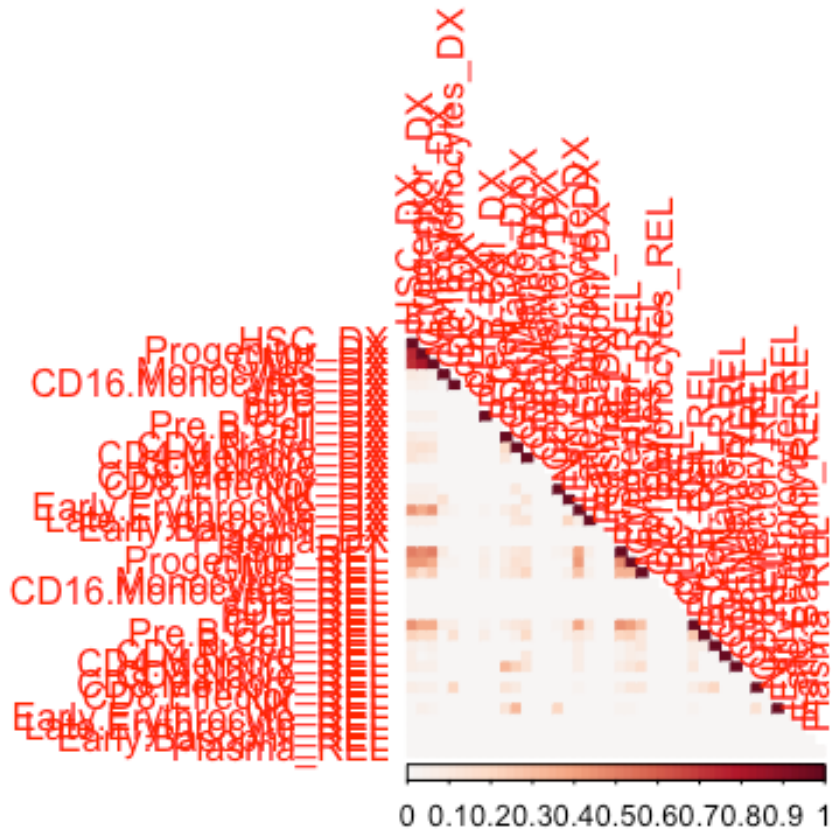
```

AMIL4

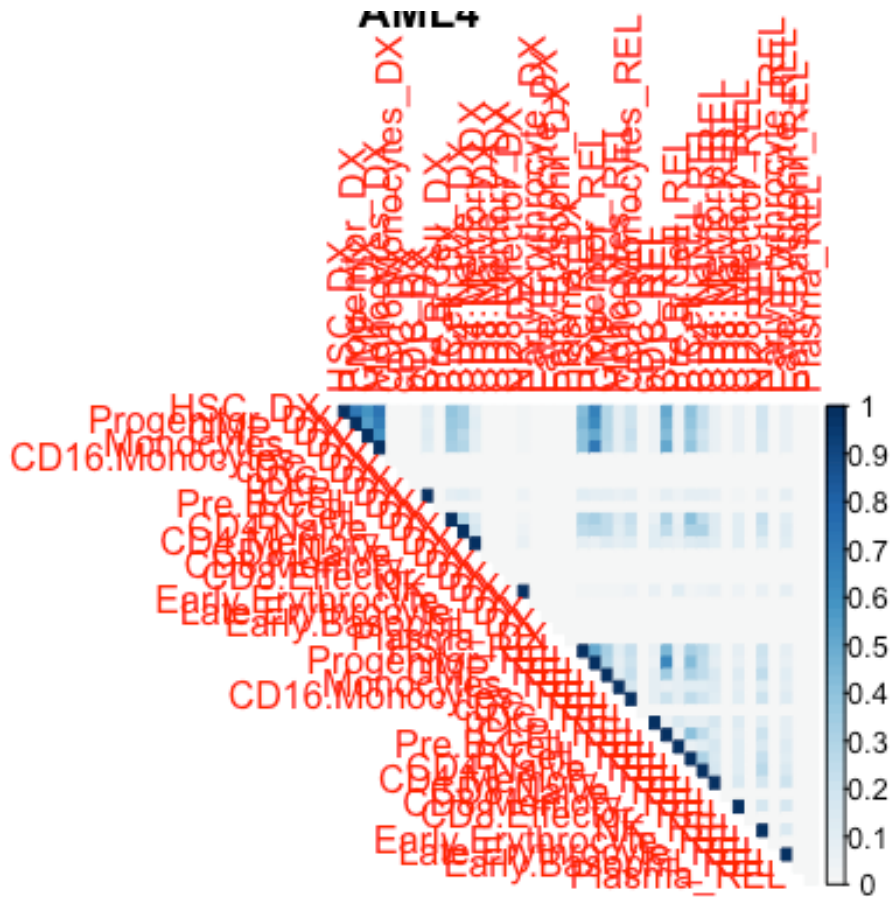


```
corrplot(as.matrix(data_mutationCelltype_similarity_scRNA_byStage_list$AML16),
  order = "original", type = 'lower',
  method = 'color', col = rev(colorRampPalette(brewer.pal(11,
"RdBu"))(100)), col.lim = c(0, 1), title = "AML16")
```

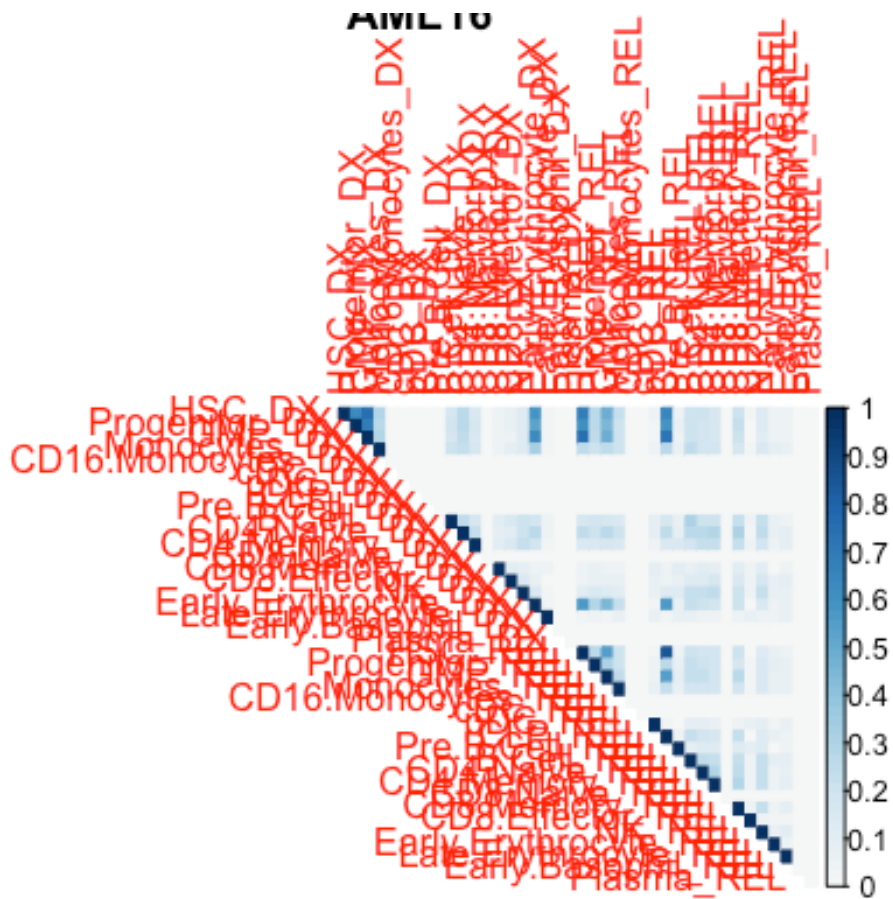
AIVIL 10



```
corrplot(as.matrix(data_mutationCelltype_similarity_scATA_byStage_list$AML4),
order = "original", type = 'upper',
        method = 'color', col = colorRampPalette(brewer.pal(11,
"RdBu"))(100), col.lim = c(0, 1), title = "AML4")
```

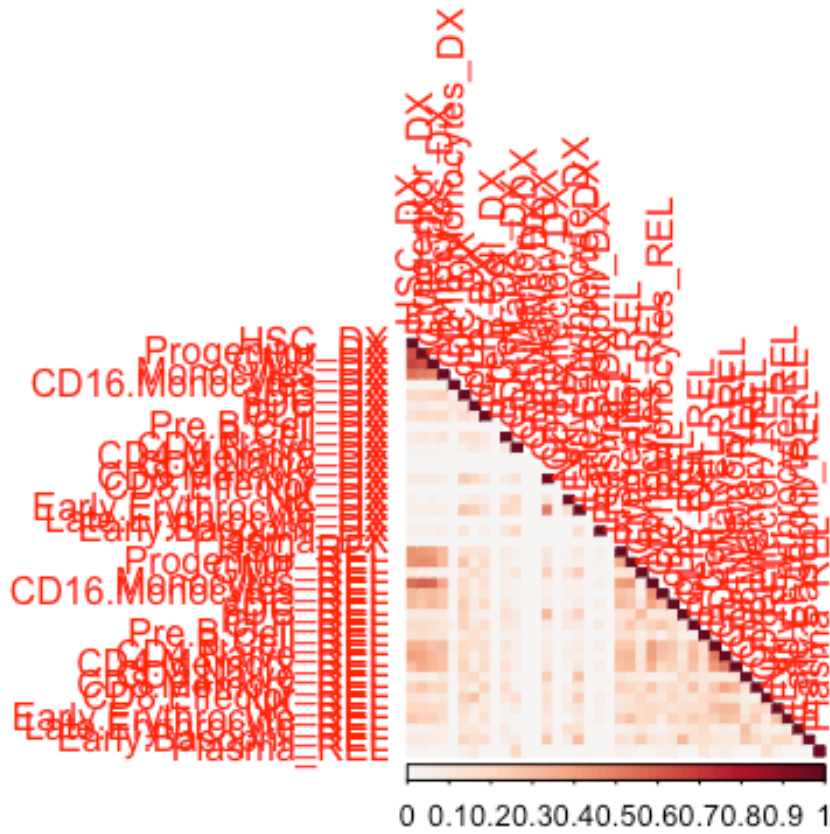


```
corrplot(as.matrix(data_mutationCelltype_similarity_scATA_byStage_list$AML16),
, order = "original", type = 'upper',
      method = 'color', col = colorRampPalette(brewer.pal(11,
"RdBu"))(100), col.lim = c(0, 1), title = "AML16")
```



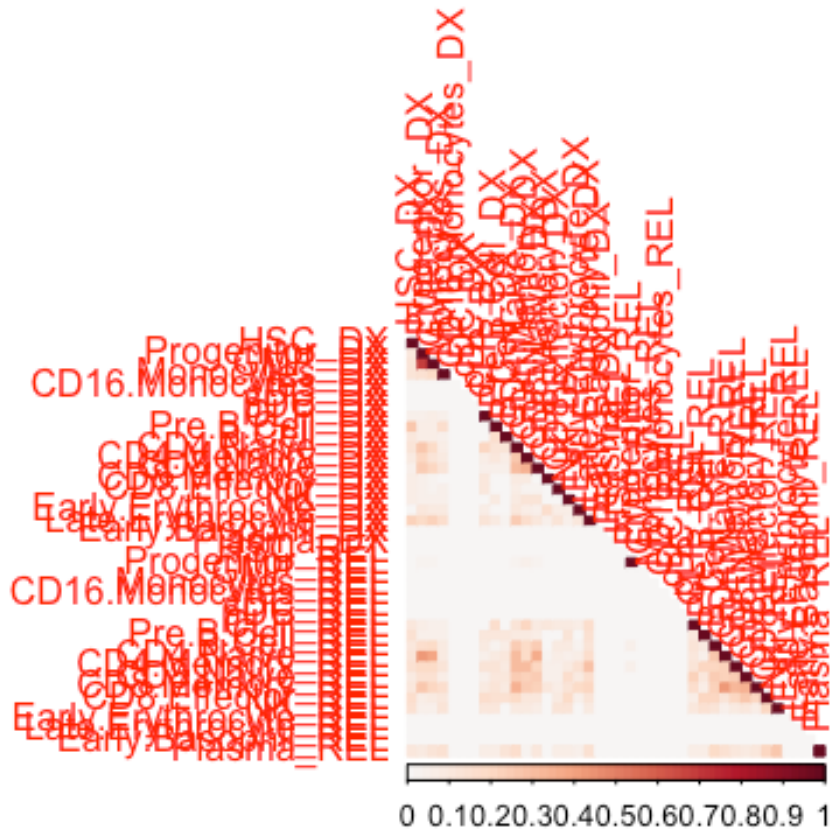
```
corrplot(as.matrix(data_mutationCelltype_similarity_scrNA_byStage_list$AML17)
, order = "original", type = 'lower',
        method = 'color', col = rev(colorRampPalette(brewer.pal(11,
"RdBu"))(100)), col.lim = c(0, 1), title = "AML17")
```

AVALI 11

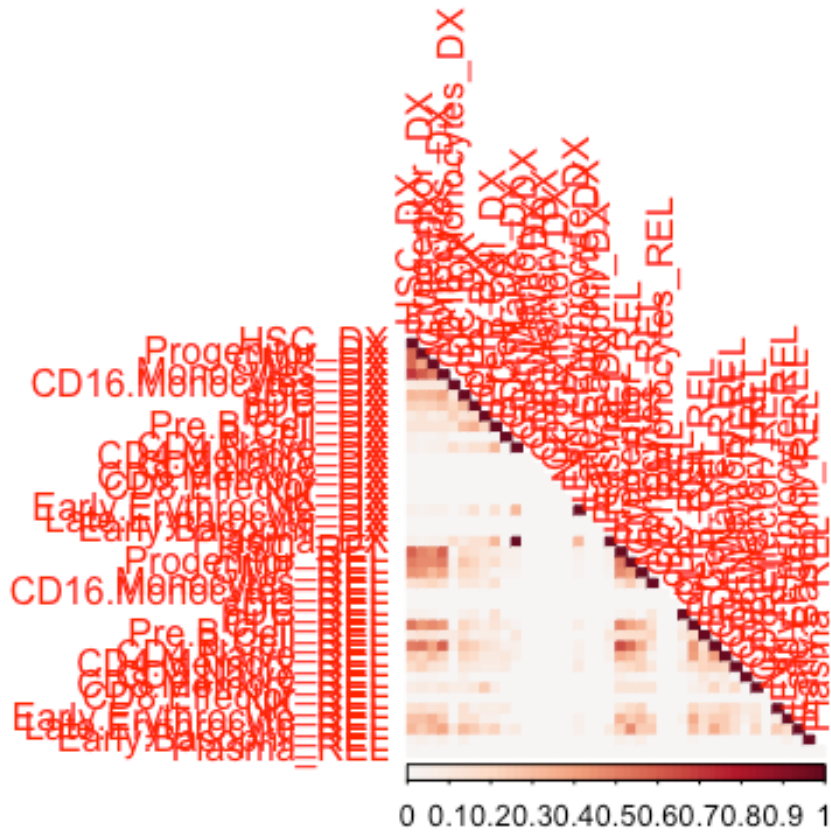


```
corrplot(as.matrix(data_mutationCelltype_similarity_scRNA_byStage_list$AML11),
  order = "original", type = 'lower',
  method = 'color', col = rev(colorRampPalette(brewer.pal(11,
"RdBu"))(100)), col.lim = c(0, 1), title = "AML11")
```

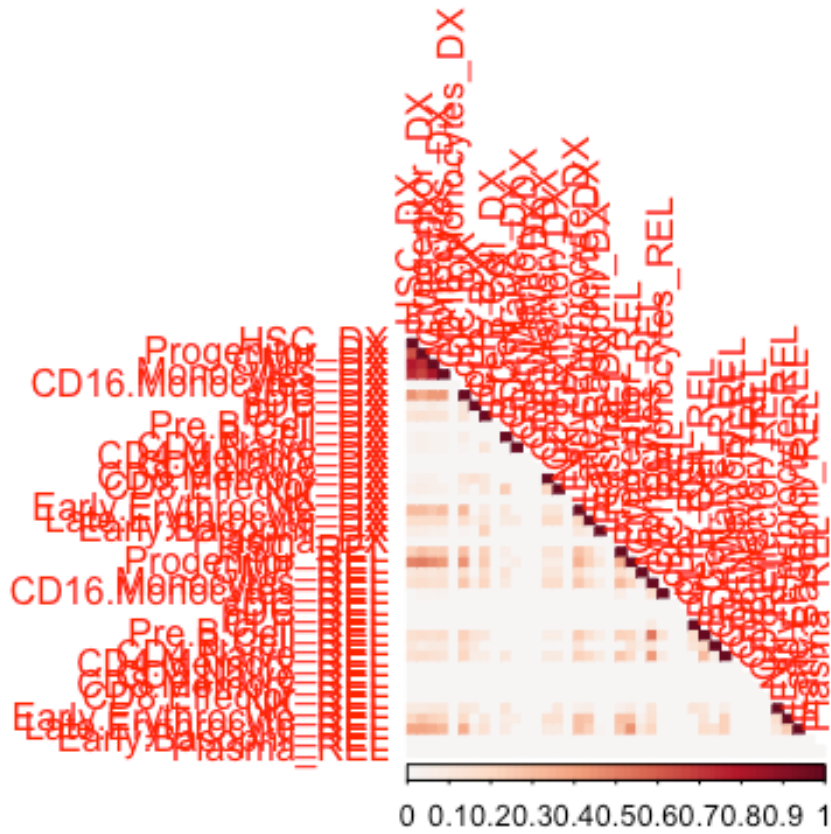

AVIL I



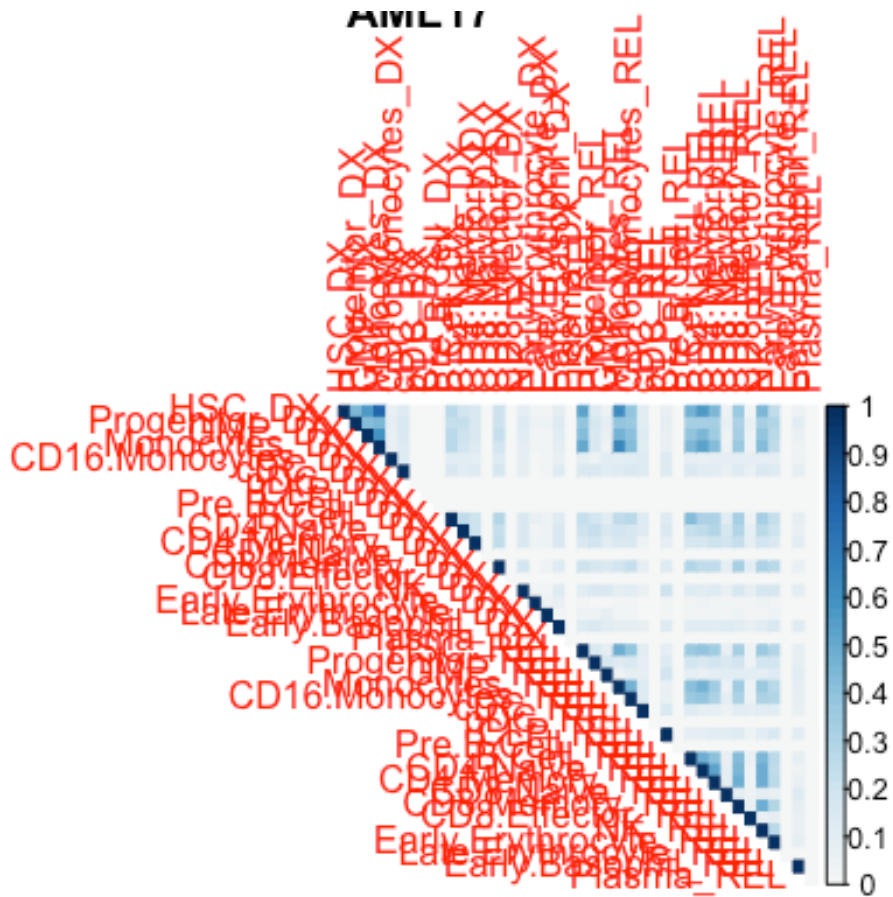
```
corrplot(as.matrix(data_mutationCelltype_similarity_scRNA_byStage_list$AML18),
, order = "original", type = 'lower',
        method = 'color', col = rev(colorRampPalette(brewer.pal(11,
"RdBu"))(100)), col.lim = c(0, 1), title = "AML18")
```

AIVIL 10

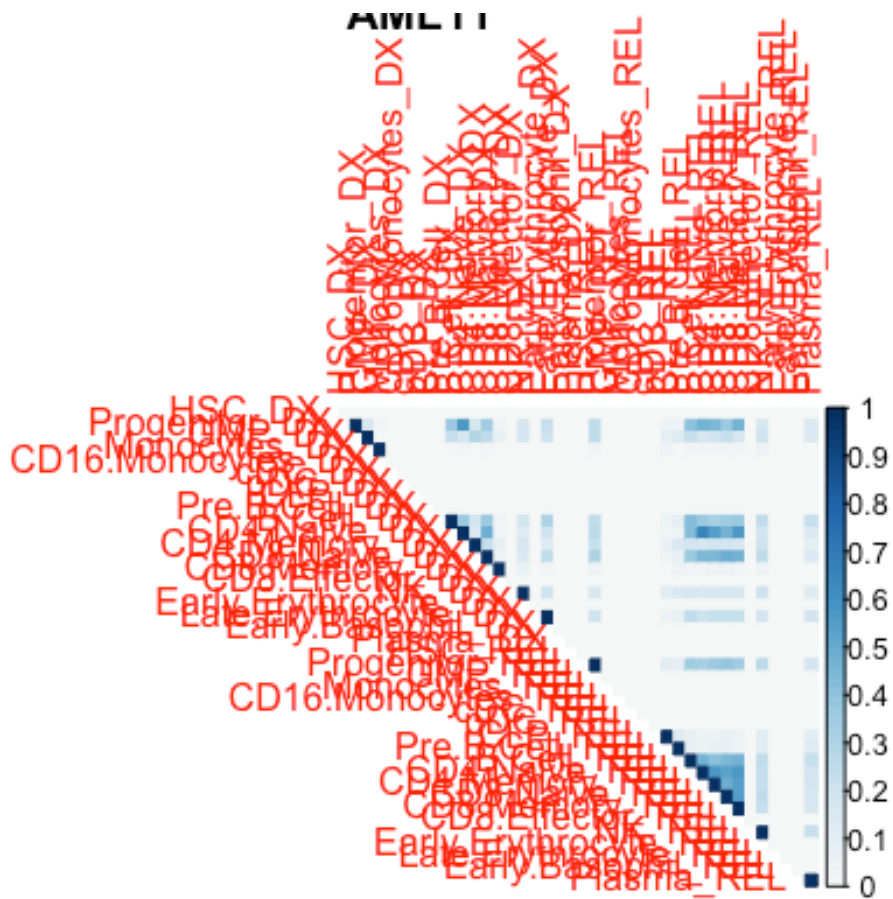
```
corrplot(as.matrix(data_mutationCelltype_similarity_scRNA_byStage_list$AML24),
  order = "original", type = 'lower',
  method = 'color', col = rev(colorRampPalette(brewer.pal(11,
"RdBu"))(100)), col.lim = c(0, 1), title = "AML24")
```

AIVIL24

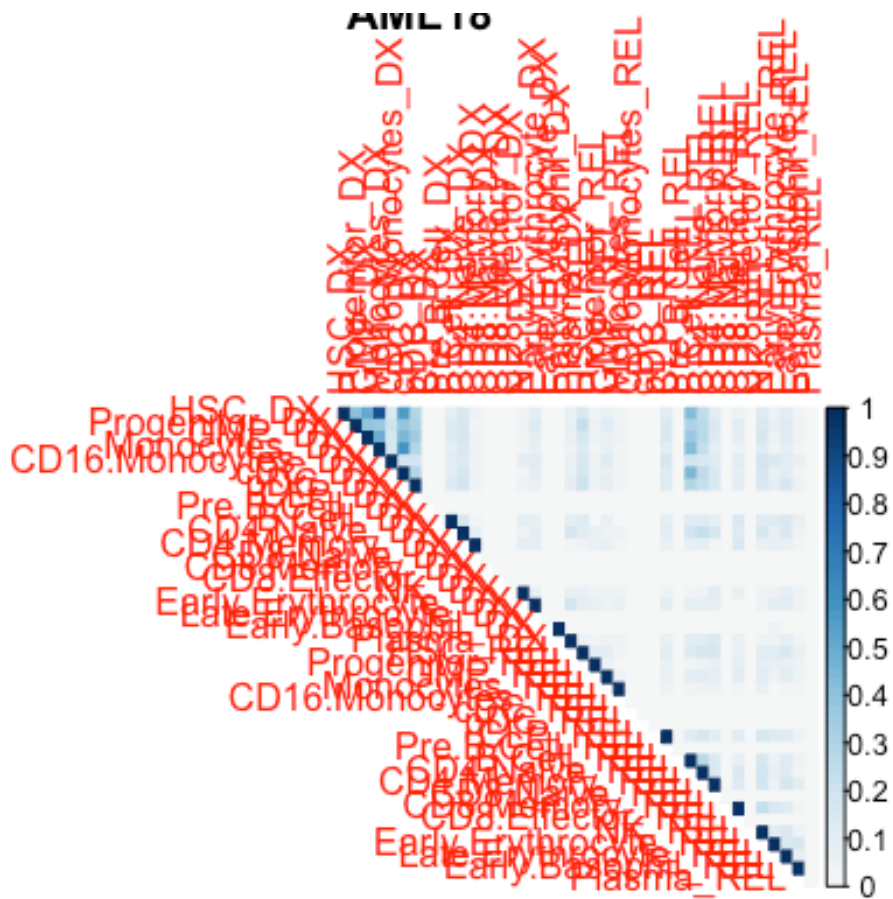
```
corrplot(as.matrix(data_mutationCelltype_similarity_scATA_byStage_list$AML17),
, order = "original", type = 'upper',
        method = 'color', col = colorRampPalette(brewer.pal(11,
"RdBu"))(100), col.lim = c(0, 1), title = "AML17")
```



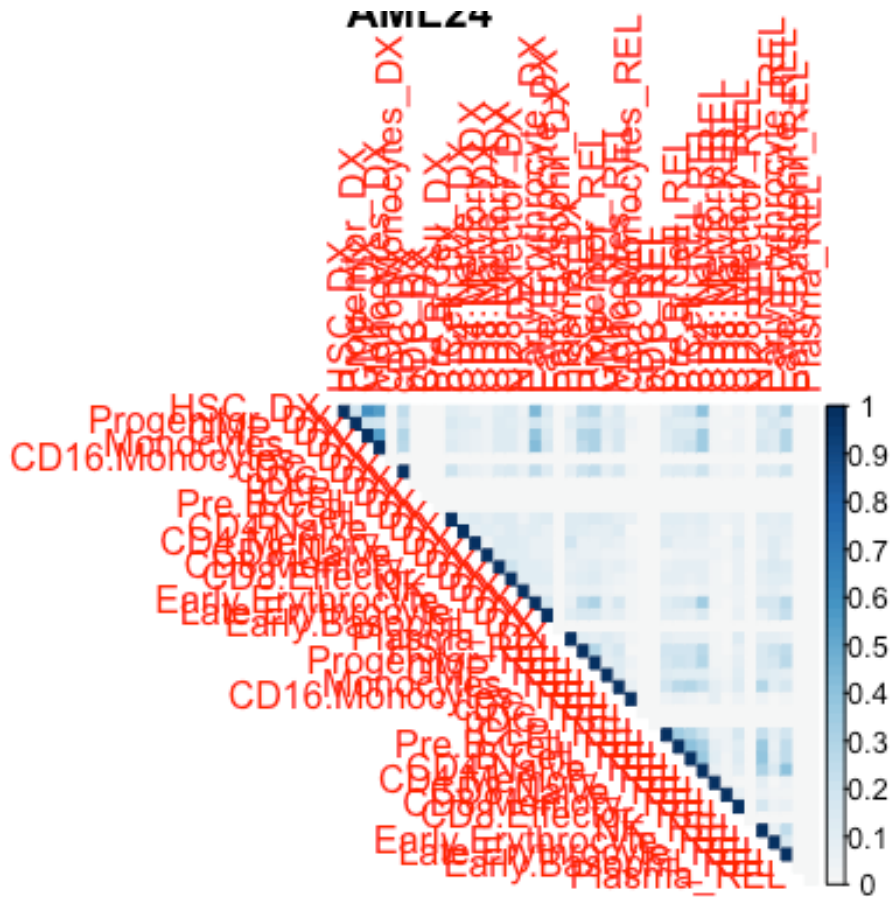
```
corrplot(as.matrix(data_mutationCelltype_similarity_scATA_byStage_list$AML11),
, order = "original", type = 'upper',
      method = 'color', col = colorRampPalette(brewer.pal(11,
"RdBu"))(100), col.lim = c(0, 1), title = "AML11")
```



```
corrplot(as.matrix(data_mutationCelltype_similarity_scATA_byStage_list$AML18),
, order = "original", type = 'upper',
      method = 'color', col = colorRampPalette(brewer.pal(11,
"RdBu"))(100), col.lim = c(0, 1), title = "AML18")
```



```
corrplot(as.matrix(data_mutationCelltype_similarity_scATA_byStage_list$AML24),
, order = "original", type = 'upper',
      method = 'color', col = colorRampPalette(brewer.pal(11,
"RdBu"))(100), col.lim = c(0, 1), title = "AML24")
```



sessionInfo

```
sessionInfo()
## R version 4.5.1 (2025-06-13)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.2
##
## Matrix products: default
## BLAS: /Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/lib/libRlapack.dylib; LAPACK version 3.12.1
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/Stockholm
## tzcode source: internal
##
## attached base packages:
## [1] stats4      grid      stats      graphics  grDevices  utils      datasets
## [8] methods    base
```



```

##
## other attached packages:
## [1] nabor_0.5.0                      htmlwidgets_1.6.4
## [3] networkD3_0.4.1                  clusterProfiler_4.16.0
## [5] ComplexHeatmap_2.24.1            org.Hs.eg.db_3.21.0
## [7] RColorBrewer_1.1-3               ggstankey_0.0.99999
## [9] UpSetR_1.4.0                     patchwork_1.3.2
## [11] lubridate_1.9.4                  forcats_1.0.0
## [13] purrr_1.1.0                      readr_2.1.5
## [15] tibble_3.3.0                     tidyverse_2.0.0
## [17] bedtoolsr_2.30.0-7              corrplot_0.95
## [19] pheatmap_1.0.13                  circlize_0.4.16
## [21] ggpmisc_0.6.2                    ggpp_0.5.9
## [23] Seurat_5.3.0                     SeuratObject_5.2.0
## [25] sp_2.2-0                         ggpubr_0.6.1
## [27] tidyr_1.3.1                      GO.db_3.21.0
## [29] AnnotationDbi_1.70.0            dplyr_1.1.4
## [31] rhdf5_2.52.1                     SummarizedExperiment_1.38.1
## [33] Biobase_2.68.0                  RcppArmadillo_15.0.2-2
## [35] Rcpp_1.1.0                       Matrix_1.7-4
## [37] GenomicRanges_1.60.0            GenomeInfoDb_1.44.3
## [39] IRanges_2.42.0                  S4Vectors_0.46.0
## [41] BiocGenerics_0.54.0             generics_0.1.4
## [43] sparseMatrixStats_1.20.0        MatrixGenerics_1.20.0
## [45] matrixStats_1.5.0               data.table_1.17.8
## [47] stringr_1.5.2                   plyr_1.8.9
## [49] magrittr_2.0.4                  ggplot2_4.0.0
## [51] gtable_0.3.6                    gtools_3.9.5
## [53] gridExtra_2.3                   devtools_2.4.5
## [55] usethis_3.2.1                   ArchR_1.0.3
## [57] vegan_2.7-1                     permute_0.9-8
## [59] ineq_0.2-13                     FSA_0.10.0
##
## Loaded via a namespace (and not attached):
## [1] fs_1.6.6                         spatstat.sparse_3.1-0  enrichplot_1.28.4
## [4] webshot_0.5.5                   httr_1.4.7            doParallel_1.0.17
## [7] profvis_0.4.0                   tools_4.5.1           sctransform_0.4.2
## [10] backports_1.5.0                 R6_2.6.1              lazyeval_0.2.2
## [13] uwot_0.2.3                      mgcv_1.9-3            GetoptLong_1.0.5
## [16] rhdf5filters_1.20.0             urlchecker_1.0.1      withr_3.0.2
## [19] progressr_0.16.0               quantreg_6.1          cli_3.6.5
## [22] Cairo_1.6-5                     spatstat.explore_3.5-3 fastDummies_1.7.5
## [25] labeling_0.4.3                  S7_0.2.0              spatstat.data_3.1-8
## [28] ggribes_0.5.7                   pbapply_1.7-4         yulab.utils_0.2.1
## [31] gson_0.1.0                      R.utils_2.13.0        DOSE_4.2.0
## [34] dichromat_2.0-0.1              parallelly_1.45.1     sessioninfo_1.2.3
## [37] rstudioapi_0.17.1              RSQLite_2.4.3         gridGraphics_0.5-1
## [40] shape_1.4.6.1                  ica_1.0-3             spatstat.random_3.4-
2
## [43] car_3.1-3                       abind_1.4-8           R.methodsS3_1.8.2

```



```

## [46] lifecycle_1.0.4
## [49] qvalue_2.40.0
## [52] blob_1.2.4
## [55] ggtangle_0.0.7
## [58] cowplot_1.2.0
## [61] knitr_1.50
## [64] future.apply_1.20.0
## [67] glue_1.8.0
4
## [70] remotes_2.5.0
## [73] png_0.1-8
## [76] xfun_0.53
## [79] survival_3.8-3
## [82] fitdistrplus_1.2-4
## [85] ggtree_3.16.3
## [88] data.tree_1.2.0
## [91] colorspace_2.1-2
## [94] tidyselect_1.2.1
## [97] SparseM_1.84-2
## [100] scales_1.4.0
## [103] rappdirs_0.3.3
## [106] spatstat.utils_3.2-0
## [109] htmltools_0.5.8.1
## [112] rlang_1.1.6
## [115] shiny_1.11.1
## [118] jsonlite_2.0.0
## [121] GOSemSim_2.34.0
## [124] Formula_1.2-5
## [127] Rhdf5lib_1.30.0
## [130] stringi_1.8.7
## [133] parallel_4.5.1
## [136] deldir_2.0-4
## [139] tensor_1.5.1
## [142] igraph_2.1.4
## [145] RcppHNSW_0.6.0
## [148] evaluate_1.0.5
## [151] httpuv_1.6.16
## [154] polyclip_1.10-7
## [157] scattermore_1.2
## [160] tidytree_0.4.6
## [163] later_1.4.4
## [166] memoise_2.0.1
## [169] globals_0.18.0

yaml_2.3.10
SparseArray_1.8.1
promises_1.3.3
miniUI_0.1.2
KEGGREST_1.48.1
fgsea_1.34.2
codetools_0.2-20
ggfun_0.2.0

treeio_1.32.0
spam_2.11-1
S4Arrays_1.8.1
iterators_1.0.14
ROCR_1.0-11
bit64_4.6.0-1
irlba_2.3.5.1
DBI_1.2.3
bit_4.6.0
DelayedArray_0.34.1
lmtest_0.9-40
digest_0.6.37
rmarkdown_2.29
pkgconfig_2.0.3
GlobalOptions_0.1.2
farver_2.1.2
BiocParallel_1.42.2
ggplotify_0.1.3
GenomeInfoDbData_1.2.14
ape_5.8-1
MASS_7.3-65
listenv_0.9.1
Biostrings_2.76.0
hms_1.1.3
spatstat.geom_3.6-0
reshape2_1.4.4
foreach_1.5.2
MatrixModels_0.5-4
clue_0.3-66
broom_1.0.10
RSpectra_0.16-2
viridisLite_0.4.2
cluster_2.1.8.1

carData_3.0-5
Rtsne_0.17
crayon_1.5.3
lattice_0.22-7
pillar_1.11.1
rjson_0.2.23
fastmatch_1.1-6
spatstat.univar_3.1-4

vctrs_0.6.5
cachem_1.1.0
mime_0.13
ellipsis_0.3.2
nlme_3.1-168
RcppAnnoy_0.0.22
KernSmooth_2.23-26
processx_3.8.6
compiler_4.5.1
plotly_4.11.0
callr_3.7.6
goftest_1.2-3
XVector_0.48.0
fastmap_1.2.0
UCSC.utils_1.4.0
zoo_1.8-14
R.oo_1.27.1
polynom_1.4-1
dotCall64_1.2
reticulate_1.43.0
pkgbuild_1.4.8
ggrepel_0.9.6
splines_4.5.1
ps_1.9.1
ggsignif_0.6.4
pkgload_1.4.0
tzdb_0.5.0
RANN_2.6.2
future_1.67.0
xtable_1.8-4
rstatix_0.7.2
aplot_0.2.9
timechange_0.3.0

```